



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO



FACULTAD DE
INGENIERÍA

Semáforos Inteligentes

Un Enfoque con Aprendizaje por Refuerzo Multiagente

TESINA

Licenciatura en Ciencias de la Computación

AUTOR

Juan Martín Morales

DIRECTORA

Dra. Ana Carolina Olivera

CIUDAD DE MENDOZA, ARGENTINA

2026

Resumen

En este trabajo se aborda el problema de la congestión vehicular en áreas urbanas mediante un sistema de control semafórico basado en el Aprendizaje por refuerzo multi-agente (*Multi-Agent Reinforcement Learning*) (MARL) cooperativo. Se analiza la evolución del parque automotor en Argentina, se describen las limitaciones del control semafórico tradicional y se formula el problema como una tarea de decisión secuencial en la que las intersecciones actúan como agentes. El objetivo es desarrollar y evaluar políticas coordinadas que reduzcan las demoras y mejoren las condiciones de operación de la red. Para ello, se emplean simulaciones microscópicas en escenarios sintéticos y en una red vial que representa el microcentro de la Ciudad de Mendoza, y se comparan algoritmos independientes y cooperativos mediante métricas de tránsito y de aprendizaje.

Abstract

This work addresses urban traffic congestion through a traffic-signal control system based on cooperative MARL. It analyzes the growth of Argentina’s vehicle fleet, describes the limitations of traditional signal control, and formulates the problem as a sequential decision task in which intersections act as agents. The objective is to develop and evaluate coordinated policies that reduce delays and improve network operating conditions. Microscopic simulations are conducted in synthetic scenarios and in an urban road network representing downtown Mendoza City, comparing independent and cooperative algorithms using traffic and learning metrics.

Agradecimientos

Página de agradecimientos

Índice

1	Introducción	1
1.1	Motivación	3
1.2	Organización del Documento	3
2	Marco Teórico	5
2.1	Ingeniería de Tránsito y Simulación	5
2.1.1	Terminología de Tránsito	6
2.1.2	Fundamentos de la Dinámica Vehicular	8
2.1.3	Sistemas de Control de Tránsito	14
2.1.4	Simulación de Tránsito	15
2.2	Aprendizaje por Refuerzo (RL)	22
2.2.1	Procesos de Decisión de Markov (MDP)	23
2.2.2	Métodos Basados en Valor y Basados en Política	25
2.2.3	Aprendizaje Profundo y Redes Neuronales	28
2.2.4	Aprendizaje por Refuerzo Profundo (DRL)	33
2.3	Aprendizaje por Refuerzo Multi-Agente (MARL)	42
2.3.1	Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP	43
2.3.2	Desafíos del Aprendizaje Multi-Agente	45
2.3.3	Paradigmas de Entrenamiento y Ejecución	47
2.3.4	MADRL y líneas de base	48
2.3.5	MAPPO	49
2.3.6	HAPPO	53
2.3.7	Antecedentes recientes del control semafórico multiagente	59
3	Metodología y Diseño Experimental	63
3.1	Enfoque y Diseño de Investigación	63
3.2	Especificación del Ecosistema Tecnológico	67
3.3	Diseño y Calibración de Escenarios de Tránsito en SUMO	68
3.3.1	Modelado Geométrico y Sanitización Topológica	69
3.3.2	Escenarios Sintéticos: 3×1 , 3×3 y $4 \times 4_H$	69

3.3.3	Escenario Urbano Real: Microcentro de la Ciudad de Mendoza . . .	70
3.4	Arquitectura de Integración Entorno-Agentes	76
3.4.1	Ciclo Episódico e Interfaz de Simulación	76
3.5	Instanciación del modelo Dec-POMDP	76
3.5.1	Espacio de Observaciones y Codificación Logarítmica	76
3.5.2	Espacio de Acciones y Enmascaramiento Dinámico	77
3.5.3	Función de Recompensa Compuesta y Coordinación Vecinal	78
3.6	Implementación Algorítmica e Hiperparámetros	79
3.6.1	Mecanismos de Aprendizaje Independiente y Centralizado	79
3.7	Búsqueda de Hiperparámetros (HPO)	80
3.7.1	Infraestructura de Cómputo y Orquestación	82
3.8	Evaluación de Rendimiento	83
4	Análisis y discusión de resultados	84
4.1	Resultados del ajuste de hiperparámetros	84
4.1.1	Diseño del proceso de ajuste	84
4.1.2	Resultados del ajuste y análisis de sensibilidad	84
4.1.3	Configuración seleccionada para la evaluación final	84
4.2	Resultados de los modelos finales	84
4.2.1	Criterios de evaluación y métricas de comparación	85
4.2.2	Desempeño en escenarios sintéticos: 3×1 , 3×3 , 4×4 heterogéneo	85
4.2.3	Desempeño en la red de carreteras de la Ciudad de Mendoza	85
4.3	Discusión e interpretación de resultados	85
5	Conclusiones	86
A	Arquitectura de Software y Patrones de Diseño	87
A.1	Diseño Orientado a Objetos del Entorno de Simulación	87
A.2	Jerarquía de Funciones de Recompensa y Patrón Strategy	89
A.3	Patrones de Diseño Implementados	90

Índice de figuras

1.1	Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.	1
2.1	Movimientos vehiculares, fases y cruces peatonales	7
2.2	Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.	8
2.3	Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).	11
2.4	Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de pelotones. Las curvas representan situaciones conceptuales y no una distribución empírica particular.	13
2.5	Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.	15
2.6	Correspondencia entre la circulación real y su representación en SUMO . .	16
2.7	Interfaz sumo-gui durante una simulación microscópica	17
2.8	Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.	19
2.9	Interacción agente-entorno en un proceso de decisión de Markov [SB18]. . .	23
2.10	Arquitectura actor-crítico aplicada al control semafórico. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].	27
2.11	Relación entre las familias de algoritmos utilizadas en esta investigación. Los cuatro métodos evaluados son libres de modelo y se derivan de las líneas DQN y PPO.	28
2.12	Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación. .	29

2.13	Arquitectura conceptual de una red neuronal <i>feedforward</i> con dos capas ocultas.	31
2.14	Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de <i>bootstrapping</i> del blanco Diferencia temporal (<i>Temporal-Difference</i>) (TD). Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS ⁺ 15].	35
2.15	Red de política para un espacio discreto de acciones. La observación determina una distribución sobre las acciones disponibles; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].	38
2.16	Interacción agente-entorno en MARL cooperativo	44
2.17	Evolución de las políticas de dos agentes bajo WoLF-PHC en Piedra, Papel o Tijera	46
2.18	Paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.	49
2.19	Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m} , que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.	56
3.1	Flujo metodológico de la investigación experimental. Los siete recuadros representan, de arriba hacia abajo, las etapas de modelado de redes, formulación del problema, desarrollo del entorno, integración algorítmica, optimización de hiperparámetros, ejecución distribuida y evaluación estadística; las flechas indican la dependencia secuencial entre ellas.	66
A.1	Diagrama de clases del entorno de simulación y de sus relaciones principales.	88
A.2	Especificación UML del módulo de recompensas.	89

Índice de tablas

2.1	Variables de tránsito macroscópicas y microscópicas	10
2.2	Archivos de entrada y configuración empleados en una simulación de SUMO.	21
2.3	Comparación sintética de familias algorítmicas PPO en entornos multi- agente cooperativos.	59
3.1	Especificación de las tecnologías y herramientas utilizadas en la investigación.	68
3.2	Métricas operacionales de validación del escenario canónico v14 de Mendoza.	75

Siglas

A2C *Advantage Actor-Critic* (método actor–crítico basado en una estimación de ventaja).

AFAC Asociación de Fábricas Argentinas de Componentes.

API Interfaz de programación de aplicaciones (*Application Programming Interface*).

ASHA Algoritmo de reducción sucesiva de configuraciones asíncrono (*Asynchronous Successive Halving Algorithm*).

BFS Búsqueda en anchura (*Breadth-First Search*).

CCAD Centro de Cómputo de Alto Desempeño.

CCAR Centro de Cómputo de Alto Rendimiento.

CI Tasa de completitud de los vehículos insertados (*Completion of Inserted Vehicles*).

CLIP Objetivo sustituto recortado (*Clipped surrogate objective*).

CO Monóxido de carbono.

CO₂ Dióxido de carbono.

CONICET Consejo Nacional de Investigaciones Científicas y Técnicas.

CR Tasa de completitud de viajes (*Completion Rate*).

CTDE Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*).

CVaR Valor en riesgo condicional (*Conditional Value at Risk*).

DAG Grafo acíclico dirigido (*Directed Acyclic Graph*).

DDPG *Deep Deterministic Policy Gradient* (gradiente determinista de política con redes profundas).

Dec-POMDP Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*).

DLR Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*).

DQN *Deep Q-Network* (red neuronal profunda que aproxima valores de acción).

DRL Aprendizaje por refuerzo profundo (*Deep Reinforcement Learning*).

E/S Entrada/salida (*Input/Output*).

FCEFN Facultad de Ciencias Exactas, Físicas y Naturales.

FHWA Administración Federal de Carreteras (*Federal Highway Administration*).

FLIR Tecnología infrarroja con barrido hacia adelante (*Forward Looking Infrared*).

GAE *Generalized Advantage Estimation* (estimación de la ventaja mediante errores temporales ponderados).

GEH Estadístico empírico de comparación de flujos viales de Geoffrey E. Havers.

GUI Interfaz gráfica de usuario (*Graphical User Interface*).

HAPPO *Heterogeneous-Agent Proximal Policy Optimization* (PPO con actualización secuencial de agentes heterogéneos).

HARL Aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*).

HATRPO *Heterogeneous-Agent Trust Region Policy Optimization* (optimización con región de confianza para agentes heterogéneos).

HBEFA Manual de factores de emisión para transporte por carretera (*Handbook Emission Factors for Road Transport*).

HC Hidrocarburos no combustionados.

HPC Computación de alto rendimiento (*High-Performance Computing*).

HPO Optimización de hiperparámetros (*Hyperparameter Optimization*).

IA Inteligencia artificial (*Artificial Intelligence*).

IDQN *Independent Deep Q-Network* (DQN independiente por agente).

IL Aprendizaje independiente (*Independent Learning*).

IPPO *Independent Proximal Policy Optimization* (PPO independiente por agente).

IQM Media intercuartílica (*Interquartile Mean*).

IR Tasa de inserción (*Insertion Rate*).

ITE Instituto de Ingenieros de Transporte (*Institute of Transportation Engineers*).

KL Divergencia de Kullback–Leibler.

MADDPG *Multi-Agent Deep Deterministic Policy Gradient* (extensión multiagente de DDPG).

MADRL Aprendizaje por refuerzo profundo multi-agente (*Multi-Agent Deep Reinforcement Learning*).

MAPPO *Multi-Agent Proximal Policy Optimization* (PPO multiagente con información centralizada durante el entrenamiento).

MARL Aprendizaje por refuerzo multi-agente (*Multi-Agent Reinforcement Learning*).

MDP Proceso de decisión de Markov (*Markov Decision Process*).

ML Aprendizaje automático (*Machine Learning*).

MLP Perceptrón multicapa (*Multi-Layer Perceptron*).

NFS Sistema de archivos de red (*Network File System*).

NOx Óxidos de nitrógeno.

OD Origen-destino (*Origin-Destination*).

OOD Fuera de distribución (*Out-of-Distribution*).

OSM OpenStreetMap.

PHEMlight Modelo microscópico de emisiones de vehículos livianos y pesados (*Passenger car and Heavy duty Emission Model light*).

POMDP Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*).

POSIX Interfaz portable de sistemas operativos (*Portable Operating System Interface*).

PPO *Proximal Policy Optimization* (optimización mediante un objetivo sustituto recordado).

QMIX *Q-Mixing Network* (red que combina valores de acción individuales).

ReLU Unidad lineal rectificadora (*Rectified Linear Unit*).

RL Aprendizaje por refuerzo (*Reinforcement Learning*).

SGD Descenso de gradiente estocástico (*Stochastic Gradient Descent*).

SL Aprendizaje supervisado (*Supervised Learning*).

SLURM Gestor simple de recursos para administración de Linux (*Simple Linux Utility for Resource Management*).

SUMO Simulación de Movilidad Urbana (*Simulation of Urban MObility*).

TAZ Zona de análisis de tránsito (*Traffic Analysis Zone*).

TCP Protocolo de control de transmisión (*Transmission Control Protocol*).

TD Diferencia temporal (*Temporal-Difference*).

TPE Estimador de densidad de Parzen estructurado por árbol (*Tree-structured Parzen Estimator*).

TraCI Interfaz de control del tránsito (*Traffic Control Interface*).

TRPO *Trust Region Policy Optimization* (optimización con una restricción de región de confianza).

UL Aprendizaje no supervisado (*Unsupervised Learning*).

UML Lenguaje unificado de modelado (*Unified Modeling Language*).

UTM Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*).

VDN *Value-Decomposition Networks* (redes de descomposición del valor conjunto).

VF Función de valor (*Value Function*).

WGS84 Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*).

WoLF-PHC *Win or Learn Fast Policy Hill-Climbing* (PHC con tasas distintas según el desempeño).

XML Lenguaje de marcado extensible (*Extensible Markup Language*).

Glosario

- G_t (**retorno**) Suma acumulada, usualmente descontada, de las recompensas futuras a partir del instante t .
- $Q(s, a)$ (**valor de acción**) Retorno esperado de ejecutar la acción a en el estado s y continuar posteriormente de acuerdo con una política determinada.
- $S[\pi_\theta]$ (**entropía de la política**) Medida de la dispersión de la distribución de acciones de la política; su inclusión en el objetivo favorece la exploración.
- $V(s)$ (**valor de estado**) Retorno esperado al comenzar en el estado s y seguir una política determinada, promediando las acciones que dicha política selecciona.
- V_t^{target} (**valor objetivo**) Estimación de retorno utilizada como referencia para ajustar la predicción de valor del crítico en el instante t .
- α (**tasa de aprendizaje**) Paso de actualización que regula cuánto se modifican las estimaciones de valor o los parámetros en cada iteración de aprendizaje.
- δ_t (**error de diferencia temporal**) Diferencia entre la recompensa más el valor descontado del estado siguiente y la estimación de valor del estado actual.
- ϵ_{clip} (**margen de recorte**) Hiperparámetro de PPO que delimita el intervalo utilizado para recortar la razón entre la probabilidad nueva y la anterior de una acción.
- ϵ_{exp} (**exploración**) Probabilidad o coeficiente que controla la selección de acciones exploratorias frente a las acciones favorecidas por la política o la función de valor.
- η (**tasa del optimizador**) Tamaño de paso empleado por el optimizador en la actualización de parámetros mediante descenso de gradiente.
- γ (**factor de descuento**) Parámetro en el intervalo $[0, 1)$ que pondera la contribución de las recompensas futuras al retorno esperado.
- $\hat{A}_t$ (**ventaja estimada**) Estimación de cuánto mejor o peor fue la acción observada en el paso t respecto del valor esperado bajo la política vigente.

- λ (**suavizado de la ventaja**) Hiperparámetro que pondera errores de diferencia temporal de distintos horizontes al estimar la ventaja y equilibra sesgo y varianza.
- $\mathcal{L}(\theta)$ (**función de pérdida**) Función escalar que cuantifica el error del modelo y que se minimiza para ajustar los parámetros θ .
- ϕ (**parámetros del crítico**) Vector de parámetros entrenables de la red que estima la función de valor del crítico.
- π_θ (**política parametrizada**) Distribución de probabilidad sobre las acciones condicionada por el estado u observación y controlada por el vector de parámetros entrenables θ .
- θ (**parámetros de la política**) Vector de parámetros entrenables que determina la distribución de acciones de la política del actor.
- a_i (**acción**) Decisión ejecutada por el agente i en un paso temporal. En el control semaforico representa la fase o la modificación de fase seleccionada por el controlador.
- c_1 (**coeficiente de valor**) Peso asignado al término de pérdida de la función de valor en el objetivo combinado de PPO.
- c_2 (**coeficiente de entropía**) Peso asignado al término de entropía que incentiva la exploración en el objetivo combinado de PPO.
- o_i (**observación local**) Información disponible para el agente i durante la ejecución descentralizada. Se obtiene a partir de los sensores o variables observables de la intersección que controla.
- r_t (**recompensa**) Señal escalar recibida en el paso temporal t después de ejecutar una acción. Se utiliza para orientar la actualización de la política o de la función de valor.
- s (**estado global**) Representación conjunta de las variables relevantes del entorno en un instante determinado. En el modelo multi-agente puede incluir la información de todas las intersecciones y sus corrientes vehiculares.
- binding* Mecanismo que conecta una biblioteca escrita en un lenguaje de programación con otro lenguaje para poder utilizar sus funciones.
- bootstrap* Procedimiento de remuestreo con reemplazo utilizado para aproximar la incertidumbre de un estimador a partir de las observaciones disponibles.
- epoch de entrenamiento* Recorrido de optimización sobre el conjunto de datos recolectado para una iteración; una iteración puede reutilizar ese conjunto durante varios *epochs*.

logit Puntuación real sin normalizar que una red de política produce para una acción antes de convertirla en probabilidad.

minibatch Subconjunto de las muestras disponibles utilizado para calcular una actualización de los parámetros del modelo.

replay buffer Memoria que conserva transiciones de experiencias anteriores para volver a muestrearlas durante el entrenamiento de un método basado en valor.

rollout Secuencia ordenada de transiciones obtenida mientras uno o más agentes interactúan con el entorno bajo una política determinada.

softmax Función que transforma un conjunto de puntuaciones reales en probabilidades no negativas cuya suma es uno.

teleport Mecanismo específico de SUMO que retira temporalmente un vehículo bloqueado y lo reinserta posteriormente para evitar que la simulación quede detenida.

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semaforizadas programadas en una intersección particular.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Episodio Secuencia de interacción que comienza al reiniciar el entorno y termina cuando se alcanza una condición terminal o el horizonte máximo definido.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso a uno o más movimientos o corrientes vehiculares compatibles entre sí para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Iteración de entrenamiento Ciclo completo en el que se recolectan datos con una política y se realizan una o más actualizaciones de sus parámetros.

marco de trabajo (*framework*) Conjunto reutilizable de interfaces, componentes y reglas que facilita la construcción y la integración de un sistema de software.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

serialización Conversión de un objeto o mensaje a una representación que puede almacenarse o transmitirse y luego reconstruirse.

telemetría Conjunto de mediciones que el simulador registra durante la ejecución, como velocidades, posiciones, detenciones y tiempos de espera.

Transición Registro de un paso de interacción que contiene el estado u observación actual, la acción ejecutada, la recompensa recibida, el estado siguiente y, cuando corresponde, un indicador de terminación.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

Introducción

En las últimas décadas, el crecimiento del parque automotor y la urbanización sostenida han convertido a la congestión vehicular en uno de los principales desafíos para la movilidad urbana. Según los reportes de flota vehicular de Argentina elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y tomando 2012 como línea base, el volumen de vehículos aumentó un 21,7% en 2017. Esta tendencia continuó durante la última década y alcanzó un incremento acumulado del 44,4% en 2025 (véase la Figura 1.1) [Aso18, Aso26].

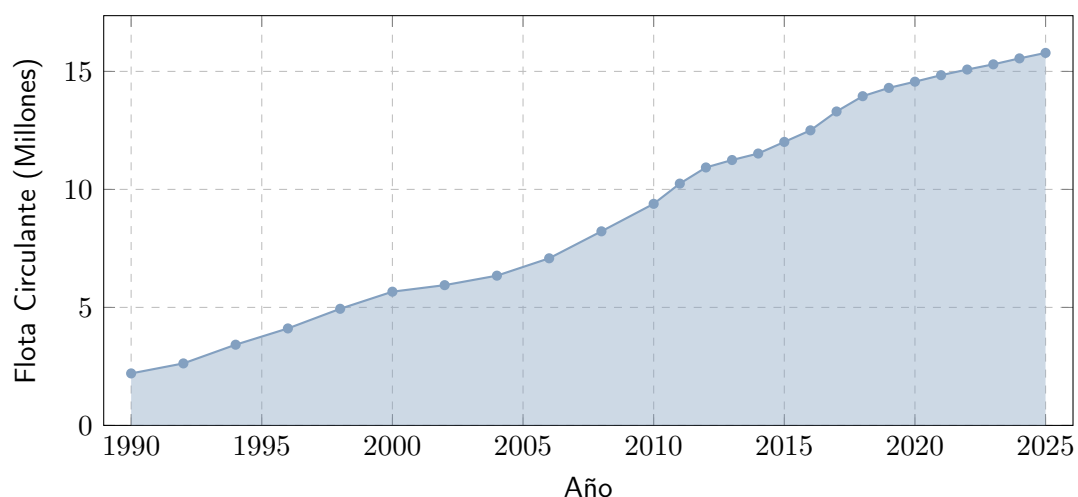


Figura 1.1: Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.

La Figura 1.1 debe leerse de izquierda a derecha: el eje horizontal indica el año y el vertical, la cantidad estimada de vehículos en millones. La línea azul une las observaciones y los puntos identifican cada año informado; el sombreado no representa otra variable, sino el área bajo la misma serie. La tendencia general es creciente: pasa de aproximadamente 2,2 millones de vehículos en 1990 a 10,93 millones en 2012, 13,30 millones en 2017 y 15,78 millones en 2025. En los reportes de AFAC, la flota circulante comprende automóviles y vehículos comerciales livianos y pesados en condiciones de circular; excluye acoplados y otros remolques. Por tanto, la figura representa una estimación del parque activo bajo esa definición, no el total histórico de unidades patentadas ni la cantidad de viajes realizados [Aso18, Aso26].

Las congestiones vehiculares deterioran las condiciones de operación de la red e incrementan las demoras y las detenciones de los vehículos [CyMR18]. En el modelado microscópico, las emisiones de gases como el monóxido de carbono (CO), el dióxido de carbono (CO₂), los óxidos de nitrógeno (NO_x) y los hidrocarburos no combustionados (HC) se estiman a partir de variables cinemáticas, entre ellas la velocidad, la aceleración y el tiempo en ralentí [DLR26]. Este problema es especialmente relevante en ciudades con alta densidad vehicular. En este trabajo se utilizan escenarios sintéticos y un escenario urbano restringido por datos: el microcentro de la Ciudad de Mendoza, delimitado por las avenidas San Martín, Colón, Belgrano y Las Heras.

Frente a la congestión, una alternativa que no requiere modificaciones importantes de la infraestructura consiste en optimizar la gestión del tránsito mediante sistemas de control semafórico adaptativos. Los enfoques tradicionales pueden presentar limitaciones para responder a variaciones en el flujo vehicular [PW16]. En este contexto, los avances en Inteligencia artificial (*Artificial Intelligence*) (IA) y en particular el Aprendizaje por refuerzo (*Reinforcement Learning*) (RL) permiten modelar los semáforos como agentes que aprenden políticas de control a partir de la interacción con el entorno. Las acciones corresponden a cambios de fase y la recompensa puede definirse, por ejemplo, como el negativo del tiempo de espera acumulado. Diversos estudios han mostrado la viabilidad de este enfoque, desde algoritmos tabulares aplicados a una sola intersección [Wie00] hasta modelos de redes coordinadas basados en redes neuronales profundas y control multi-agente [W⁺19].

Sin embargo, la aplicación simultánea de este enfoque en múltiples intersecciones introduce desafíos adicionales. Un esquema centralizado puede resultar poco escalable, pues el espacio de acciones conjunto crece multiplicativamente con el número de intersecciones [ACS24]. En una formulación de MARL con Aprendizaje independiente (*Independent Learning*) (IL), cada semáforo optimiza su propia intersección sin considerar explícitamente el efecto de sus decisiones sobre el resto de la red [AS22]. Además, como todos los agentes aprenden y modifican sus políticas al mismo tiempo, cada uno observa un entorno no estacionario. Esto dificulta la convergencia de los algoritmos de agente único

[FNF⁺17].

Por esta razón, este trabajo aborda el problema desde el marco del MARL cooperativo. En este paradigma, los controladores actúan de manera coordinada y buscan reducir una señal de recompensa compartida, orientada a disminuir el tiempo de espera promedio de la red. La recompensa es un valor numérico que representa el efecto de la acción conjunta sobre el desempeño del sistema.

En este contexto, el objetivo general fue desarrollar y evaluar un sistema de control semafórico basado en el enfoque cooperativo para mejorar la operación del tránsito urbano. Para ello, se estudió el comportamiento de distintos algoritmos MARL en entornos no estacionarios mediante simulaciones en escenarios sintéticos y en la red vial modelada del microcentro de la Ciudad de Mendoza. La comparación permite evaluar el desempeño de una arquitectura descentralizada y coordinada frente a los enfoques de control tradicionales y a propuestas recientes de la literatura.

1.1 Motivación

La motivación principal de este trabajo radica en la marcada asimetría entre una demanda vehicular en constante crecimiento y una infraestructura vial que permanece estática. Dado que ensanchar avenidas o modificar físicamente el trazado urbano en un sector denso como el microcentro de la Ciudad de Mendoza es económica y estructuralmente inviable, la alternativa más sostenible es la optimización lógica de los recursos existentes.

El desarrollo de soluciones basadas en software permite dotar de adaptabilidad a la red semafórica actual. De este modo, un sistema de control adaptativo puede ajustar sus decisiones a las condiciones observadas y contribuir a reducir las demoras mediante una gestión más eficiente de la infraestructura existente [MMLK25].

1.2 Organización del Documento

El presente trabajo se estructura de la siguiente manera:

- **Capítulo 1 – Introducción:** contextualiza la problemática de la congestión vehicular urbana, define los objetivos del proyecto y expone las motivaciones que impulsan esta investigación.
- **Capítulo 2 – Marco Teórico:** formaliza los conceptos fundamentales de la ingeniería de tránsito, la teoría de procesos de decisión de Markov, las redes neuronales profundas y los paradigmas de aprendizaje por refuerzo multi-agente.
- **Capítulo 3 – Metodología:** detalla el diseño metodológico y experimental, la calibración de la demanda vial mediante el estadístico Estadístico empírico de comparación de flujos viales de Geoffrey E. Havers (GEH), el modelado del microcentro de la Ciudad de Mendoza en Simulación de Movilidad Urbana (*Simulation of Urban MObility*) (SUMO) y la formulación algorítmica de los controladores.

- **Capítulo 4 – Resultados:** presenta la evaluación empírica y comparativa de los algoritmos de control sobre redes sintéticas y el escenario urbano restringido por datos.
- **Capítulo 5 – Conclusiones:** sintetiza los hallazgos principales, discute las limitaciones del modelo y propone direcciones de investigación futuras.
- **Apéndice A – Arquitectura de Software:** describe las especificaciones orientadas a objetos, los diagramas de clases Lenguaje unificado de modelado (*Unified Modeling Language*) (UML) y los patrones de diseño aplicados en el marco computacional `marlTLC`.

2

Marco Teórico

Este capítulo presenta los conceptos necesarios para plantear el control semafórico como un problema de decisión secuencial. Primero se describen las variables y los elementos de la operación del tránsito; luego se introduce su representación en un simulador microscópico; finalmente se presentan los fundamentos del RL y del MARL, que permiten formular políticas de control para la red vial.

2.1 Ingeniería de Tránsito y Simulación

La ingeniería de tránsito es una rama de la ingeniería de transporte, tradicionalmente vinculada con la ingeniería civil. Se ocupa de la planificación, el diseño geométrico y la operación segura y eficiente del tránsito en calles, carreteras y redes viales, de acuerdo con manuales especializados como el *Traffic Engineering Handbook* [PW16] y con la literatura clásica del área [CyMR18]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa. Permiten describir el funcionamiento físico de la red, caracterizar la oferta y la demanda de circulación e identificar las variables sobre las que puede intervenir el sistema de control.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en proveer y ampliar la capacidad física vial mediante la construcción de nuevas obras o el ensanche de calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y geoméricamente inviable [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito,

priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

Estas magnitudes cumplen una función doble en el modelo computacional: pueden formar parte de las observaciones y de la señal de recompensa de los agentes, y también se utilizan para evaluar el desempeño del controlador. Su definición permite vincular la descripción física del tránsito con la formulación posterior del problema de aprendizaje.

2.1.1 Terminología de Tránsito

Antes de formular el problema de control, es necesario distinguir algunos términos operativos que se utilizarán a lo largo del capítulo. En particular, se diferencian los movimientos que realizan los vehículos, las fases que habilitan esos movimientos y los intervalos durante los cuales se mantienen las indicaciones semafóricas [Fed08].

En este contexto, se emplearán las siguientes nociones:

- **Intersección (*intersection*), acceso (*approach*) y carril (*lane*):** una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].
- **Movimiento vehicular y movimientos en conflicto:** un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una salida determinada, por ejemplo, un giro o un movimiento directo [W⁺19]. Dos movimientos pueden generar puntos de conflicto cuando sus trayectorias se intersectan (véase la Figura 2.2). Cuando la geometría y la señalización no permiten proteger su operación simultánea, esos movimientos deben programarse en intervalos distintos por razones de seguridad [FA11].
- **Fase (*phase*), ciclo (*cycle*) e intervalo (*interval*):** una fase es una configuración de señales que habilita simultáneamente un conjunto de movimientos compatibles [PW16]. El ciclo corresponde a la secuencia completa de fases necesaria para atender

los movimientos de la intersección. Un intervalo es el período durante el cual las indicaciones semafóricas permanecen constantes [CyMR18].

La Figura 2.1 ilustra estos conceptos en una intersección de cuatro accesos. Los números del 1 al 8 identifican movimientos vehiculares individuales y las etiquetas $P2$, $P4$, $P6$ y $P8$ identifican cruces peatonales del esquema. Esta numeración sigue de manera esquemática una convención habitual de los manuales de temporización semafórica; no representa una configuración experimental ni una asignación obligatoria de Fases (Ingeniería de Tránsito) para el caso de estudio. Las flechas, los accesos y las etiquetas de calles permiten distinguir trayectorias y sentidos, mientras que los cruces peatonales muestran otros usuarios que también deben considerarse al ordenar los movimientos. En este trabajo, la figura se utiliza para separar la noción geométrica de movimiento de la regla semafórica que habilita un conjunto de movimientos compatibles [Fed08]. Por ejemplo, dos movimientos que no comparten un punto de conflicto pueden incluirse en una misma fase, mientras que los que sí lo comparten deben atenderse en fases compatibles o sucesivas.

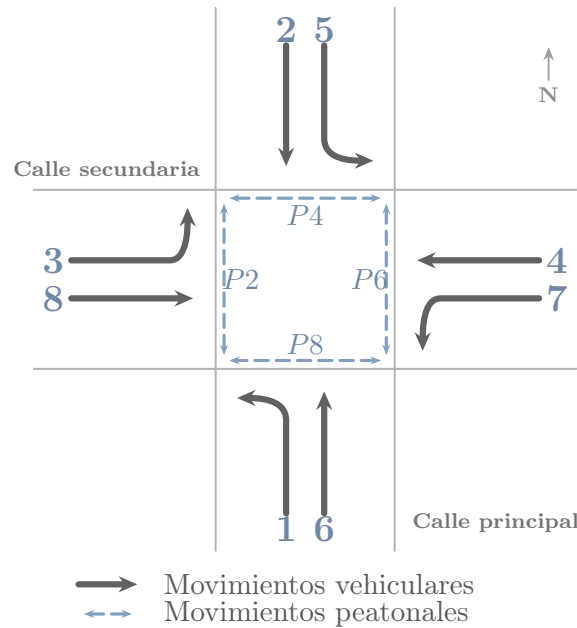


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de Administración Federal de Carreteras (*Federal Highway Administration*) (FHWA) [Fed08].

La Figura 2.2 presenta tres formas esquemáticas de relación entre trayectorias. Hay *cruce* cuando dos trayectorias se intersectan en un punto; *convergencia* cuando trayectorias distintas se unen en una misma corriente; y *divergencia* cuando una corriente se separa en trayectorias diferentes. Los puntos coloreados marcan el lugar de interacción que se analiza en cada caso. Cada relación puede originar un punto de conflicto según la

geometría y la simultaneidad de los movimientos: por eso, la incompatibilidad no depende solo de que las líneas se relacionen, sino de que su operación conjunta comprometa la seguridad. En el modelo semafórico, estos casos se utilizan para determinar qué movimientos pueden habilitarse en una misma Fase (Ingeniería de Tránsito) y cuáles deben retenerse; la figura muestra relaciones geométricas, no una programación temporal concreta.

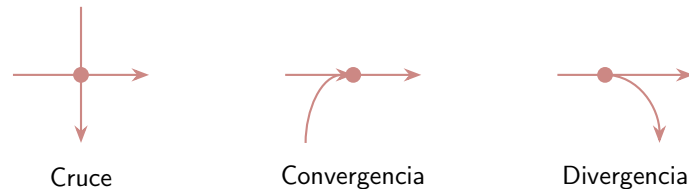


Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16]. Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí, como resume la Figura 2.2. Por ello, su operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2 Fundamentos de la Dinámica Vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flujo vehicular, y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópi-

co, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la Tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son:

- **Flujo o tasa de flujo (q):** representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora (veh/h), calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas [CyMR18].
- **Densidad o concentración (k):** indica el número de vehículos que ocupan simultáneamente una longitud específica de vía, expresada en vehículos por kilómetro (veh/km), calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado [CyMR18]. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].
- **Velocidad media espacial (v_s):** es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial [CyMR18].
- **Velocidad media temporal (v_t):** representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado [CyMR18].

En el nivel microscópico, las variables asociadas describen el movimiento del vehículo individual y la separación entre vehículos consecutivos (véase la Figura 2.3):

- **Velocidad puntual (v_i):** es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.
- **Intervalo temporal (h):** es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos, por ejemplo, los parachoques delanteros [CyMR18]. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.
- **Espaciamiento simple (s):** es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo [CyMR18]. El espaciamiento prome-

dio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

La Figura 2.3 complementa estas definiciones con las magnitudes que se miden en el espacio y en el tiempo. El largo del vehículo l y la separación libre e son distancias, expresadas en metros (m); el espaciamiento s es la distancia entre puntos homólogos de vehículos consecutivos, también en m. El avance temporal β es el tiempo asociado al recorrido del largo del vehículo a la velocidad v , la brecha temporal g es el tiempo entre el paso del vehículo precedente y el siguiente respecto de puntos de referencia distintos, y el intervalo temporal h es el tiempo entre pasos de puntos homólogos; β , g y h se expresan en segundos (s). Las cotas inferiores representan relaciones espaciales y las superiores relaciones temporales; no son dos vehículos distintos ni dos mediciones independientes. En este trabajo, estas magnitudes sirven para vincular la descripción individual de los vehículos con las variables agregadas del flujo.

Tabla 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	Variable Principal	Relación / Conversión
Macroscópica	Flujo (q , veh/h)	$q = 3600/h_{\text{prom}}$
	Densidad (k , veh/km)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s , km/h)	$q = k \cdot v_s$
	Velocidad media temporal (v_t , km/h)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i , km/h)	Variable individual de cada vehículo
	Intervalo temporal (h , s)	Tiempo entre pasos de vehículos sucesivos
	Espaciamiento simple (s , m)	Distancia espacial entre parachoques homólogos

La Figura 2.3 presenta de manera conjunta estas distancias y tiempos entre vehículos, y permite interpretar cómo las magnitudes microscópicas describen la separación y el avance de una corriente.

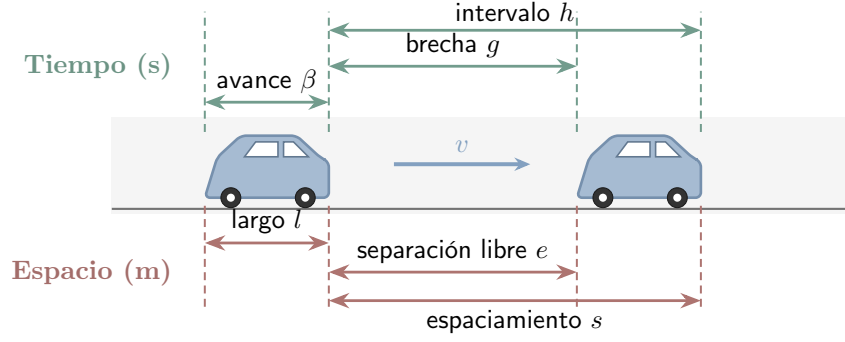


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).

Estas variables macroscópicas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \quad (2.1)$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. La ecuación (2.1) utiliza la velocidad media espacial, ya que relaciona el flujo y la cantidad de vehículos presentes en un tramo. La velocidad media temporal (v_t) es la media aritmética de las velocidades observadas en un punto durante un intervalo, mientras que v_s es la media armónica de las velocidades observadas en un tramo. Cuando existe variabilidad entre las velocidades, ambas magnitudes pueden diferir. La relación entre ellas se expresa mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \quad (2.2)$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. La ecuación (2.2) muestra que, dado que la varianza es no negativa ($\sigma_s^2 \geq 0$), se cumple $v_t \geq v_s$, con igualdad únicamente cuando todos los vehículos circulan a la misma velocidad. En una intersección semaforizada, una indicación roja puede reducir localmente la velocidad media y aumentar la acumulación de vehículos en los accesos. Estos efectos se manifiestan en la formación de colas y en el incremento de las demoras [CyMR18].

Métricas de Ineficiencia Operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** Representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de

vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.

- **Colas:** Corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de ser servidos. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección aguas arriba y propagar la congestión al resto de la red.
- **Detenciones:** Cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Estas magnitudes permiten describir el desempeño operacional de una intersección y de la red. En la formulación computacional, también pueden utilizarse para construir observaciones, señales de recompensa o criterios de evaluación del controlador.

Descripción Probabilística del Flujo y Pelotones Vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \quad (2.3)$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. La ecuación (2.3) formaliza esta probabilidad discreta. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \quad (2.4)$$

donde h es el intervalo temporal en segundos, q es el flujo en veh/h, y t es el umbral de tiempo analizado. La ecuación (2.4) expresa la probabilidad de que el intervalo supere dicho umbral.

Esta representación probabilística supone independencia entre arribos sucesivos y resulta adecuada para flujos bajos o moderados en condiciones de flujo libre, sin influencia de semáforos cercanos aguas arriba [CyMR18]. El supuesto pierde validez en condiciones de congestión o en redes urbanas semaforizadas. En estos casos, los semáforos interrumpen el Flujo vehicular y agrupan los vehículos en pelotones (*platoons*), lo que introduce dependencia espacio-temporal entre arribos consecutivos. La Figura 2.4 compara conceptualmente los intervalos temporales resultantes bajo ambas condiciones. En ella, $h_{\min}$ representa un umbral mínimo conceptual para el intervalo mostrado en la curva con pelotones; no es una estimación obtenida de datos de este trabajo. Las curvas y ese umbral tienen, por lo tanto, función ilustrativa y no constituyen una distribución empírica particular.

En este trabajo, la limitación de esos modelos analíticos motiva el uso de simulación microscópica para estudiar el control semafórico bajo condiciones variables. En SUMO, la formación, dispersión e interacción de los pelotones emergen de la dinámica simulada de los vehículos y de sus cambios de carril. El uso de distintos perfiles de demanda y semillas de simulación permite evaluar el comportamiento del controlador frente a variaciones en las condiciones de operación.

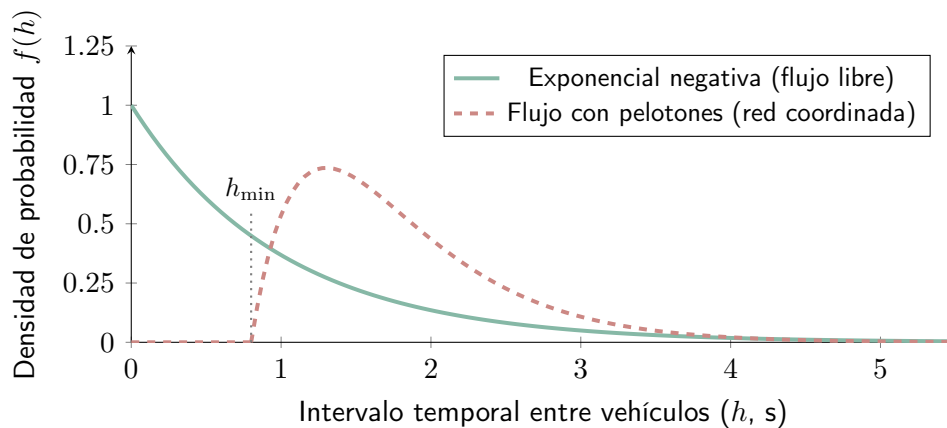


Figura 2.4: Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de pelotones. Las curvas representan situaciones conceptuales y no una distribución empírica particular.

La Figura 2.4 se interpreta comparando la forma de ambas curvas. El eje horizontal representa el intervalo h entre vehículos y el eje vertical la densidad de probabilidad $f(h)$, es decir, la concentración relativa de intervalos alrededor de cada valor. En el modelo exponencial, la densidad disminuye desde el origen y asigna mayor frecuencia relativa a los intervalos breves. En la representación con pelotones, la densidad comienza después de $h_{\min}$ y alcanza un máximo alejado del origen, lo que ilustra que el arribo de un vehículo queda condicionado por la separación y la coordinación con los que lo preceden. La comparación permite reconocer por qué el supuesto de arribos independientes puede resultar insuficiente en una red semaforizada.

2.1.3 Sistemas de Control de Tránsito

Para ordenar los movimientos en conflicto y regular el paso en las intersecciones urbanas se utilizan sistemas de control de tránsito [PW16]. El semáforo asigna el derecho de paso de manera alternada mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelo computacional, el semáforo actúa como el mecanismo mediante el cual el controlador interviene sobre la red. La selección de fases, la duración del verde efectivo y la coordinación entre intersecciones modifican la prioridad de paso y afectan la evolución de las colas y las demoras.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje. Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. Para el presente trabajo, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema semaforico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la Capacidad y el Verde Efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la Figura 2.5, q_{max} representa la tasa de descarga de saturación, g el intervalo de verde efectivo, y l_1 y l_2 las pérdidas inicial y final de servicio, respectivamente; el tiempo se expresa en s y la tasa de flujo en veh/h. La barra inferior distingue los intervalos de rojo, verde y amarillo, mientras que la curva superior muestra cómo la descarga alcanza y abandona el régimen de saturación. La descarga observada no depende solo de la duración nominal del verde, sino también de pérdidas de arranque, transiciones y otras restricciones temporales del

control. La figura es conceptual: estos parámetros no se estiman en este trabajo. En su lugar, los efectos se representan mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

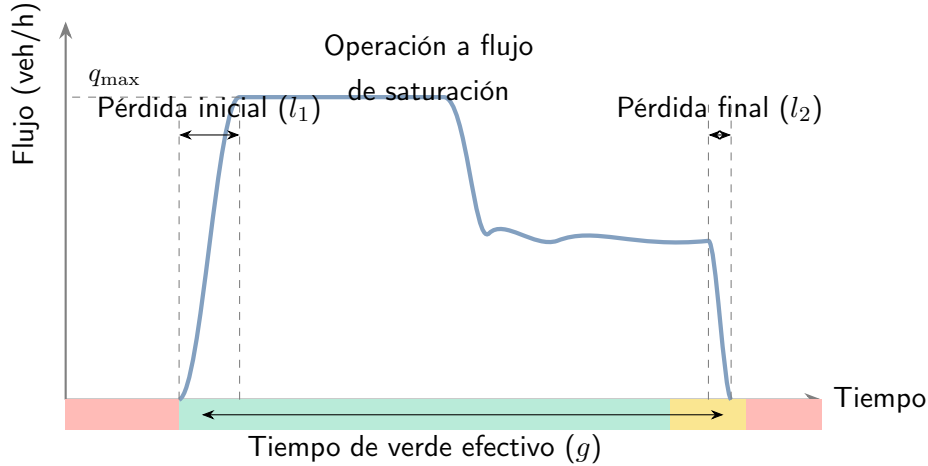


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

La Figura 2.5 debe leerse como una relación temporal: las pérdidas l_1 y l_2 reducen el intervalo en que la fase puede descargar vehículos a la tasa $q_{\max}$. Por tanto, el verde efectivo g no equivale necesariamente al verde nominal programado en el controlador.

Clasificación de los Controladores de Semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo, la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite de seguridad y funcionamiento, como el verde mínimo ($g_{\min}$) y el verde máximo ($g_{\max}$) [PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4 Simulación de Tránsito

Debido a la densidad y complejidad de las redes urbanas, la sincronización y optimización de los tiempos semafóricos requiere representar la evolución del tránsito bajo condiciones variables. Los modelos analíticos y agregados resultan útiles bajo determinados supuestos,

pero pueden ser limitados para describir la formación de colas y las interacciones locales durante la operación dinámica de una red [PW16]. De acuerdo con la taxonomía clásica [FA11], los modelos de tránsito pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los mesoscópicos representan entidades intermedias, como grupos o pelotones de vehículos; y los microscópicos modelan individualmente los vehículos, sus rutas y su evolución dentro de la red. Para el presente trabajo resulta especialmente relevante este último nivel, porque permite estudiar cómo las decisiones semafóricas afectan paso a paso la formación de colas, las detenciones y la interacción local entre vehículos.

La simulación de tránsito constituye una representación abstracta del fenómeno real. Su objetivo no es reproducir visualmente una escena particular, sino modelar los procesos que determinan la evolución de la circulación mediante reglas de comportamiento y variables observables. La calidad de los resultados depende de la representación de la red, la demanda, la flota, los modelos de comportamiento y las reglas de control. Por ello, la simulación debe entenderse como un entorno experimental sujeto a supuestos y parámetros, cuyos resultados requieren una evaluación operacional acorde con el objetivo del estudio. La correspondencia entre el fenómeno y su representación se resume en la Figura 2.6: el tránsito observado aporta el fenómeno de referencia y el simulador representa sus vehículos, red y reglas dinámicas para permitir experimentos controlados.



Figura 2.6: Correspondencia entre la circulación real y su representación microscópica en SUMO. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La Figura 2.6 muestra, de forma sintética, la correspondencia entre una escena vial y su representación microscópica en SUMO. Su función es ilustrar cómo los vehículos y la

infraestructura se transforman en entidades computacionales; no constituye una validación del simulador ni una representación del escenario utilizado en este trabajo.

Para el presente trabajo resulta de mayor interés el nivel microscópico, porque representa explícitamente cada vehículo, su ruta y su movimiento dentro de la red [FA11]. SUMO pertenece a esta categoría. Es una suite libre y de código abierto iniciada en el Instituto de Sistemas de Transporte del Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*) (DLR), que permite modelar sistemas de tránsito intermodales, incluidos vehículos, transporte público y peatones. También ofrece herramientas para importar redes, calcular rutas, visualizar y evaluar simulaciones, además de interfaces para controlar su ejecución [DLR26].

En SUMO, la evolución vehicular se actualiza en pasos temporales discretos sobre una representación espacial continua. El entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR26].

La interfaz gráfica *sumo-gui* permite inspeccionar visualmente la red, los vehículos y algunos valores de la simulación durante la ejecución. Esta visualización cumple una función de apoyo para la inspección del escenario y la depuración del modelo; las métricas utilizadas para la evaluación se obtienen de las salidas de simulación y de las interfaces de control [DLR26]. La interfaz y su función de inspección se muestran en la Figura 2.7; la Interfaz gráfica de usuario (*Graphical User Interface*) (GUI) no constituye la fuente de las métricas ni reemplaza su cálculo operacional.

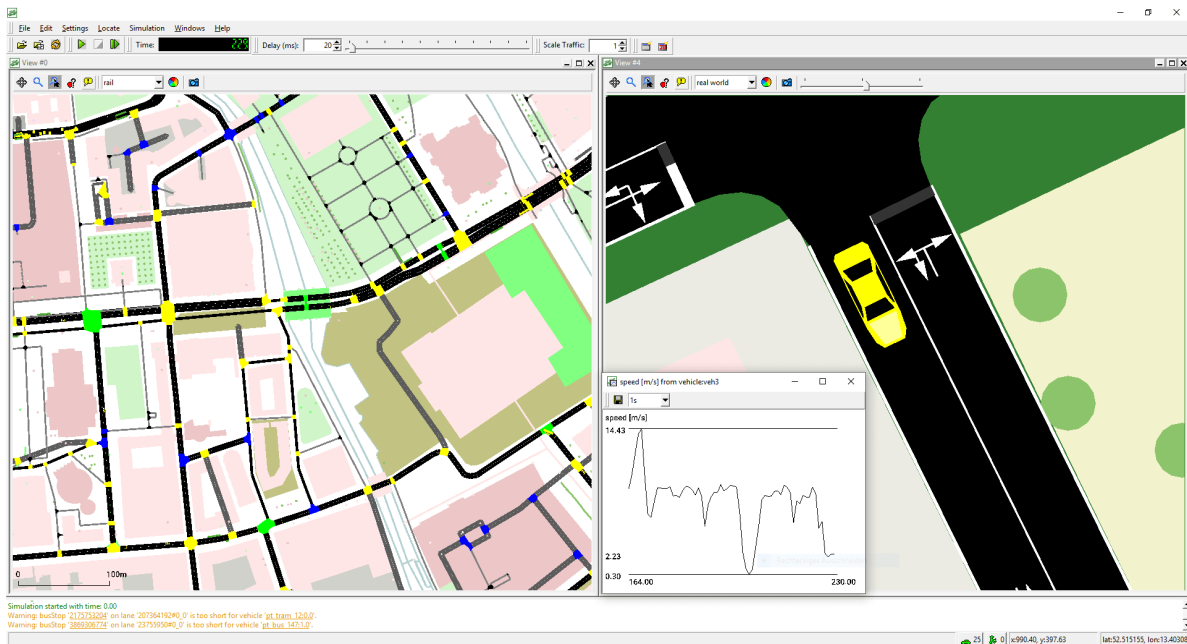


Figura 2.7: Interfaz *sumo-gui* durante una simulación microscópica. El panel izquierdo presenta una vista general de la red; el derecho amplía un vehículo sobre la calzada; el recuadro inferior grafica su velocidad y la zona inferior muestra mensajes de ejecución. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La Figura 2.7 reúne cuatro zonas de inspección. A la izquierda se observa la red completa, útil para localizar la parte del escenario que se está examinando. A la derecha se muestra una vista ampliada de la calzada y de un vehículo seleccionado, mientras que el gráfico superpuesto en la parte inferior representa la evolución temporal de su velocidad, expresada en metros por segundo. La franja inferior contiene el registro de mensajes y advertencias producidos durante la ejecución, y la barra superior agrupa los controles de avance, pausa, tiempo y escala de visualización. En conjunto, estos elementos permiten verificar visualmente la presencia de vehículos, su movimiento y posibles inconsistencias de configuración. En este trabajo la interfaz se usa con fines de inspección y depuración; la evaluación cuantitativa se realiza con las salidas de simulación y las interfaces de control [DLR26].

Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semafórico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio pueden evaluarse mediante sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles, completitud de viajes y propagación de congestión. En la formulación computacional, las variables de tránsito constituyen observaciones, las fases admisibles constituyen acciones y la evolución simulada constituye la transición del entorno. La definición operacional de estas variables y su procedimiento de cálculo se presentan en la metodología.

Representación Microscópica de Intersecciones en SUMO

La naturaleza microscópica de la simulación no se reduce a representar visualmente vehículos. Cada unidad modifica su velocidad y posición de acuerdo con modelos de seguimiento vehicular y cambio de carril, los parámetros de su tipo de vehículo y las restricciones de la red. De este modo, variables como la aceleración, la separación entre vehículos, la velocidad deseada y la disponibilidad de carriles intervienen en la formación y dispersión de colas. Los valores concretos utilizados para la flota y para estos parámetros pertenecen a la configuración experimental y se detallan en la metodología [DLR26].

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semafórico. Para el esquema conceptual de la Figura 2.8, que adopta circulación por la derecha, se distinguen carriles, líneas de detención, conexiones entre carriles y estados de control semafórico [DLR26]. La fase representada es solo una ilustración de cómo pueden habilitarse movimientos directos opuestos; la ausencia visual de los estados g e y no implica que no existan en la lógica de SUMO ni que se excluyan de otras fases o transiciones:

- **Carriles entrantes (L_{in}) y salientes (L_{out}):** Los carriles entrantes conducen el flujo vehicular hacia la intersección, mientras que los carriles salientes reciben el flujo descargado tras cruzar el nodo.
- **Línea de detención (*stop line*):** Referencia operacional próxima al final del carril

entrante, delante de la cual se detienen los vehículos cuando el movimiento no está habilitado.

- **Conexiones (*connections* o *links*):** Relaciones que conectan un carril entrante con un carril saliente y definen los movimientos posibles a través de la intersección. Cuando están controladas por un semáforo, se asocian con un índice de enlace y con un estado de señal. En SUMO, G representa verde con prioridad, g verde sin prioridad y sujeto a cesión, y amarillo y r rojo. La figura muestra únicamente una fase conceptual en la que los movimientos directos opuestos del eje horizontal están habilitados y los movimientos transversales o conflictivos permanecen retenidos.

Esta representación es compatible con la formulación algorítmica del problema, donde una fase define un conjunto de estados sobre los enlaces controlados. Una acción válida debe seleccionar una fase compatible con la geometría de la intersección y respetar las transiciones y restricciones de seguridad definidas por el entorno.

La Figura 2.8 ilustra esos carriles, líneas de detención y conexiones en una fase conceptual; su función es relacionar la geometría de la red con las acciones semafóricas del modelo, no representar un programa experimental completo.

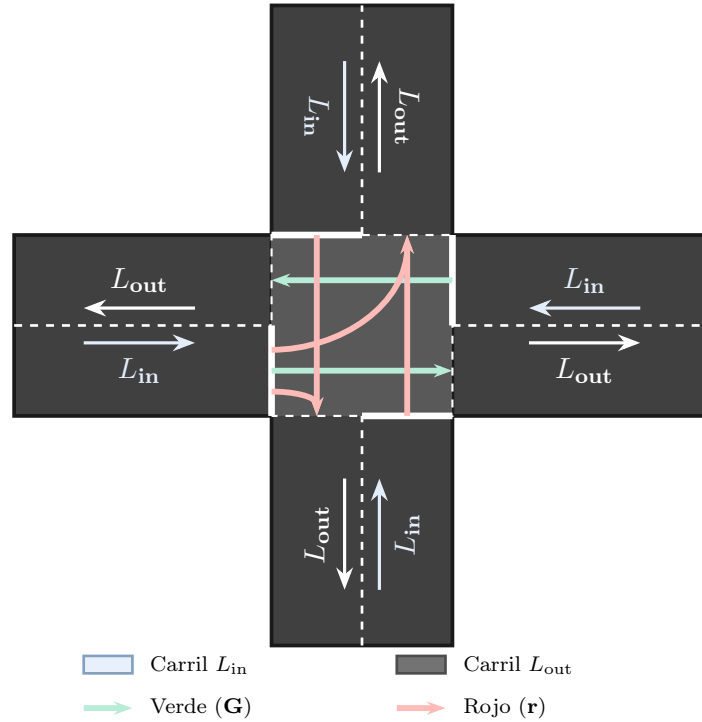


Figura 2.8: Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.

La Figura 2.8 se lee desde los carriles de aproximación hacia el centro de la intersección. Las flechas blancas indican el sentido de circulación de L_{in} y L_{out} , y las líneas blancas

continuas situadas antes del área de cruce representan las líneas de detención. Dentro de la intersección, las flechas verdes corresponden a conexiones habilitadas con prioridad y las rojas a conexiones retenidas. Por consiguiente, una Fase (Ingeniería de Tránsito) no modifica la geometría ni el sentido de los carriles: determina qué conexiones pueden utilizarse durante ese intervalo de control.

Componentes y Configuración de SUMO

La suite SUMO reúne aplicaciones para construir redes, generar demandas, ejecutar simulaciones, visualizar sus resultados y controlar la ejecución mediante interfaces externas [DLR26]:

- ***netconvert***: Aplicación que importa redes desde distintas fuentes y compila su representación en el formato nativo de SUMO. También permite reconstruir o ajustar elementos de la red, como las conexiones y los programas semafóricos.
- ***netedit***: Editor gráfico para crear y modificar de forma interactiva nodos, calles, carriles, conexiones, restricciones de giro y elementos de control semafórico.
- ***osmWeb Wizard***: Herramienta que automatiza la generación de un escenario a partir de un área seleccionada de OpenStreetMap (OSM). Se considera una alternativa de importación y no reemplaza el procesamiento específico realizado con *netconvert*.
- **Interfaz de control del tránsito (*Traffic Control Interface*) (TraCI)**: Interfaz que permite consultar el estado de una simulación en ejecución y modificarlo durante el avance paso a paso, incluyendo variables de vehículos, carriles, detectores y semáforos [DLR26].
- ***libsumo***: Implementación embebida que ofrece la interfaz de programación de SUMO sin la comunicación mediante sockets de la modalidad TraCI estándar. Esta alternativa resulta útil para integrar el simulador con controladores externos y reducir la sobrecarga de comunicación.

Desde la perspectiva del modelado computacional, una simulación de SUMO se construye a partir de archivos Lenguaje de marcado extensible (*Extensible Markup Language*) (XML) de red, demanda, elementos adicionales y configuración [DLR26]. Los archivos Zona de análisis de tránsito (*Traffic Analysis Zone*) (TAZ) pueden utilizarse durante la estructuración espacial y la generación de demanda, mientras que las salidas de la simulación se solicitan mediante opciones de configuración específicas. Sus funciones generales se resumen en la Tabla 2.2:

Tabla 2.2: Archivos de entrada y configuración empleados en una simulación de SUMO.

Archivo	Extensión	Función en el Modelo Computacional
Red Vial	.net.xml	Topología física, carriles, conexiones y lógicas semafóricas.
Rutas y Demanda	.rou.xml	Tipos de vehículos, viajes, flujos e itinerarios.
Zonas TAZ	.taz.xml	Zonas para estructurar espacialmente la demanda.
Elementos Adicionales	.add.xml	Detectores, tipos de vehículo y programas auxiliares.
Configuración	.sumocfg	Archivos de entrada, opciones de ejecución y salidas.

Métricas de Evaluación Operacional

La simulación microscópica produce salidas que permiten evaluar el desempeño del control desde la perspectiva de la ingeniería de tránsito. En este trabajo se consideran las siguientes categorías de indicadores:

- **Demora y tiempo de espera:** Cuantifican el tiempo adicional o el tiempo detenido que experimentan los vehículos durante su recorrido. Pueden analizarse de forma agregada en la red y desagregarse por intersección, acceso o agente, siempre que se mantenga una definición operacional consistente [CyMR18, DLR26].
- **Complejidad y flujo servido:** La cantidad de vehículos que completa su itinerario y su relación con la demanda generada permiten distinguir una reducción genuina de la demora de una disminución del flujo atendido.
- **Equidad espacial:** La distribución de las demoras entre intersecciones permite identificar si una política mejora el desempeño global a costa de concentrar la espera en accesos o agentes particulares.
- **Emisiones contaminantes (CO₂, CO, NO_x, HC):** SUMO estima las emisiones de dióxido de carbono (CO₂), monóxido de carbono (CO), óxidos de nitrógeno (NO_x) e hidrocarburos (HC) a partir del estado cinemático de los vehículos, su tipo y la clase de emisiones configurada [DLR26]. La masa total debe interpretarse junto con los vehículos completados y la distancia recorrida, o expresarse mediante una medida normalizada.
- **Indicadores de validez de la simulación:** Los mecanismos *teleport* de SUMO, los vehículos pendientes y otras incidencias de ejecución permiten detectar bloqueos o condiciones que comprometen la comparabilidad de una ejecución.

Las definiciones operacionales, fórmulas, unidades, ventanas de evaluación y procedimientos de agregación empleados en este trabajo se presentan en la metodología. Las recompensas y las pérdidas de optimización corresponden al entrenamiento de los algoritmos de aprendizaje y se desarrollan en las secciones dedicadas a RL y MARL.

En este marco, SUMO permite observar el estado de la red y modificar las fases semafóricas durante la ejecución mediante interfaces de control. Esta capacidad convierte el control semafórico en un problema de decisión secuencial, lo que motiva la introducción del aprendizaje por refuerzo en la siguiente sección [DLR26].

2.2 Aprendizaje por Refuerzo (RL)

El Aprendizaje automático (*Machine Learning*) (ML) suele dividirse en tres paradigmas. En el Aprendizaje supervisado (*Supervised Learning*) (SL), el modelo se entrena con ejemplos etiquetados que indican la salida esperada. En el Aprendizaje no supervisado (*Unsupervised Learning*) (UL), se busca descubrir estructura en datos no etiquetados, por ejemplo mediante agrupamiento o reducción de dimensionalidad. El Aprendizaje por Refuerzo (*Reinforcement Learning*) (RL) aborda un problema distinto: una entidad aprende mediante interacción con un entorno qué acciones elegir para maximizar una recompensa acumulada [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el agente (*agent*), que aprende y toma decisiones, y el entorno (*environment*), que representa todo aquello externo al agente con lo cual este interactúa. En cada paso de tiempo, el agente observa una situación del entorno, selecciona una acción y recibe como respuesta una nueva observación junto con una recompensa. El comportamiento del agente está determinado por una política (*policy*), denotada por π , que especifica cómo se eligen las acciones a partir de los estados u observaciones disponibles. La señal de recompensa (*reward signal*) define el objetivo inmediato del problema, mientras que la función de valor estima la conveniencia de estados o acciones en términos de recompensa futura esperada. Sutton y Barto también distinguen el modelo del entorno, entendido como una descripción que permite predecir sus transiciones y recompensas; cuando dicho modelo no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

En el control semafórico, el controlador de una intersección puede interpretarse como el

agente y la red vial simulada en SUMO como el entorno. La selección de una fase constituye la acción, mientras que la evolución posterior del tránsito produce nuevas observaciones y una señal de recompensa que orienta el aprendizaje [DLR26].

Como ejemplo conductor, considérese un par de intersecciones vecinas. El agente observa localmente las colas y demoras de su intersección, selecciona una fase y modifica la corriente de vehículos que llegará a la siguiente. Esa consecuencia aguas abajo puede cambiar sus arribos y colas, y se refleja en una recompensa asociada al desempeño de la red. El retorno permite acumular esos efectos a lo largo de varias decisiones, en lugar de evaluar una fase solo por su consecuencia inmediata.

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la experiencia futura disponible para el aprendizaje [SB18].

2.2.1 Procesos de Decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el Proceso de decisión de Markov (*Markov Decision Process*) (MDP). En un MDP finito, el agente y el entorno interactúan en una secuencia de pasos temporales discretos $t = 0, 1, 2, \dots$. En cada instante t , el agente recibe una representación del estado del entorno $S_t \in \mathcal{S}$ y selecciona una acción $A_t \in \mathcal{A}(S_t)$. Como consecuencia de esa acción, el entorno emite una recompensa numérica $R_{t+1} \in \mathcal{R}$ y transita a un nuevo estado S_{t+1} [LML⁺25]. Este ciclo se representa en la Figura 2.9.

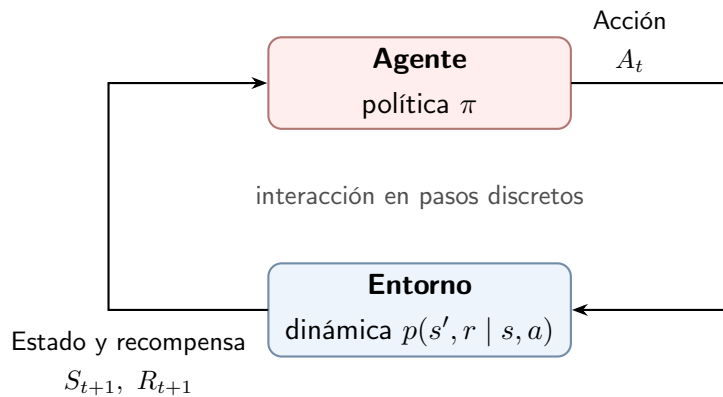


Figura 2.9: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

La Figura 2.9 debe leerse de izquierda a derecha y de arriba hacia abajo: la política del agente produce la acción A_t , el entorno aplica esa acción y devuelve el nuevo estado junto con la recompensa. La función $p(s', r | s, a)$ resume probabilísticamente ese retorno;

no representa una segunda política ni una etapa adicional de decisión.

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r \mid s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s :

$$p(s', r \mid s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \quad (2.5)$$

En esta ecuación, las letras minúsculas s , a , s' y r designan valores concretos que pueden tomar las variables aleatorias respectivas. $S_t \in \mathcal{S}$ representa el estado en el instante t , $A_t \in \mathcal{A}$ es la acción seleccionada, $R_{t+1} \in \mathbb{R}$ es la recompensa escalar recibida y $S_{t+1} \in \mathcal{S}$ es el nuevo estado resultante. En sentido estricto, el estado reúne toda la información relevante para la dinámica; en la implementación, el controlador recibe un vector de observaciones de tránsito, que puede ser solo una representación parcial de ese estado. Por ello, la ecuación (2.5) ofrece el modelo conceptual de la transición posterior a seleccionar una fase y recibir una recompensa de demora compartida.

La propiedad de Markov establece que, dado el estado y la acción actuales, la distribución del próximo estado y recompensa no depende explícitamente de la historia completa de estados y acciones anteriores. En otras palabras, el estado debe contener toda la información relevante del pasado para predecir la evolución futura del proceso [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*) (POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t , que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.6)$$

En esta ecuación, G_t representa la suma acumulada descontada de recompensas futuras a partir del instante t , y γ es el factor de descuento. Un valor de γ cercano a cero prioriza recompensas inmediatas, mientras que un valor cercano a uno otorga un peso significativo a recompensas futuras de largo plazo [ACS24]. En el modelo de este trabajo, la ecuación (2.6) acumula las consecuencias futuras de una fase sobre la demora y las colas de la red,

no solo su efecto inmediato.

2.2.2 Métodos Basados en Valor y Basados en Política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

En los métodos basados en valor, la estrategia consiste en estimar qué tan conveniente es encontrarse en un estado o ejecutar una acción determinada. Esto se formaliza mediante la función de valor de estado $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$ y la función de valor de acción $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$. Ambas funciones satisfacen ecuaciones recursivas conocidas como ecuaciones de Bellman, que expresan la relación entre el valor de una situación actual y el valor esperado de sus posibles sucesores [SB18].

Para una política fija, las ecuaciones de Bellman permiten evaluar el valor esperado de los estados y acciones. Cuando el objetivo es encontrar la mejor acción disponible, se utilizan las ecuaciones de optimalidad de Bellman:

$$q_*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a \right]. \quad (2.7)$$

La ecuación (2.7) justifica el objetivo de actualización de Q-learning y, posteriormente, de *Deep Q-Network* (red neuronal profunda que aproxima valores de acción) (DQN). La esperanza $\mathbb{E}$ promedia las posibles recompensas y estados siguientes según la dinámica del entorno; a' recorre las acciones que podrían elegirse en el estado siguiente. La maximización sobre a' convierte el problema de evaluación de una política en un problema de control orientado a aproximar la función de acción-valor óptima q_* . Para el control semafórico, esta relación expresa el compromiso entre el efecto de la fase actual y las condiciones de tránsito que esa fase produce en los pasos siguientes [SB18].

Cuando la dinámica del entorno no se conoce completamente, estas funciones pueden estimarse a partir de la experiencia. Los métodos de TD combinan el muestreo de transiciones reales con actualizaciones basadas en estimaciones previas, mecanismo conocido como *bootstrapping*. Un ejemplo central es *Q-learning*, que actualiza la función de acción-valor hacia una estimación de la función óptima $q_*(s, a)$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \quad (2.8)$$

En esta ecuación, $Q(S_t, A_t)$ es la estimación del valor de la acción A_t en el estado S_t , $\alpha \in (0, 1]$ representa la tasa de aprendizaje (*learning rate*), R_{t+1} es la recompensa observada, γ es el factor de descuento, y $\max_a Q(S_{t+1}, a)$ estima el valor máximo alcanzable en el siguiente estado S_{t+1} . La ecuación (2.8) muestra cómo una nueva observación de tránsito corrige el valor asignado a la acción ejecutada.

Q-learning es un método *off-policy*: la política que genera las transiciones puede diferir

de la política cuyo valor óptimo se estima. En cambio, los métodos de gradiente de política utilizan, en principio, muestras generadas por la política que se está optimizando y se consideran *on-policy*. Esta distinción resulta central para comparar *Independent Deep Q-Network* (DQN independiente por agente) (IDQN) con *Independent Proximal Policy Optimization* (PPO independiente por agente) (IPPO), *Multi-Agent Proximal Policy Optimization* (PPO multiagente con información centralizada durante el entrenamiento) (MAPPO) y *Heterogeneous-Agent Proximal Policy Optimization* (PPO con actualización secuencial de agentes heterogéneos) (HAPPO), especialmente en entornos multiagente donde las políticas de los demás agentes cambian durante el entrenamiento.

Por el contrario, los métodos basados en política parametrizan directamente la política $\pi(a | s, \theta) = \Pr\{A_t = a | S_t = s, \theta_t = \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables (pesos neuronales). En lugar de derivar la acción exclusivamente de una función de valor, ajustan los parámetros θ para maximizar una medida de rendimiento escalar $J(\theta)$. Para ello aplican ascenso de gradiente sobre el rendimiento esperado:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \quad (2.9)$$

La ecuación (2.9) incrementa los parámetros en la dirección del gradiente del rendimiento. En esta ecuación, $\nabla J(\theta_t)$ representa el gradiente de la función de rendimiento respecto a los parámetros de la política y $\alpha > 0$ es la tasa de paso que determina el tamaño de la actualización. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona la base teórica para estimar este gradiente sin derivar explícitamente la distribución de estados inducida por la política [SB18]. En este trabajo, el ascenso modifica la política para aumentar la probabilidad de acciones que producen mejores recompensas de demora.

Los métodos actor-crítico combinan una política parametrizada con una estimación de valor y actualizan ambos componentes de manera concurrente [KT03, SB18]. El **actor** representa la política $\pi_\theta(a | s)$ y determina la distribución utilizada para seleccionar la acción. El **crítico** no controla el entorno: estima, por ejemplo, el valor de estado $V_\phi(s)$ y produce una señal que indica si el resultado observado fue mejor o peor que lo esperado bajo la política vigente. Los parámetros θ y ϕ pueden pertenecer a redes separadas o a una arquitectura con capas compartidas y cabezales distintos.

En una formulación actor-crítico de un paso, después de ejecutar A_t en S_t , el entorno devuelve la recompensa R_{t+1} , el estado S_{t+1} y el indicador terminal d_{t+1} . El crítico calcula el error de diferencia temporal mediante:

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t), \quad (2.10)$$

La ecuación (2.10) define el error temporal que utiliza el crítico para evaluar la transición. En esta ecuación, el factor $(1 - d_{t+1})$ elimina el valor del estado siguiente cuando finaliza el episodio. Si V_ϕ aproxima correctamente a V^π , δ_t constituye una estimación de un paso de

la ventaja de la acción ejecutada. Por ello, un valor positivo indica un resultado superior al esperado para ese estado y uno negativo indica lo contrario. El crítico ajusta ϕ para reducir el error entre $V_\phi(S_t)$ y el objetivo temporal $R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1})$, mientras que el actor utiliza la misma señal para reforzar o desalentar la acción observada:

$$\theta \leftarrow \theta + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t | S_t). \quad (2.11)$$

La actualización del actor queda expresada en (2.11), donde $\alpha_\theta > 0$ es su tasa de aprendizaje y $\nabla_\theta \log \pi_\theta(A_t | S_t)$ indica cómo cambia la log-probabilidad de la acción ejecutada al modificar los parámetros. Multiplicar este término por δ_t aumenta, en promedio, la probabilidad de acciones con resultados mejores que los previstos y la reduce cuando ocurre lo contrario. Esta forma canónica permite explicar el intercambio de información entre ambos componentes, aunque los algoritmos modernos suelen sustituir δ_t por estimadores de ventaja que combinan varios pasos, como *Generalized Advantage Estimation* (estimación de la ventaja mediante errores temporales ponderados) (GAE), y realizan las actualizaciones sobre *minibatches*.

La Figura 2.10 representa el ciclo completo de aprendizaje. La observación S_t ingresa al Actor (Aprendizaje por Refuerzo), que aplica la política π_θ y selecciona la acción A_t (que en este dominio determina la fase semafórica a activar); el Entorno devuelve la recompensa R_{t+1} , el estado siguiente S_{t+1} y el indicador terminal d_{t+1} . El Crítico (Aprendizaje por Refuerzo) recibe la información de la transición y estima los valores $V_\phi(S_t)$ y $V_\phi(S_{t+1})$, con los que calcula el error temporal δ_t o una ventaja $\hat{A}_t$. Las flechas continuas representan la interacción con el entorno y las discontinuas el flujo de aprendizaje que ajusta los parámetros del actor; de este modo, la figura distingue la decisión operativa de la evaluación utilizada para entrenarla.

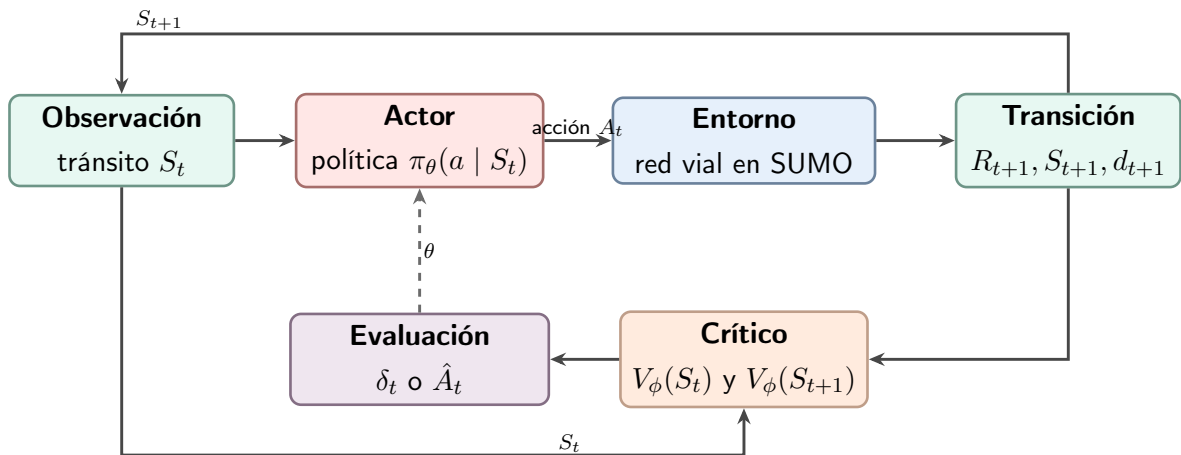


Figura 2.10: Arquitectura actor-crítico aplicada al control semafórico. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].

Antes de introducir las redes neuronales, la Figura 2.11 ubica los algoritmos evalu-

ados dentro de las familias de métodos libres de modelo basados en valor y basados en política. La rama basada en valor conduce desde RL hasta DQN y su variante independiente IDQN; la rama basada en política conduce hasta *Proximal Policy Optimization* (optimización mediante un objetivo sustituto recortado) (PPO) y sus variantes IPPO, MAPPO y HAPPO. En este trabajo, esta clasificación permite distinguir el mecanismo de selección de acciones y no pretende agotar la taxonomía de RL.

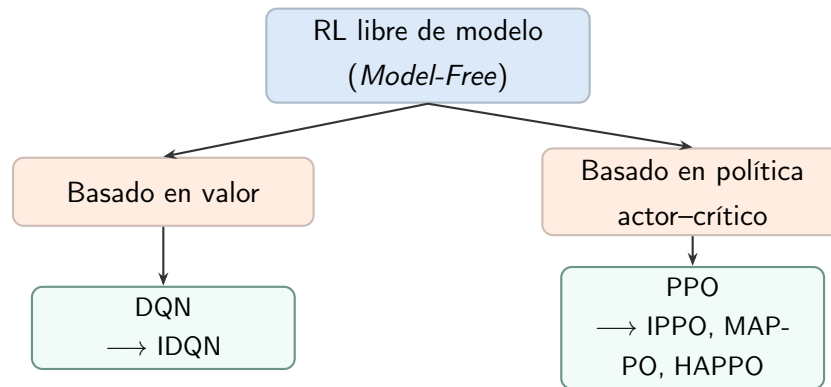


Figura 2.11: Relación entre las familias de algoritmos utilizadas en esta investigación. Los cuatro métodos evaluados son libres de modelo y se derivan de las líneas DQN y PPO.

La Figura 2.11 no indica una secuencia de entrenamiento ni una jerarquía de calidad. Solo organiza los métodos según el objeto que aprende el modelo: valores de acción en la rama DQN o una distribución de acciones en la rama PPO. Las variantes independientes y multiagente se muestran como extensiones de esas dos familias.

En síntesis, el estado o la observación resume la situación que el agente puede utilizar, la acción corresponde a una fase, la recompensa refleja la consecuencia operativa y el valor estima sus efectos futuros. Las redes neuronales que se presentan a continuación aproximarán funciones de valor o políticas a partir de esta información, sin reemplazar la formulación de RL.

2.2.3 Aprendizaje Profundo y Redes Neuronales

Para seguir el desarrollo, primero se presenta la unidad neuronal y la organización de una red; luego se explica cómo se ajustan sus parámetros mediante pérdida, retropropagación y descenso de gradiente; finalmente se muestra cómo estas redes se incorporan a RL mediante DQN y PPO. La distinción es importante: la red neuronal es un aproximador de funciones dentro de RL, mientras que el objetivo de RL proviene de las recompensas y de los retornos que resultan de interactuar con el entorno.

El aprendizaje profundo (*Deep Learning*) es un subcampo del aprendizaje automático centrado en el entrenamiento de redes neuronales con múltiples capas de parámetros ajustables. Estas arquitecturas permiten aprender representaciones jerárquicas y no lineales a partir de los datos, en lugar de depender exclusivamente de características fijadas de antemano [BB24, GBC16]. En este trabajo, esta capacidad se utiliza para aproximar

funciones de valor y políticas de control cuando una representación tabular no resulta adecuada.

El término “neuronal” alude a una inspiración muy simplificada en unidades biológicas, no a una equivalencia con el cerebro. Una red artificial realiza operaciones matemáticas definidas y aprende sus parámetros a partir de datos y señales de entrenamiento; no reproduce la estructura, la percepción ni el funcionamiento de un sistema nervioso. Asimismo, aumentar el número de capas o de neuronas puede ampliar la capacidad de representación, pero no garantiza por sí mismo un mejor resultado: el desempeño también depende de los datos, el objetivo, la optimización y la generalización [BB24, GBC16].

La Unidad Neuronal y Redes *Feedforward*

En problemas de control con espacios de estados continuos o de gran escala, los métodos tabulares resultan impracticables [SB18]. La aproximación de funciones resuelve esta limitación mediante representaciones parametrizadas, lo que permite generalizar lo aprendido a situaciones similares no observadas. Para este fin se emplean redes neuronales artificiales [BB24].

El elemento básico de una red es la unidad neuronal artificial (o neurona) [BB24, GBC16]. Conceptualmente, opera como una pequeña unidad de decisión: recibe varios datos de entrada, evalúa la influencia de cada uno mediante multiplicadores denominados pesos, suma un valor base de ajuste (sesgo) y aplica una regla de activación para emitir una respuesta. Los pesos no son importancias fijadas de antemano: se ajustan durante el entrenamiento para reducir la discrepancia entre la salida producida y el objetivo disponible [BB24, GBC16].

La Figura 2.12 muestra esta secuencia de entrada, suma ponderada con sesgo, activación y salida. La operación $a = \mathbf{w}^\top \mathbf{x} + b$ es una transformación afín: primero pondera cada entrada mediante w_i y luego incorpora el sesgo b . Sus símbolos representan las entradas x_i , los pesos w_i , la preactivación a y la salida y ; en el modelo de este trabajo, las entradas pueden ser variables observadas del tránsito y la salida puede alimentar una función de valor o una política.

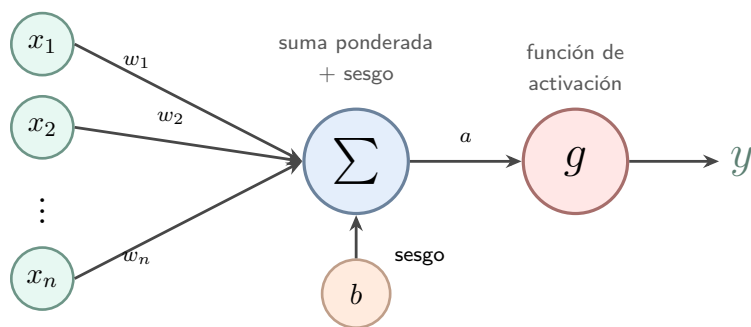


Figura 2.12: Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación.

La Figura 2.12 esquematiza este flujo de izquierda a derecha:

1. **Entradas** ($x_1, \dots, x_n$): Mediciones del entorno que ingresan a la unidad (por ejemplo, longitud de colas o velocidades medias).
2. **Pesos ($\mathbf{w}$) y sesgo (b)**: Cada entrada x_i se multiplica por un peso w_i , que regula su influencia local en la combinación de esta unidad; no representa una importancia global de la variable en toda la red. Al sumarse las entradas ponderadas, se añade un sesgo b que ajusta el nivel base de respuesta, obteniendo la preactivación $a = \mathbf{w}^\top \mathbf{x} + b$.
3. **Función de activación** ($y = g(a)$): transforma la preactivación mediante una regla, generalmente no lineal. Sin funciones de activación no lineales entre capas, la composición completa se reduciría a una única transformación afín y no podría representar relaciones no lineales complejas.
4. **Salida** (y): respuesta generada por la neurona. En una capa oculta, y alimenta las unidades de la capa siguiente; en la capa final, puede contribuir a estimar valores de acción o probabilidades de las fases de control.

Como ejemplo conceptual, una red podría recibir la cantidad de vehículos detenidos, la ocupación y la velocidad de un acceso, y producir puntajes para clasificar o seleccionar alternativas de fase. La clasificación simple sirve para entender la transformación entrada-salida, pero en el control semafórico la salida no tiene por qué ser una etiqueta: puede ser un valor para cada acción o una probabilidad sobre las fases válidas.

En una red multicapa, la operación para una unidad u dentro de la capa k se formaliza mediante la ecuación (2.12) [BB24, GBC16]:

$$a_{k,u} = \mathbf{w}_{k,u}^\top \mathbf{x}_{k-1} + b_{k,u}, \quad y_{k,u} = g_k(a_{k,u}) \quad (2.12)$$

En esta ecuación, $\mathbf{x}_{k-1}$ proviene de la capa precedente, $\mathbf{w}_{k,u}$ es el vector de pesos, $b_{k,u}$ es el sesgo, g_k es la función de activación y $y_{k,u}$ es la salida de la unidad. Las funciones más utilizadas en capas ocultas son:

- **Unidad lineal rectificada (*Rectified Linear Unit*) (ReLU)**: $g(z) = \max\{0, z\}$, que transmite directamente los valores positivos y anula los negativos, favoreciendo un entrenamiento eficiente.
- **Leaky ReLU**: $g(z) = \max\{0, z\} + \alpha \min\{0, z\}$, donde α controla la pendiente en la región negativa; así, puede reducir el riesgo de neuronas inactivas.
- **Tanh**: $g(z) = \tanh(z)$, que produce salidas suaves acotadas en el intervalo $[-1, 1]$.

Las redes *feedforward* organizan sus unidades en capas acíclicas, es decir, la información avanza desde la entrada hacia la salida sin regresar a una capa anterior. Cada capa recibe la salida de la anterior, aplica una transformación afín y, salvo cuando la tarea requiere una salida lineal, una función de activación. Para una red con L capas parametrizadas, esta operación puede escribirse como:

$$\mathbf{h}_k = g_k(\mathbf{W}_k^\top \mathbf{h}_{k-1} + \mathbf{b}_k), \quad k = 1, \dots, L, \quad \mathbf{h}_0 = \mathbf{x}, \quad \mathbf{y} = \mathbf{h}_L \quad (2.13)$$

Aquí, $\mathbf{W}_k$ y $\mathbf{b}_k$ son, respectivamente, la matriz de pesos y el vector de sesgos de la capa k ; $\mathbf{h}_{k-1}$ es la salida de la capa anterior y g_k se aplica componente a componente. La entrada se identifica como $\mathbf{h}_0 = \mathbf{x}$ y la salida de la última capa como $\mathbf{y} = \mathbf{h}_L$. La arquitectura suele describirse mediante una capa de entrada, una o más capas ocultas y una capa de salida, aunque el número de capas se cuenta de acuerdo con la convención adoptada para los parámetros entrenables [BB24, GBC16]. En el control semafórico, $\mathbf{x}$ puede ser el vector de observaciones de tránsito y $\mathbf{y}$ puede contener valores de acción o probabilidades de fase. La ecuación (2.13) resume la composición de funciones que permite construir una red profunda.

La Figura 2.13 representa una red con dos capas ocultas, entradas $x_1, x_2, \dots, x_n$ y salidas $y_1, \dots, y_r$. Cada círculo es una unidad que recibe todas las salidas de la columna anterior y las transforma según la ecuación (2.12); las conexiones avanzan únicamente hacia la derecha. La figura es esquemática: los puntos suspensivos indican unidades omitidas y no fijan el número de neuronas de la arquitectura experimental.

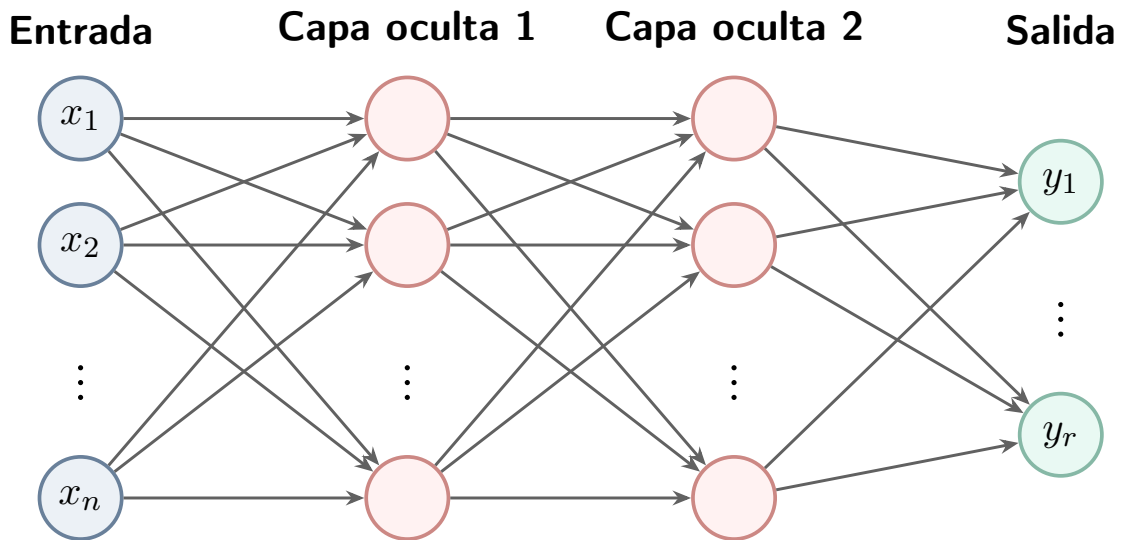


Figura 2.13: Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas.

El Teorema de Aproximación Universal es un resultado de existencia: bajo condiciones apropiadas, una red de dos capas parametrizadas, con una capa oculta suficientemente

ancha, puede aproximar una función continua sobre un dominio compacto con una precisión arbitraria. El teorema no proporciona por sí mismo un procedimiento para encontrar los parámetros, no garantiza un entrenamiento eficiente, una buena generalización ni un buen desempeño práctico sobre datos no observados. La profundidad puede ofrecer representaciones jerárquicas y una mayor eficiencia representacional para ciertas familias de funciones, pero agregar capas o neuronas no garantiza por sí solo un menor error de prueba [BB24].

El Proceso de Optimización

En cada paso de entrenamiento, la propagación hacia adelante lleva la entrada por las capas hasta producir una salida. Luego, una función de pérdida diferenciable y escalar $\mathcal{L}(\theta)$ establece el criterio que se busca optimizar. Medir la discrepancia entre la salida y un objetivo es un caso particular de esa pérdida. La retropropagación calcula cómo contribuyen los parámetros al valor de la pérdida y el descenso de gradiente usa esos gradientes para modificarlos según la ecuación (2.14):

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t), \quad (2.14)$$

En esta ecuación, θ reúne todos los pesos y sesgos ajustables de la red, $\nabla_{\theta} \mathcal{L}(\theta_t)$ indica la dirección en que la pérdida aumenta con mayor rapidez y $\eta > 0$ es la tasa de aprendizaje. El signo negativo hace que la actualización se desplace en la dirección opuesta, buscando reducir la pérdida. En la práctica, el gradiente puede calcularse sobre todo el conjunto de datos de entrenamiento, sobre una observación individual mediante el Descenso de gradiente estocástico (*Stochastic Gradient Descent*) (SGD) o sobre un subconjunto denominado *minibatch*; el tamaño del *minibatch* modifica el costo computacional y la variabilidad de la estimación, pero no constituye un equilibrio óptimo garantizado [BB24].

Una vez obtenida la salida y evaluada la pérdida, el recorrido de la retropropagación comienza en la salida y atraviesa las capas en sentido inverso, hasta las más cercanas a la entrada. Mediante la regla de la cadena calcula las derivadas de la pérdida respecto de los pesos y sesgos, es decir, los gradientes; el optimizador, ya sea descenso de gradiente, Adam u otro método, utiliza esos gradientes para actualizar los parámetros. El mecanismo de optimización es el mismo, aunque en RL la pérdida se construye a partir de recompensas, retornos o errores de diferencia temporal, y no necesariamente de etiquetas supervisadas.

- **Función de pérdida (*loss function*):** es un criterio escalar que asigna un valor a la calidad de la salida respecto del objetivo de optimización. Puede adoptar formas distintas según la tarea; por ejemplo, el error cuadrático medio en regresión o la entropía cruzada en clasificación. En DQN, la pérdida se formula a partir del error de diferencia temporal presentado más adelante.
- **Descenso por *minibatches*:** utiliza un subconjunto de datos para estimar el gra-

diente en cada actualización. La mezcla aleatoria de las observaciones puede reducir correlaciones debidas al orden de los datos, pero el procedimiento no garantiza por sí mismo estabilidad ni convergencia.

- **Adam:** adapta la tasa de aprendizaje de cada parámetro utilizando promedios móviles del gradiente y de su magnitud al cuadrado. Su eficacia es principalmente empírica y depende de la función de pérdida, los datos y la configuración del entrenamiento; no constituye una garantía general de convergencia [BB24].
- **Retropropagación (*backpropagation*):** calcula eficientemente las derivadas parciales de la pérdida respecto de los pesos y sesgos mediante la regla de la cadena. No es el optimizador: proporciona los gradientes que luego utilizan el descenso de gradiente, Adam u otros métodos de actualización [BB24].

En un problema supervisado el objetivo puede ser una etiqueta o un valor conocido; en RL, en cambio, el objetivo se construye a partir de las recompensas y transiciones producidas por la interacción con el entorno. La red aprende a aproximar ese objetivo, pero no define por sí misma qué comportamiento debe considerarse bueno.

2.2.4 Aprendizaje por Refuerzo Profundo (DRL)

El DRL combina algoritmos de RL con aproximadores neuronales. En el control semafórico, las observaciones del tránsito, como colas, ocupación o velocidades, pueden alimentar la red; sus salidas pueden representar valores de acción o parámetros de una política; y las recompensas asociadas con demoras, detenciones y colas orientan la función de pérdida del algoritmo de aprendizaje. Esta integración resulta útil cuando el espacio de estados es continuo o de alta dimensionalidad, aunque exige mecanismos de estabilización y una evaluación cuidadosa de la eficiencia de las muestras [ACS24].

El Problema de la Tríada Mortal (*Deadly Triad*)

La aplicación de aproximadores de funciones al aprendizaje por refuerzo introduce dificultades de estabilidad. Sutton y Barto [SB18] denominan Tríada Mortal (*Deadly Triad*) a la combinación de tres elementos que puede producir inestabilidad o divergencia en las estimaciones de valor, especialmente cuando no se cumplen condiciones suficientes de representación y actualización:

- **Aproximación de Funciones:** Uso de aproximadores potentes y escalables, como las redes neuronales, que extienden el aprendizaje generalizando a través del espacio de estados.
- **Bootstrapping:** La práctica de actualizar estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (como en los métodos TD), en lugar de esperar la consecución de retornos reales completos.

- **Entrenamiento *Off-Policy*:** Optimizar la función de valor utilizando una distribución de datos o transiciones generadas por una política de comportamiento distinta a la política objetivo que se desea evaluar o mejorar.

La tríada no implica divergencia inevitable ni obliga a conservar sus tres componentes. La aproximación de funciones suele ser necesaria para escalar a espacios de estados grandes o continuos, mientras que el *bootstrapping* y el aprendizaje *off-policy* pueden evitarse mediante métodos Monte Carlo y métodos *on-policy*, respectivamente. Estos intercambios modifican la varianza, el sesgo y la eficiencia de muestras del aprendizaje; por ello, los algoritmos profundos incorporan mecanismos de estabilización en lugar de asumir que la divergencia es inevitable [SB18].

***Deep Q-Network* (DQN): aproximación neuronal de valores de acción**

En los métodos basados en valor, el DRL sustituye la tabla clásica por una red neuronal $Q(s, a; \theta)$. En espacios de acción discretos, la DQN estima el valor de las acciones disponibles. En este trabajo, las observaciones semafóricas se organizan como un vector de entrada y la red produce una estimación para cada fase posible; esta representación vectorial es propia de la formulación adoptada aquí y no una condición de toda implementación de DQN [MKS⁺15].

El valor $Q^\pi(s, a)$ no es una probabilidad ni una puntuación fijada por una persona: es el retorno esperado si el agente ejecuta la acción a en el estado s y luego continúa actuando según la política π . En cambio, la función óptima $q_*(s, a)$ representa el mejor retorno esperado al considerar las acciones futuras más convenientes. Q-learning y DQN buscan aproximar esta función óptima mediante el término máximo de su actualización, no una función Q^π asociada a una política fija. En este trabajo, s resume las observaciones de tránsito y cada acción discreta representa una fase semafórica admisible; el significado de “mejor” proviene de la recompensa definida por RL.

La inestabilidad descrita por la Tríada Mortal se manifiesta en DQN como el “problema del blanco móvil”: al generalizar, una actualización que incrementa el valor de un estado puede modificar también los valores estimados para otros estados. Además, las transiciones consecutivas presentan correlación temporal y pueden inducir sobreajuste a la experiencia reciente. Para mitigar estos efectos y estabilizar empíricamente la optimización, DQN emplea dos componentes estructurales [MKS⁺15]:

- **Redes Objetivo (*Target Networks*):** Se emplea una red neuronal secundaria separada cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular los valores objetivo en el error TD, amortiguando las oscilaciones y estabilizando el blanco de aprendizaje.
- ***Replay buffer* (memoria de repetición de experiencias):** almacena las transiciones históricas del agente (s, a, r, s') . El entrenamiento se realiza extrayendo *mini-*

batches aleatorios de esta memoria, lo cual reduce la correlación temporal intrínseca de las trayectorias y aproxima el entrenamiento a la hipótesis de muestras independientes. Este mecanismo no constituye, por sí mismo, una garantía de convergencia con aproximadores no lineales.

La unidad básica de aprendizaje es una transición $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$: estado observado, fase ejecutada, recompensa recibida, nuevo estado e indicador de terminación. El objetivo de la actualización combina la recompensa inmediata con una estimación del valor futuro; si la transición es terminal, no se incorpora valor posterior. Así, la red no aprende a imitar una etiqueta de fase, sino a aproximar qué decisiones conducen a mejores retornos según la experiencia acumulada.

La Figura 2.14 distingue el circuito de interacción del circuito de entrenamiento. Durante la interacción, la red en línea transforma la observación del tránsito en un valor para cada fase disponible y una regla ϵ_{exp} -greedy selecciona la acción. La transición resultante se almacena en el búfer. Durante el entrenamiento, el *minibatch* extraído del búfer se utiliza para calcular el objetivo con la red objetivo y la pérdida que actualiza la red en línea; solo los parámetros θ se modifican en cada paso, mientras que θ^- se reemplaza periódicamente por una copia de θ .

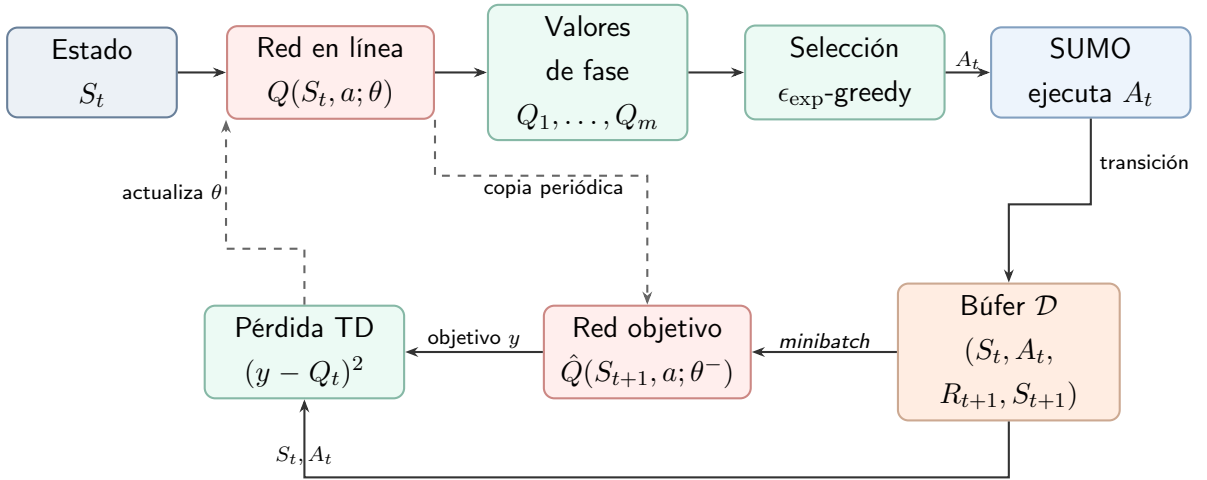


Figura 2.14: Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del blanco TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS⁺15].

La función de pérdida minimizada por DQN, representada en el bloque inferior de la

Figura 2.14, es la de la ecuación (2.15):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción actual}} \right)^2 \right] \quad (2.15)$$

En esta ecuación, $\mathcal{D}$ es el búfer de repetición de experiencias y la esperanza se aproxima en la práctica mediante el promedio sobre un *minibatch*. $(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1})$ representa una transición muestreada; d_{j+1} vale uno cuando la transición es terminal y cero en caso contrario. θ indica los parámetros de la red Q actual, θ^- representa los parámetros congelados de la red objetivo, γ es el factor de descuento y a' recorre las fases admisibles en el estado siguiente. El subíndice j solo identifica una muestra del *minibatch*: no representa un instante temporal adicional.

En términos intuitivos, la pérdida de la ecuación (2.15) acerca la predicción $Q(s_j, a_j; \theta)$ al blanco construido con la recompensa y el valor futuro estimado por la red objetivo. La red objetivo y el muestreo de transiciones desde $\mathcal{D}$ buscan reducir oscilaciones y correlaciones durante el ajuste; no cambian el objetivo de RL, que sigue siendo mejorar el retorno esperado.

Los cuatro bloques numerados como algoritmos en este capítulo son pseudocódigos: descripciones ordenadas e independientes de un lenguaje de programación, no fragmentos ejecutables. En ellos, una Transición registra lo ocurrido en un paso; un Episodio agrupa pasos desde el reinicio hasta la terminación; y un *rollout* es la secuencia de transiciones recolectada con una política. Una Iteración de entrenamiento comprende la recolección de datos y su optimización, mientras que una *epoch* de entrenamiento es una reutilización completa de esos datos dentro de la misma iteración. Cada actualización puede calcularse sobre un *minibatch*, es decir, un subconjunto de las muestras. Las líneas “Entrada” y “Salida” indican, respectivamente, qué información necesita el procedimiento y qué componentes entrenados produce; las líneas indentadas pertenecen al ciclo inmediatamente superior. En lo sucesivo se conservan los términos ingleses *epoch* y *minibatch*, habituales en la literatura, junto con esta definición en español.

El procedimiento conceptual de DQN combina la recolección de transiciones, la selección de acciones mediante una política ϵ_{exp} -greedy y la actualización de la red a partir de *minibatches* extraídos del *replay buffer*. En esa regla de selección, con probabilidad ϵ_{exp} se explora una fase al azar y, con probabilidad $1 - \epsilon_{\text{exp}}$, se elige la fase admisible con mayor valor estimado. El Algoritmo 1 resume este ciclo sin imponer decisiones particulares de implementación:

Algoritmo 1 Deep Q-Network con repetición de experiencias

Entrada: entorno, fases admisibles, capacidad N , horizonte T en pasos de decisión, γ y ϵ_{exp}

Salida: parámetros entrenados θ de la red de acción-valor

```
1: Inicializar el búfer de experiencias  $\mathcal{D}$  con capacidad  $N$ 
2: Inicializar la red de acción-valor  $Q(s, a; \theta)$  con parámetros aleatorios
3: Inicializar la red objetivo  $\hat{Q}(s, a; \theta^-)$  con  $\theta^- \leftarrow \theta$ 
4: para episodio = 1, 2, ... hacer
5:   Inicializar el estado  $s_t$ 
6:   para  $t = 1, 2, \dots, T$  hacer
7:     Seleccionar  $a_t$  mediante una política  $\epsilon_{\text{exp}}$ -greedy basada en  $Q(s_t, a; \theta)$ 
8:     Ejecutar  $a_t$  y observar la recompensa  $r_{t+1}$ , el estado  $s_{t+1}$  y si el episodio terminó
9:     Almacenar  $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$  en  $\mathcal{D}$ 
10:    si  $\mathcal{D}$  contiene al menos un minibatch completo entonces
11:      Extraer un minibatch de transiciones de  $\mathcal{D}$ 
12:      Calcular  $y_j = r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$ 
13:      Actualizar  $\theta$  minimizando  $(y_j - Q(s_j, a_j; \theta))^2$ 
14:    fin si
15:    Actualizar periódicamente la red objetivo:  $\theta^- \leftarrow \theta$ 
16:     $s_t \leftarrow s_{t+1}$ 
17:    si  $d_{t+1} = 1$  entonces
18:      Terminar el ciclo interno y comenzar un nuevo episodio
19:    fin si
20:  fin para
21: fin para
```

El algoritmo se lee en dos ciclos. El ciclo interno alterna una decisión en el entorno y una actualización de la red a partir de experiencias almacenadas; el externo reinicia el entorno al comienzo de cada episodio. Su entrada operacional es el vector s_t con las condiciones de tránsito de la intersección, y la acción a_t es una acción admisible (que representa una fase semafórica). La recompensa r_{t+1} evalúa el efecto de esa decisión sobre demoras o colas, y d_{t+1} detiene el episodio cuando finaliza la ejecución o se alcanza una condición terminal. La salida del entrenamiento no es una secuencia fija de fases, sino el conjunto de parámetros θ que permite estimar sus valores en nuevas observaciones. La capacidad N , el horizonte T , el tamaño de los *minibatches*, ϵ_{exp} y la frecuencia de copia de la red objetivo son decisiones de implementación, por lo que sus valores se especifican en la metodología y no forman parte de la definición del algoritmo.

Aproximación de Políticas, Función de Ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s; \theta)$ [ACS24]. En un problema de acciones discretas, la red recibe el estado u observación, calcula un puntaje para cada acción y transforma esos puntajes en una distribución de probabilidad, habitualmente mediante una función *softmax*. En este trabajo, las acciones del agente corresponden a las fases semafóricas admisibles: las observaciones de tránsito ingresan a la red y sus salidas representan probabilidades sobre dicho conjunto de acciones. La acción puede muestrearse de esa distribución durante el entrenamiento, preservando la exploración estocástica.

La Figura 2.15 muestra este recorrido para un controlador semafórico. A diferencia de DQN, las salidas no son estimaciones de valor: son probabilidades que suman uno. La señal de ventaja no entra a la red para seleccionar la acción; se utiliza durante el entrenamiento para modificar los parámetros en la dirección indicada por el gradiente de la log-probabilidad de la acción observada.

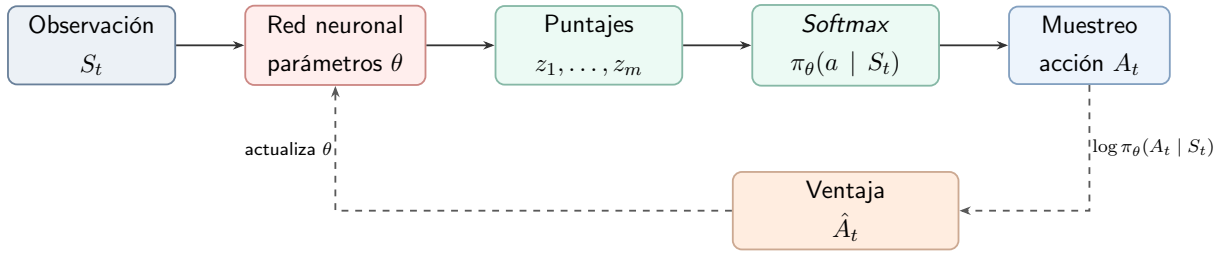


Figura 2.15: Red de política para un espacio discreto de acciones. La observación determina una distribución sobre las acciones disponibles; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].

Para entender la actualización de PPO conviene distinguir dos estimaciones. El valor de estado $V^\pi(s)$ resume qué retorno se espera, en promedio, desde una situación; el valor de acción $Q^\pi(s, a)$ considera además la acción concreta. La diferencia entre ambos indica si una acción resultó mejor o peor que lo habitual en esa observación. Esa diferencia se denomina **Función de Ventaja** (*Advantage Function*) y se define mediante la ecuación (2.16) [SB18]:

$$A^\pi(s, a) \doteq Q^\pi(s, a) - V^\pi(s) \quad (2.16)$$

En esta ecuación, $Q^\pi(s, a)$ representa el retorno esperado al tomar la acción a en el estado s bajo la política π , y $V^\pi(s)$ es el valor promedio de estado. Conceptualmente, la ventaja mide cuánto mejor ($A^\pi(s, a) > 0$) o peor ($A^\pi(s, a) < 0$) resulta la acción a en comparación con el desempeño medio del agente en el estado s .

En arquitecturas actor-crítico, la ventaja no suele calcularse esperando siempre el final de una ejecución. La GAE (*Generalized Advantage Estimation*), desarrollada por Schulman et al. [SML⁺16], combina errores temporales sucesivos para establecer un compromiso

entre sesgo y varianza del estimador. La ecuación (2.17) formaliza este estimador:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} \doteq \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad (2.17)$$

En esta ecuación, $\delta_t^V = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t)$ representa el error de TD estimado por la red del crítico V_ϕ , d_{t+1} indica si la transición es terminal, $\gamma \in [0, 1]$ es el factor de descuento y $\lambda \in [0, 1]$ es el hiperparámetro de suavizado GAE. El índice l cuenta cuántos pasos posteriores se incorporan. La suma infinita expresa el caso conceptual; en una ejecución finita se corta al terminar la trayectoria o al finalizar el segmento recolectado. Los errores posteriores se atenúan mediante $(\gamma \lambda)^l$: variar λ modifica el compromiso entre sesgo y varianza, sin garantizar por sí solo una mejora en ambos. La expresión utiliza explícitamente el mismo crítico parametrizado que interviene en el objetivo de valor y permite incorporar el efecto acumulado de una acción sobre las condiciones posteriores del tránsito.

En lugar de minimizar un error cuadrático de predicción del actor, la optimización paramétrica de la política usa la ventaja para decidir hacia dónde cambiar sus parámetros. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) expresa este principio mediante el gradiente del rendimiento esperado [SB18]. El operador $\hat{\mathbb{E}}_t$ denota el promedio empírico sobre las transiciones del lote recolectado en los instantes t :

$$\nabla_\theta J(\theta) \approx \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(A_t | S_t) \hat{A}_t \right] \quad (2.18)$$

En esta expresión, $\nabla_\theta J(\theta)$ representa el gradiente del desempeño esperado, $\pi_\theta(A_t | S_t)$ es la probabilidad otorgada a la acción ejecutada A_t en el estado S_t , y $\hat{A}_t$ es la ventaja estimada. La esperanza empírica de la ecuación (2.18) es un estimador muestral del gradiente, no una igualdad exacta del teorema para una muestra finita. Como ilustra la Figura 2.15, el término $\nabla_\theta \log \pi_\theta(A_t | S_t)$ indica cómo modificar los parámetros para cambiar la probabilidad de la acción observada, mientras que $\hat{A}_t$ determina el signo y la magnitud de esa modificación. En promedio, el estimador aumenta la probabilidad de elegir acciones con ventaja positiva y reduce la de aquellas con ventaja negativa.

Proximal Policy Optimization (PPO) y control de las actualizaciones

Una dificultad conocida de los métodos de gradiente de política tradicionales (como REINFORCE o *Advantage Actor-Critic* (método actor-crítico basado en una estimación de ventaja) (A2C)) es su sensibilidad a la tasa de aprendizaje: actualizaciones de parámetros con pasos demasiado grandes pueden modificar la distribución de la política de forma abrupta, con riesgo de deterioro del desempeño o de inestabilidad durante el aprendizaje.

Para limitar el riesgo de actualizaciones demasiado grandes, Kakade y Langford [KL02] y Schulman et al. (*Trust Region Policy Optimization* (optimización con una restricción de

región de confianza) (TRPO) [SLA⁺15]) estudiaron métodos de región de confianza. Bajo supuestos específicos y una optimización que respeta la restricción de divergencia entre políticas, el procedimiento teórico de TRPO puede establecer una mejora monótona del retorno:

$$J(\pi_{k+1}) \geq J(\pi_k) \quad (2.19)$$

La ecuación (2.19) corresponde al procedimiento teórico de TRPO y depende de sus supuestos; en ella, $J(\pi_k)$ representa el desempeño esperado de la política en la iteración k . La TRPO implementa esta idea mediante una restricción explícita sobre la divergencia de Kullback–Leibler (KL). Por su parte, la PPO [SWD⁺17] utiliza un objetivo sustituto recortado (CLIP) computable con métodos de primer orden. El recorte limita el incentivo a realizar cambios excesivos para las muestras consideradas, pero no constituye una región de confianza, no impone una cota sobre el cambio total de la política y no garantiza una mejora monótona en cada actualización.

Dada la razón de probabilidad entre la política actual y la anterior:

$$r_t(\theta) \doteq \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \quad (2.20)$$

En la ecuación (2.20), $\pi_\theta(a_t \mid s_t)$ es la probabilidad de la acción a_t bajo los nuevos parámetros θ , y $\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ es la probabilidad registrada durante la recolección del lote. Esta razón compara, para la misma transición, la política que se está optimizando con la política que generó los datos. PPO maximiza el siguiente objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t \right) \right] \quad (2.21)$$

La ecuación (2.21) define el objetivo CLIP utilizado para limitar las modificaciones de la política. En esta expresión, $\epsilon_{\text{clip}} > 0$ es el margen de recorte y $\text{clip}(r, 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}})$ acota la razón r al intervalo $[1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}]$. El subíndice distingue este hiperparámetro de ϵ_{exp} , empleado para la exploración de DQN. El operador $\min$ toma el valor más pesimista entre la ventaja ponderada simple y la ventaja recortada. Intuitivamente, si una fase fue favorable, PPO reduce el incentivo a aumentar demasiado su probabilidad de una sola vez; si fue desfavorable, reduce el incentivo a disminuirla de manera excesiva para las muestras del *minibatch*. Esto no equivale a una región de confianza ni a una cota del cambio total de la política, y tampoco garantiza una mejora monótona.

En la práctica, el objetivo del actor de PPO se combina con una pérdida de valor y, opcionalmente, con un término de entropía para favorecer la exploración. Si se expresa todo como un objetivo a maximizar, una forma compacta es:

$$J^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \hat{\mathbb{E}}_t \left[(V_\phi(s_t) - V_t^{\text{target}})^2 \right] + c_2 \hat{\mathbb{E}}_t [S[\pi_\theta](s_t)] \quad (2.22)$$

En esta expresión, ϕ representa los parámetros de la red del crítico V_ϕ , V_t^{target} es el retorno objetivo estimado para ajustar el crítico, $S[\pi_\theta](s_t)$ es la entropía de la distribución de la política en s_t , c_1 pondera el error de valor y c_2 pondera el incentivo a conservar exploración, con $c_1, c_2 > 0$. La ecuación (2.22) reúne la mejora de la política, el ajuste del crítico y la exploración en el mismo entrenamiento. Si se maximiza J^{PPO} , los parámetros se ajustan mediante ascenso de gradiente; de forma equivalente, puede aplicarse descenso de gradiente sobre $-J^{\text{PPO}}$. El objetivo de RL sigue siendo maximizar el retorno generado por las recompensas del entorno, no minimizar una etiqueta externa.

PPO es un algoritmo predominantemente *on-policy* que reutiliza de forma limitada el lote recolectado con la política vigente. En esta investigación constituye la base conceptual de IPPO, MAPPO y HAPPO; las diferencias entre estas variantes se explican en la sección multiagente [YVV⁺22, KCW⁺22].

El Algoritmo 2 resume el ciclo conceptual de PPO. La recolección de trayectorias se realiza con la política vigente y el lote se reutiliza durante un número limitado de *epochs*; posteriormente se descarta para obtener nuevas muestras con la política actualizada:

Algoritmo 2 Proximal Policy Optimization (PPO)

Entrada: entorno, horizonte de recolección, *epochs* K , γ , λ_{GAE} y ϵ_{clip}

Salida: parámetros entrenados θ del actor y ϕ del crítico

- 1: Inicializar los parámetros del actor θ y del crítico ϕ
 - 2: Inicializar la política de recolección con $\theta_{\text{old}} \leftarrow \theta$
 - 3: **para** iteración = 1, 2, ... **hacer**
 - 4: Recolectar trayectorias con la política $\pi_{\theta_{\text{old}}}$
 - 5: Calcular los retornos objetivo y las ventajas estimadas $\hat{A}_t$
 - 6: **para** *epoch* = 1 hasta K **hacer**
 - 7: Muestrear *minibatches* del conjunto de trayectorias recolectado
 - 8: Calcular $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
 - 9: Actualizar θ maximizando $L^{\text{CLIP}}(\theta)$
 - 10: Actualizar ϕ minimizando el error cuadrático del valor
 - 11: Incorporar, si corresponde, un término de entropía para favorecer la exploración
 - 12: **fin para**
 - 13: $\theta_{\text{old}} \leftarrow \theta$
 - 14: **fin para**
-

El lote se recolecta con una copia fija de la política y se reutiliza solo dentro de esa iteración. Por eso, θ_{old} debe conservar las probabilidades con las que se generaron las acciones hasta completar los K *epochs*; recién entonces se reemplaza por la política actualizada. La entrada de cada iteración es un *rollout* que contiene observaciones de tránsito, fases seleccionadas, recompensas y estados siguientes. El actor utiliza esas observaciones

para producir probabilidades sobre las fases, mientras que el crítico estima el retorno esperado y permite calcular las ventajas. La salida son los parámetros actualizados θ y ϕ ; durante la ejecución del controlador solo el actor es necesario para seleccionar una fase. Esta secuencia explica por qué PPO es predominantemente *on-policy*, aun cuando cada lote se use más de una vez.

2.3 Aprendizaje por Refuerzo Multi-Agente (MARL)

El MARL generaliza los principios del RL hacia sistemas donde múltiples entidades aprenden e interactúan simultáneamente en un entorno compartido [ACS24]. A diferencia del caso individual, las acciones de cada agente modifican el estado global del sistema, lo que induce una dinámica no estacionaria desde la perspectiva local de cualquier participante. En el contexto de sistemas dinámicos cooperativos, múltiples controladores coordinan sus decisiones para optimizar un objetivo común.

Liang et al. [LML⁺25] proponen una clasificación operativa de los problemas multi-agente según la relación entre sus funciones de recompensa: completamente cooperativos, cuando los agentes comparten el objetivo; completamente competitivos, cuando la mejora de uno implica el deterioro de otro; y mixtos, cuando coexisten intereses alineados y contrapuestos. Esta distinción es relevante porque separa la estructura de incentivos del algoritmo utilizado. El presente trabajo se ubica en la primera categoría: las intersecciones actúan como agentes independientes durante la ejecución, pero las políticas se entrenan para reducir una señal de demora compartida en la red.

En una intersección aislada, el control puede analizarse mediante un único agente. Sin embargo, en una red vial la fase seleccionada en una intersección modifica las corrientes que llegan a otras, por lo que los controladores quedan acoplados por la dinámica del tránsito. Esta dependencia espacial motiva la extensión del aprendizaje por refuerzo hacia un escenario multi-agente.

Por ejemplo, cuando una fase libera vehículos en una intersección, modifica los arribos y las colas de la intersección vecina. Cada controlador observa principalmente su entorno local, pero sus acciones forman una acción conjunta y pueden evaluarse mediante una recompensa común asociada al desempeño de la red. Este vínculo concreto motiva los formalismos de juego estocástico y Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*) (Dec-POMDP) que se presentan a continuación.

2.3.1 Fundamentos Teóricos: Juegos Estocásticos, Equilibrio de Nash y Dec-POMDP

La formalización matemática de los entornos multiagente extiende el MDP individual e incorpora nociones de la teoría de juegos. Cuando las acciones de los agentes modifican conjuntamente los estados y las recompensas, el modelo individual deja de describir adecuadamente la interacción. En ese caso, las decisiones secuenciales se representan mediante un juego estocástico o juego markoviano (*Stochastic Game* o *Markov Game*), introducido por Shapley [Sha53] y adaptado al aprendizaje por refuerzo por Albrecht et al. [ACS24].

Un Juego Estocástico para n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, \{R_i\}_{i=1}^n, \gamma \rangle \quad (2.23)$$

En esta tupla, $\mathcal{N} = \{1, \dots, n\}$ representa el conjunto de agentes, $\mathcal{S}$ es el espacio de estados globales, $\mathcal{A}_i$ es el espacio de acciones del agente i , $P(s' \mid s, \mathbf{a})$ representa la función de transición de estados condicionada a la acción conjunta $\mathbf{a} = (a_1, \dots, a_n) \in \mathcal{A} \equiv \mathcal{A}_1 \times \dots \times \mathcal{A}_n$, y $R_i(s, \mathbf{a})$ representa la función de recompensa asignada al agente i . En un entorno estrictamente cooperativo, todos los agentes comparten exactamente la misma señal de recompensa ($R_1 = R_2 = \dots = R_n = R$), de modo que el objetivo de cada agente es maximizar el retorno global esperado del sistema.

En el control semafórico de una red, el conjunto de agentes puede corresponder a las intersecciones controladas, mientras que cada espacio de acciones contiene las fases válidas de una intersección. El estado global y la transición representan la evolución conjunta de la red, que puede ser simulada en SUMO, y R_i resume la señal de aprendizaje asociada con el desempeño del controlador i . La ecuación (2.23) resume esta estructura conjunta.

En la teoría de juegos, el **Equilibrio de Nash** es un concepto de solución que caracteriza la estabilidad frente a desviaciones unilaterales [ACS24]. Una política conjunta $\boldsymbol{\pi}^* = (\pi_1^*, \dots, \pi_n^*)$ constituye un Equilibrio de Nash si ningún agente $i \in \mathcal{N}$ puede incrementar unilateralmente su retorno esperado desviándose de su política π_i^* , asumiendo que los demás agentes mantienen sus políticas fijas en $\boldsymbol{\pi}_{-i}^*$:

$$J(\pi_i, \boldsymbol{\pi}_{-i}^*) \leq J(\pi_i^*, \boldsymbol{\pi}_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \quad (2.24)$$

La ecuación (2.24) expresa la ausencia de una mejora unilateral. En esta ecuación, $\boldsymbol{\pi}^*$ denota la política conjunta en equilibrio; π_i representa cualquier política alternativa ejecutable por el agente i y Π_i denota su espacio de políticas válidas. El término $\boldsymbol{\pi}_{-i}^*$ reúne las políticas fijas de todos los agentes excepto i . En un problema cooperativo de recompensa común, este concepto describe estabilidad, pero no garantiza por sí mismo que la política conjunta maximice el retorno global. Por esa razón, el objetivo operativo de este trabajo se formula como la maximización del retorno conjunto, mientras que el equilibrio

de Nash se utiliza como concepto de análisis de convergencia.

Cuando los agentes de un sistema de control distribuido no disponen de observabilidad global, cada participante recopila información únicamente de sus sensores locales. Por consiguiente, los entornos cooperativos multi-agente con observabilidad parcial se formalizan matemáticamente mediante el marco de los Dec-POMDP [AOS20].

Un Dec-POMDP para n agentes se define como una tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^n, P, R, \{\Omega_i\}_{i=1}^n, O, \gamma \rangle \quad (2.25)$$

En esta tupla, P describe la transición del estado global, R es la recompensa común, γ descuenta las recompensas futuras, Ω_i denota el conjunto de observaciones individuales del agente i y O es la función de observación probabilística. Esta última puede escribirse como $O(o_1, \dots, o_n \mid s', \mathbf{a}) = \Pr\{O_1 = o_1, \dots, O_n = o_n \mid S' = s', \mathbf{A} = \mathbf{a}\}$. En este esquema, cada agente i basa sus decisiones en su observación local $o_i \in \Omega_i$ o en una representación de su historial de observaciones, con el propósito de aprender una política descentralizada $\pi_i(a_i \mid o_i)$ que, al combinarse con las políticas de los demás agentes en la política conjunta $\boldsymbol{\pi} = (\pi_1, \dots, \pi_n)$, maximice el retorno acumulado conjunto del sistema [AOS20]. En el modelo de este trabajo, o_i contiene observaciones locales de tránsito, a_i es una fase de la intersección i y R corresponde a la recompensa común asociada principalmente con la demora de la red; la ecuación (2.25) formaliza esa ejecución descentralizada.

La Figura 2.16 representa esta factorización: cada agente recibe su observación local y la recompensa común ilustrada como r_i , produce una acción local a_i y todas las acciones forman la acción conjunta $\mathbf{a}$ que modifica el entorno. El subíndice de r_i identifica el canal asociado al agente, no una recompensa distinta en la formulación cooperativa de este trabajo.

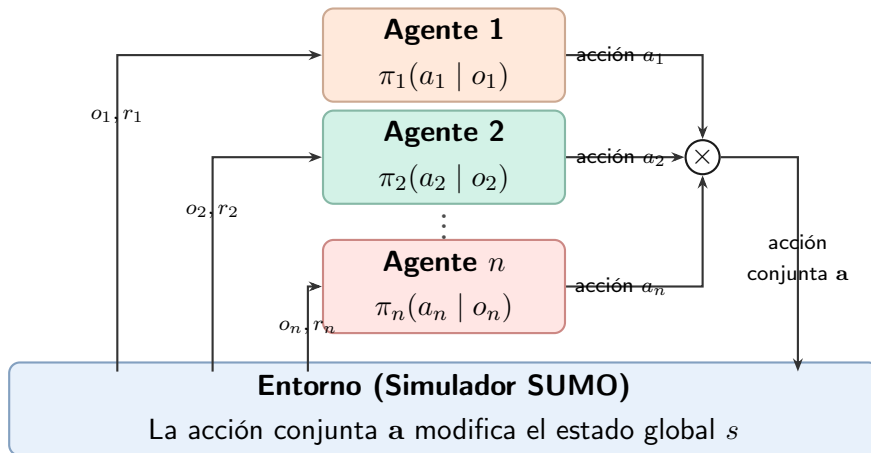


Figura 2.16: Interacción Agente-Entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

La Figura 2.16 muestra cómo las acciones locales de los controladores se combinan

antes de modificar el estado global de la red. En particular, cada política transforma su observación o_i en una acción a_i ; el operador central reúne esas acciones en $\mathbf{a}$, que es la entrada conjunta del entorno. Las etiquetas r_i representan la misma recompensa común observada desde cada canal y no objetivos independientes.

2.3.2 Desafíos del Aprendizaje Multi-Agente

En el aprendizaje multi-agente surgen dificultades que no se presentan en el caso de agente único [ACS24]. Una de ellas es la escalabilidad del espacio de acciones conjuntas. Si cada agente i dispone de un espacio local $\mathcal{A}_i$, el número de combinaciones posibles queda dado por:

$$|\mathcal{A}| = \prod_{i=1}^n |\mathcal{A}_i|, \quad (2.26)$$

donde $\mathcal{A}$ es el espacio de acciones conjuntas, $\mathcal{A}_i$ es el espacio de acciones del agente i y n es el número total de agentes. La ecuación (2.26) muestra que cada agente agrega un factor multiplicativo al número de combinaciones. Por tanto, aunque los espacios locales mantengan un tamaño fijo, un método que represente o evalúe explícitamente todas las acciones conjuntas enfrenta un crecimiento combinatorio a medida que aumenta el número de agentes. Esta observación describe el tamaño del espacio de búsqueda; no implica que todo método centralizado enumere explícitamente sus combinaciones ni constituye, por sí sola, un resultado experimental.

Además, la interacción entre las políticas introduce dificultades propias del aprendizaje multi-agente:

- **No estacionariedad del entorno:** en un entorno multi-agente, la función de transición percibida por el agente i ,

$$\mathcal{P}_i(s' \mid s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' \mid s, a_i, \mathbf{a}_{-i}) \boldsymbol{\pi}_{-i}(\mathbf{a}_{-i} \mid s) \quad (2.27)$$

cambia a lo largo del entrenamiento. La ecuación (2.27) muestra esta dependencia de las políticas de los demás agentes. Aquí $\mathbf{a}_{-i}$ representa las acciones conjuntas del resto de los agentes y $\boldsymbol{\pi}_{-i}$ sus respectivas políticas. Aunque el sistema global puede continuar describiéndose mediante un juego markoviano, la dinámica percibida por un agente local se vuelve no estacionaria cuando las políticas de los demás agentes cambian y no forman parte de su observación. En consecuencia, las condiciones de convergencia de los algoritmos de agente único dejan de ser suficientes [FNF⁺17, LML⁺25].

Este fenómeno fue formalizado por Bowling y Veloso [BV02] como el problema del blanco móvil (*moving target*): una política que es óptima frente a los demás participantes en un momento dado puede dejar de serlo cuando estos se adaptan.

En juegos matriciales como Piedra, Papel o Tijera, el aprendizaje independiente con tasas de actualización fijas puede producir oscilaciones alrededor del equilibrio. Como ilustra la Figura 2.17, con algoritmos como *Win or Learn Fast Policy Hill-Climbing* (PHC con tasas distintas según el desempeño) (WoLF-PHC) (*Win or Learn Fast Policy Hill-Climbing*) las trayectorias de ambos agentes amortiguan sus oscilaciones y se aproximan al equilibrio de Nash, donde cada acción se selecciona con probabilidad uniforme $\pi^*(a) = 1/3$.

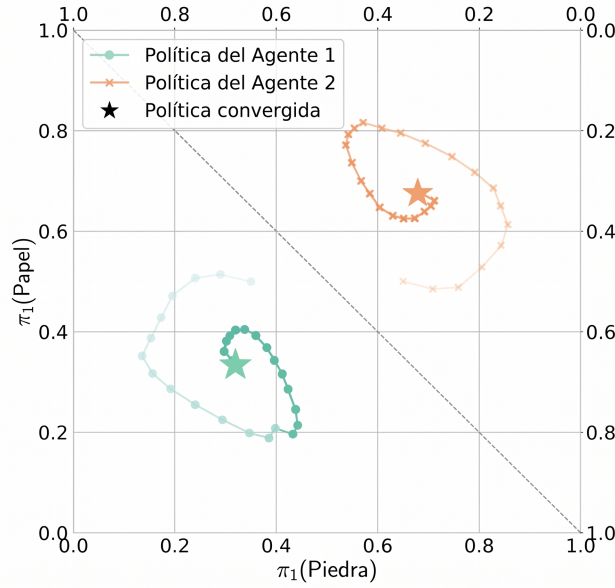


Figura 2.17: Evolución de las políticas de dos agentes con el algoritmo WoLF-PHC en el juego de Piedra, Papel o Tijera. La diagonal discontinua divide el gráfico en dos simplex de probabilidad, uno para cada agente. Cada punto en el simplex representa una distribución sobre las tres acciones disponibles; las trayectorias muestran el avance de las políticas durante el entrenamiento y los marcadores centrales destacan la política convergida en el equilibrio de Nash ($\pi^*(a) = 1/3$ para cada acción). Adaptado de [BV02, ACS24].

Para leer la Figura 2.17, cada región triangular debe interpretarse como el espacio de Políticas (Aprendizaje por Refuerzo) de uno de los agentes. Dos probabilidades se muestran sobre los ejes y la tercera queda determinada por la condición de que las tres sumen uno. Los puntos sucesivos de cada trayectoria representan actualizaciones de la política, no acciones individuales del juego. Las espirales muestran que la respuesta de un agente desplaza el objetivo de aprendizaje del otro; su contracción hacia los marcadores centrales ilustra estabilización alrededor de la distribución uniforme. Esta figura ofrece un ejemplo conceptual de no estacionariedad y no constituye un resultado experimental del control semafórico desarrollado en este trabajo.

En el control semafórico urbano, este fenómeno puede manifestarse de forma direc-

ta: cuando un semáforo modifica sus tiempos de verde para desahogar un acceso, altera los arribos y las colas de las intersecciones adyacentes aguas abajo. Si cada controlador se entrena de manera independiente asumiendo un entorno estacionario, las políticas pueden sobreajustarse a la dinámica observada o responder de forma reactiva y descoordinada.

- **Observabilidad parcial:** dado el modelo Dec-POMDP, la falta de telemetría global restringe la visión de cada agente a variables locales. Esta información incompleta aumenta la incertidumbre de las estimaciones y dificulta el aprendizaje preciso de las funciones de valor.
- **Asignación de crédito multiagente (*Credit Assignment*):** en juegos de recompensa común, resulta complejo determinar la contribución marginal de la acción de un agente sobre el desempeño global. Una decisión local subóptima podría recibir un refuerzo positivo si el resto del sistema operó de manera eficiente, lo que sesgaría las actualizaciones de la política [ACS24].

2.3.3 Paradigmas de Entrenamiento y Ejecución

Para abordar estos desafíos en escenarios Dec-POMDP, la literatura clasifica los algoritmos en función de cómo fluye la información durante el proceso de optimización. Para el análisis de este trabajo se distinguen dos enfoques: el IL (*Independent Learning*) y el Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*) (CTDE) [ZLYZ23].

En el primero, los agentes se entrenan localmente utilizando métodos de agente único, tratando empíricamente al resto de los agentes como parte de la dinámica del entorno. En el segundo, durante el entrenamiento, los algoritmos pueden explotar información global privilegiada (historiales conjuntos o concatenación de observaciones) para guiar y estabilizar la optimización de las funciones de valor. Durante la ejecución, el componente centralizado no participa en la selección de acciones y los actores utilizan únicamente sus observaciones locales. Esta separación entre información disponible durante el entrenamiento y durante la ejecución constituye la característica central de CTDE.

En esta investigación, el paradigma CTDE se entiende conceptualmente como una separación entre dos disponibilidades de información: durante el entrenamiento puede utilizarse el s (estado global) o información conjunta para orientar la evaluación y la coordinación; durante la ejecución, cada Actor (Aprendizaje por Refuerzo) selecciona una acción a_i (correspondiente a una fase semafórica) únicamente a partir de su o_i (observación local). La Figura 2.18 resume esta división, sin fijar una arquitectura concreta de Crítico (Aprendizaje por Refuerzo) ni detalles de implementación. En esta figura, los números 1 y 2 indican etapas del uso del modelo; el término *etapa* no debe confundirse con una fase semafórica.

2.3.4 Aprendizaje por refuerzo profundo multi-agente (*Multi-Agent Deep Reinforcement Learning*) (MADRL) y líneas de base

La integración de los formalismos Dec-POMDP con la capacidad de aproximación generalizada del aprendizaje profundo ha dado origen al campo del MADRL. *Multi-Agent Deep Deterministic Policy Gradient* (extensión multiagente de DDPG) (MADDPG) constituye un antecedente influyente del patrón de críticos centralizados durante el entrenamiento y actores descentralizados durante la ejecución: extiende el algoritmo de gradiente determinista de política monoagente *Deep Deterministic Policy Gradient* (gradiente determinista de política con redes profundas) (DDPG) hacia múltiples agentes mediante críticos entrenados con información conjunta [LWT⁺17]. Esta área reúne enfoques para el control adaptativo de sistemas complejos a gran escala.

En esta investigación, para evaluar rigurosamente el desempeño de las arquitecturas de entrenamiento centralizado (MAPPO y HAPPO), resulta metodológicamente conveniente establecer **líneas de base de aprendizaje independiente (IL)**. En los algoritmos independientes, cada controlador actúa como un agente aislado que no modela la presencia explícita de los demás participantes.

Aprendizaje Independiente (IL): IDQN e IPPO como Líneas de Base

- **IDQN como línea de base basada en valor:** Representa una extensión directa de la DQN en la que cada agente mantiene su propia función de acción-valor [TMK⁺17]. Como método *off-policy*, puede reutilizar transiciones históricas mediante un *replay buffer*; en MARL, esas transiciones fueron generadas mientras los demás agentes seguían políticas posiblemente distintas. Este desfase puede agravar la no estacionariedad y las dificultades de estabilidad del aprendizaje basado en valor [FNF⁺17]. En la evaluación experimental de este trabajo, IDQN sirve como línea de base independiente para los métodos basados en valor.
- **IPPO como línea de base basada en política:** Asigna una instancia de PPO a cada agente. Al utilizar lotes recolectados con las políticas vigentes y descartar la memoria histórica acumulativa, reduce la exposición a transiciones generadas bajo políticas antiguas, aunque no elimina la no estacionariedad causada por el aprendizaje simultáneo [dWGM⁺20]. El recorte de PPO limita cambios excesivos en cada actualización, pero no ofrece una garantía general de convergencia. IPPO constituye la línea de base independiente principal frente a los algoritmos coordinados.

La Figura 2.18 ilustra esta separación entre entrenamiento y ejecución: el estado global y el crítico centralizado aparecen solo en el primer circuito, mientras que los actores seleccionan acciones a partir de observaciones locales en el segundo.

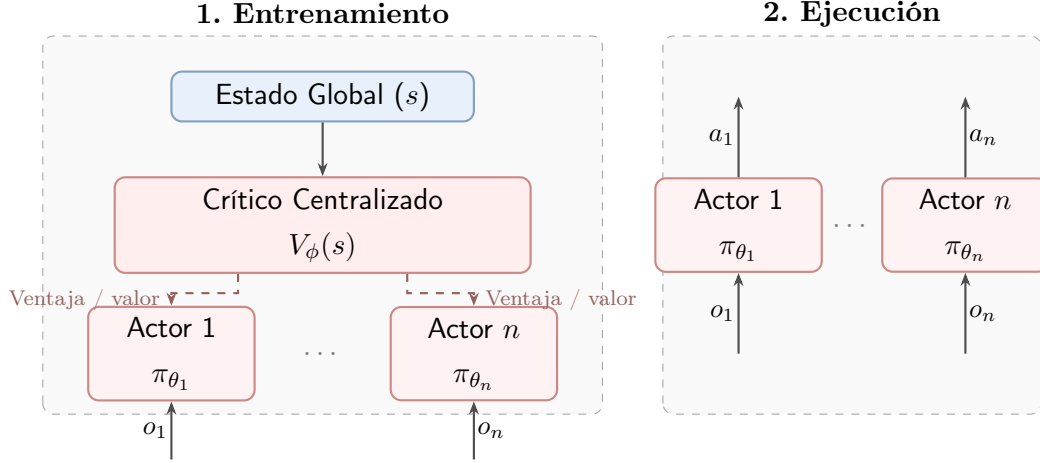


Figura 2.18: Paradigma de Entrenamiento Centralizado y Ejecución Descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.

La mitad izquierda de la Figura 2.18 representa el entrenamiento: las observaciones locales $o_1, \dots, o_n$ alimentan a los actores, mientras que el estado global s se reserva para el crítico centralizado $V_\phi(s)$. Las flechas discontinuas indican que sus estimaciones de valor o ventaja orientan la actualización de las políticas. La mitad derecha representa la ejecución: el crítico y el estado global desaparecen del circuito, y cada actor transforma su observación local o_i en una acción a_i . Esta separación permite entrenar con información adicional sin exigir comunicación global para seleccionar las acciones durante la operación.

2.3.5 Multi-Agent Proximal Policy Optimization (MAPPO)

En una red cooperativa, cada política local debe decidir con observaciones de su propia intersección, aunque la calidad de esa decisión dependa también de los efectos sobre la red. MAPPO aborda esta necesidad mediante el entrenamiento de políticas locales con información global disponible durante el entrenamiento, mientras conserva la selección descentralizada de acciones durante la ejecución.

Para estudiar cómo una configuración de PPO puede funcionar en tareas cooperativas, Yu et al. [YVV⁺22] evaluaron empíricamente la variante conocida como *Multi-Agent Proximal Policy Optimization* (MAPPO), adaptando PPO al entorno multiagente bajo el paradigma CTDE (véase la Figura 2.18). Su contribución principal fue sistematizar prácticas de implementación y evaluar su efecto, no proponer desde cero un algoritmo independiente de PPO.

En MAPPO, la estructura distingue entre actores descentralizados y un crítico centralizado:

- **Actores descentralizados ($\pi_{\theta_i}(a_i | o_i)$):** Cada agente i mantiene una política que procesa su observación local o_i y produce una distribución sobre las acciones lo-

cales a_i . En entornos homogéneos puede utilizarse compartición de parámetros para mejorar la eficiencia de aprendizaje, pero esta práctica no constituye un requisito conceptual de MAPPO.

- **Crítico centralizado ($V_\phi(s)$):** Durante el entrenamiento, una red de función de valor puede recibir el estado global s del entorno o una representación construida a partir de observaciones conjuntas. El crítico estima el retorno y proporciona valores para calcular ventajas mediante GAE; durante la ejecución no es necesario disponer de este componente [YVV⁺22].

En la formulación práctica de MAPPO, el objetivo del actor se expresa como un promedio de términos PPO individuales. La política conjunta se factoriza en políticas locales, pero la pérdida no debe interpretarse como una única razón de probabilidad conjunta:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t), \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \quad (2.28)$$

donde $r_t^i(\theta) = \frac{\pi_{\theta_i}(a_t^i | o_t^i)}{\pi_{\theta_{i,\text{old}}}(a_t^i | o_t^i)}$ es la razón de probabilidad individual del agente i , n es el número total de agentes, $\hat{A}_t$ es la ventaja utilizada por la implementación y ϵ_{clip} es el margen de recorte. En entornos con recompensa común, esta ventaja puede calcularse utilizando un crítico centralizado; la forma exacta depende de la definición del estado y de la implementación. La ecuación (2.28) conecta las observaciones locales y las acciones individuales con la señal compartida de demora que orienta la coordinación.

Por su parte, el crítico centralizado se actualiza minimizando el error cuadrático medio respecto al retorno objetivo descontado normalizado:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right] \quad (2.29)$$

La ecuación (2.29) ajusta los parámetros ϕ del crítico para aproximar el retorno conjunto observado. En esta ecuación, R_t^{target} representa el objetivo de retorno calculado a partir de las recompensas compartidas y de los valores predichos por el propio crítico.

Factores Críticos de Diseño y Optimización en MAPPO

A pesar de su simplicidad conceptual, Yu et al. [YVV⁺22] demostraron empíricamente que la efectividad de MAPPO depende fuertemente de varias decisiones de diseño e hiperparámetros:

1. **Normalización del valor:** La normalización de los retornos puede mejorar el ajuste del crítico cuando la escala de las recompensas cambia durante el aprendizaje. Yu et al. analizan esta decisión junto con otras prácticas de implementación; PopArt es una alternativa posible, no una condición necesaria de MAPPO [YVV⁺22].

2. **Representación del Estado Global en el Crítico:** Yu et al. [YVV⁺22] evaluaron distintas opciones para estructurar la entrada s del crítico centralizado:

- Concatenación de Observaciones Locales (*Concatenated Local, CL*): forma el estado uniendo las observaciones locales de todos los agentes $s = (o_1, \dots, o_n)$.
- Estado Global del Entorno (*Environment Provided, EP*): utiliza un vector de estado global provisto directamente por el simulador.
- Estado Específico por Agente (*Agent-Specific State, AS*): combina el estado del entorno con la información local del agente i ($AS_i = EP \cup o_i$).
- *Feature Pruning* (filtrado de características redundantes, *FP*): remueve variables locales duplicadas o de baja varianza en el vector AS_i .

El estudio encuentra que las representaciones específicas por agente y el filtrado de redundancias pueden ser útiles en determinados bancos de prueba. Esta evidencia debe trasladarse al escenario de tránsito mediante evaluación, no como una regla universal.

3. **Reutilización restringida de datos:** La cantidad de reutilización de los datos recolectados y la forma de particionar los lotes pueden afectar la estabilidad del aprendizaje. Se trata de decisiones sensibles a la dificultad del problema, no de garantías de estabilidad para cualquier red vial; su evaluación corresponde a la metodología.
4. **Recorte y tamaño de lote:** El rango de recorte y el tamaño de los lotes pueden modificar el equilibrio entre estabilidad, variabilidad y aprovechamiento de las muestras. Estos efectos deben evaluarse en la metodología y no se interpretan como una garantía general de desempeño.

El Algoritmo 3 resume la recolección de trayectorias, el cálculo centralizado de ventajas y la actualización de los actores locales y del crítico compartido.

Algoritmo 3 Algoritmo *Multi-Agent Proximal Policy Optimization* (MAPPO)

Entrada: entorno multiagente, n agentes, horizonte de recolección, $epochs$ K , γ , λ_{GAE}
y ϵ_{clip}

Salida: parámetros entrenados de los actores θ y del crítico centralizado ϕ

- 1: Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
- 2: Inicializar el búfer de datos $\mathcal{D} \leftarrow \emptyset$
- 3: **para** iteración = 1, 2, ... **hacer**
- 4: Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta π_θ
- 5: Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
- 6: Calcular los retornos objetivos R_t^{target} normalizados para el crítico
- 7: Almacenar las transiciones $(s_t, \mathbf{o}_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
- 8: **para** $epoch = 1$ hasta K **hacer**
- 9: Muestrear *minibatches* de datos desde $\mathcal{D}$
- 10: Actualizar la política maximizando el objetivo de recorte PPO por agente:
 $\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^n \min \left(r_t^i(\theta) \hat{A}_t, \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t \right) \right]$
- 11: Actualizar el crítico ϕ minimizando el error cuadrático medio del valor:
 $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$
- 12: **fin para**
- 13: Vaciar el búfer de datos $\mathcal{D} \leftarrow \emptyset$
- 14: **fin para**

En este algoritmo, la información global se concentra en el cálculo del crítico y de la ventaja durante el entrenamiento. Cada actor, en cambio, conserva una política que recibe su propia observación; por ello puede seleccionar una acción sin requerir el estado global cuando se ejecuta el controlador. El vaciado de $\mathcal{D}$ al final de cada iteración evita tratar este conjunto como una memoria histórica y mantiene el carácter predominantemente *on-policy* del procedimiento.

En términos operativos, la entrada de cada iteración es un conjunto de *rollouts*, es decir, secuencias de transiciones obtenidas al interactuar con el entorno bajo la política conjunta vigente. En cada instante, el agente i recibe su observación local o_t^i , selecciona la acción a_t^i y, junto con las demás acciones, produce una transición con recompensa común r_t , siguiente estado y siguientes observaciones. Cada *rollout* conserva también el estado global s_t disponible para el crítico, las acciones conjuntas y la información necesaria para evaluar las probabilidades de las políticas antiguas. A partir de estas transiciones, el crítico calcula los retornos objetivo y GAE estima $\hat{A}_t$; después, los *minibatches* se utilizan para actualizar las políticas locales mediante la razón r_t^i y el objetivo recortado de la ecuación (2.28), mientras ϕ se ajusta con la pérdida de valor de la ecuación (2.29). La salida de

la iteración son los nuevos parámetros de los actores y del crítico, junto con un nuevo conjunto de *rollouts* que reemplaza al anterior.

La arquitectura de actores locales, el crítico centralizado y el esquema de entrenamiento centralizado con ejecución descentralizada son componentes generales del método. En cambio, K , ϵ_{clip} , la tasa de aprendizaje, el tamaño de *minibatch*, la cantidad de pasos del *rollout*, el factor de descuento, el parámetro de GAE, la normalización y la elección de optimizador son decisiones metodológicas de una implementación concreta. Estas últimas deben informarse y evaluarse en la metodología, sin presentarlas como requisitos conceptuales ni como garantías de desempeño de MAPPO.

2.3.6 *Heterogeneous-Agent Proximal Policy Optimization* (HAPPO)

En sistemas de control complejos, los agentes pueden presentar asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En entornos homogéneos, compartir parámetros puede mejorar la eficiencia de aprendizaje; en entornos heterogéneos, esa práctica puede limitar la especialización. Además, las actualizaciones independientes no poseen una garantía general de convergencia debido al aprendizaje simultáneo no coordinado [KCW⁺22].

Kuba et al. y Zhong et al. [KCW⁺22, ZKF⁺24] propusieron algoritmos de aprendizaje por refuerzo multiagente para políticas heterogéneas, sin exigir que todos los agentes compartan parámetros. Esta línea se denomina aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*, HARL). Cada agente i puede mantener su propia arquitectura y parámetros θ^i , adaptados a su espacio de acciones local $\mathcal{A}_i$. En esta línea, *Heterogeneous-Agent Trust Region Policy Optimization* (optimización con región de confianza para agentes heterogéneos) (HATRPO) y HAPPO emplean una actualización secuencial de los agentes. Los resultados teóricos de mejora monótona y convergencia se refieren al procedimiento de región de confianza bajo supuestos explícitos; la variante práctica HAPPO sustituye la restricción KL por un objetivo recortado y debe interpretarse como una aproximación [KCW⁺22, ZKF⁺24].

Compartición de Parámetros y Políticas Heterogéneas

Una distinción central entre MAPPO y HAPPO radica en la homogeneidad de las políticas y en el orden de actualización:

- **Compartición de parámetros:** En MAPPO se utiliza habitualmente cuando los agentes poseen espacios de observación y acción compatibles. Todos los agentes pueden compartir los mismos pesos $\theta_1 = \dots = \theta_n = \theta$, aunque la formulación de MAPPO también puede implementarse sin esta restricción.
- **Políticas heterogéneas y actualización secuencial:** HAPPO permite que cada agente mantenga una política independiente $\pi_{\theta^i}(a_i | o_i)$ y actualiza los actores siguiendo

iendo una permutación. El lote de trayectorias de la iteración actual se reutiliza de forma limitada; esto no equivale a un búfer histórico de *experience sharing* como el empleado por algoritmos específicos de compartición de experiencia.

Lema de Descomposición de la Ventaja Multi-Agente

El soporte matemático de HAPPO se fundamenta en el Lema de Descomposición de la Ventaja Multi-Agente propuesto por Kuba et al. [KCW⁺22].

En cualquier juego markoviano cooperativo, para una política conjunta π y una permutación ordenada arbitraria de los agentes $i_{1:n} = (i_1, i_2, \dots, i_n) \in \text{Sym}(n)$, la ventaja de una acción conjunta $\mathbf{a} = (a_{i_1}, \dots, a_{i_n})$ en el estado s se descompone exactamente como la suma de las ventajas marginales de cada agente tomadas secuencialmente, condicionadas por las acciones de los agentes que lo preceden en el ordenamiento:

$$A^\pi(s, \mathbf{a}) = \sum_{m=1}^n A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \quad (2.30)$$

donde $A^\pi(s, \mathbf{a})$ representa la ventaja conjunta global, y la ventaja marginal del agente i_m se define formalmente como:

$$A_{i_m}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}}) \quad (2.31)$$

donde $Q_{i_{1:m}}^\pi(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ representa el valor esperado de la acción conjunta considerando las acciones fijadas por los primeros m agentes en la secuencia, y $Q_{i_{1:m-1}}^\pi(s, \mathbf{a}_{i_{1:m-1}})$ es el valor marginalizado sobre los agentes restantes. La ecuación (2.30), junto con la definición marginal de (2.31), permite asignar secuencialmente la contribución de cada acción de agente a la ventaja conjunta.

En esta formulación, las acciones de los agentes que aún no han actualizado su política en la secuencia $(\mathbf{a}_{i_{m+1:n}})$ se marginalizan por esperanza matemática bajo la política conjunta vigente. La gran relevancia teórica de este lema es que se cumple para cualquier juego cooperativo de recompensa común sin requerir ningún supuesto de aditividad o monotonía sobre la función de valor, a diferencia de los esquemas de descomposición de valor como *Value-Decomposition Networks* (redes de descomposición del valor conjunto) (VDN) [SLG⁺18] o *Q-Mixing Network* (red que combina valores de acción individuales) (QMIX) [RSDW⁺18].

Esquema de Actualización Secuencial y Factor de Razón Compuesto M

Para explotar la descomposición de la ventaja sin incurrir en la inestabilidad del entrenamiento simultáneo, HAPPO actualiza las políticas de los agentes de manera secuencial dentro de cada iteración de optimización k .

En cada iteración de entrenamiento k , se genera una permutación de agentes $i_{1:n} \in \text{Sym}(n)$. En el análisis teórico, asignar probabilidad no nula a los órdenes posibles es una

condición que interviene en el resultado asintótico de convergencia; en una implementación finita, la aleatorización del orden reduce sesgos de prioridad, pero no constituye por sí sola una garantía [KCW⁺22].

Cuando le corresponde el turno de optimización al agente i_m , los agentes anteriores en la secuencia ($i_{1:m-1}$) ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores ($i_{m+1:n}$) conservan sus parámetros originales $\theta_k^{i_j}$. Para guiar la actualización del agente i_m respetando la modificación sufrida por la distribución conjunta debido a los agentes precedentes, Kuba et al. [KCW⁺22] demostraron que no es necesario entrenar un crítico separado por cada subgrupo, sino que la ventaja marginal ajustada se puede calcular multiplicando la ventaja conjunta $A^{\pi_k}(s, \mathbf{a})$ por la productoria de los ratios de probabilidad de los agentes que ya se actualizaron.

Se define así la ventaja conjunta ponderada por los ratios de los agentes ya actualizados:

$$M_{i_{1:m}}(s_t, \mathbf{a}_t) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}^{i_j}}^{i_j}(a_t^{i_j} | o_t^{i_j})}{\pi_{\theta_k^{i_j}}^{i_j}(a_t^{i_j} | o_t^{i_j})} \right) A^{\pi_k}(s_t, \mathbf{a}_t) \quad (2.32)$$

Donde $\pi_{\theta_{k+1}^{i_j}}^{i_j}$ es la política recién optimizada del agente precedido i_j , $\pi_{\theta_k^{i_j}}^{i_j}$ es su política previa, y $A^{\pi_k}(s_t, \mathbf{a}_t)$ es la ventaja conjunta en la transición considerada. Para el primer agente de la secuencia ($m = 1$), la productoria es vacía y se define como $M_{i_1}(s_t, \mathbf{a}_t) \equiv A^{\pi_k}(s_t, \mathbf{a}_t)$. Por tanto, M no es únicamente un cociente de probabilidades: es una señal de ventaja reponderada.

El factor $M_{i_{1:m}}(s, \mathbf{a})$ de la ecuación (2.32) actúa como un modificador de importancia que ajusta la señal de ventaja para el agente i_m , absorbiendo analíticamente el desfase en la distribución de la acción conjunta provocado por las actualizaciones precedentes sin necesidad de realizar nuevas simulaciones en el entorno.

De este modo, para optimizar la política del agente i_m , HAPPO maximiza el objetivo sustituto recortado de PPO ponderado por la razón compuesta $M_{i_{1:m}}$:

$$L^{\text{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \left[\min \left(r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \right. \right. \\ \left. \left. \text{clip}(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) M_{i_{1:m}}(s_t, \mathbf{a}_t) \right) \right] \quad (2.33)$$

La ecuación (2.33) aplica el recorte de PPO a la ventaja reponderada de HAPPO. En ella, $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente i_m . El parámetro ϵ_{clip} es el mismo margen de recorte definido para PPO en la ecuación (2.21); aquí limita la actualización de cada política heterogénea.

Una vez realizada la actualización de descenso de gradiente sobre θ^{i_m} , es indispensable recalcular la razón de probabilidad post-optimización $\rho_{\text{post}}^{i_m} = \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ y actualizar

recursivamente el factor M para los agentes restantes mediante:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t) \quad (2.34)$$

La ecuación (2.34) propaga hacia los agentes restantes el cambio producido por la actualización del agente i_m .

Garantía de Mejora Monótona

El trabajo de Kuba et al. demuestra una propiedad de mejora monótona para un procedimiento de iteración de políticas multiagente con restricciones de región de confianza y bajo supuestos explícitos [KCW⁺22]. La variante práctica HAPPO reemplaza la restricción KL por un objetivo recortado, por lo que esta garantía no se transfiere automáticamente a cada actualización de una implementación neuronal:

En el procedimiento teórico, la actualización secuencial satisface la garantía:

$$J(\boldsymbol{\pi}_{k+1}) \geq J(\boldsymbol{\pi}_k) \quad (2.35)$$

donde $J(\boldsymbol{\pi}_k)$ representa el rendimiento esperado de la política conjunta en el paso k . La ecuación (2.35) depende de las hipótesis del procedimiento analítico y no significa que HAPPO-clip elimine las oscilaciones ni que una ejecución finita con redes neuronales converja necesariamente a un equilibrio de Nash. En esta investigación, HAPPO se evalúa como una estrategia que busca mejorar la coordinación y la estabilidad frente a actualizaciones independientes.

La Figura 2.19 ilustra el orden secuencial de actualización que distingue a HAPPO de una actualización independiente simultánea. En ella, M representa una señal de ventaja reponderada por los cambios de los agentes precedentes, no un cociente de probabilidad aislado.

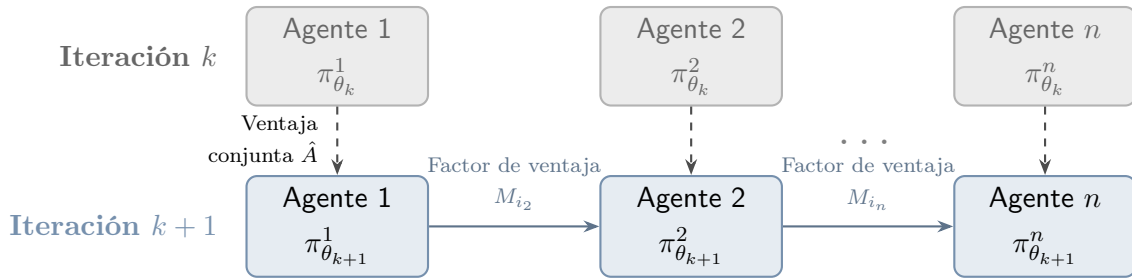


Figura 2.19: Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m} , que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.

La Figura 2.19 se lee de arriba hacia abajo y luego de izquierda a derecha. La fila superior contiene las Políticas (Aprendizaje por Refuerzo) antes de la actualización, en la

iteración k , y la fila inferior sus versiones actualizadas en $k + 1$. La primera flecha vertical muestra que el agente 1 se actualiza a partir de la ventaja conjunta $\hat{A}$. A continuación, las flechas horizontales transportan el factor M_{i_m} , que corrige esa señal para reflejar las actualizaciones ya realizadas antes de optimizar al agente siguiente. Los puntos suspensivos representan la repetición del mismo procedimiento para los agentes intermedios; no una actualización simultánea. Esta lectura visual corresponde a la recurrencia de la ecuación (2.34).

El Algoritmo 4 resume la actualización secuencial de las políticas heterogéneas y la propagación recursiva del factor M .

Algoritmo 4 Algoritmo *Heterogeneous-Agent Proximal Policy Optimization* (HAPPO)

Entrada: entorno multiagente, conjunto $\mathcal{N}$ de n agentes, horizonte de recolección, γ , λ_{GAE} y ϵ_{clip}

Salida: políticas heterogéneas entrenadas $\{\theta^i\}_{i=1}^n$ y crítico centralizado ϕ

- 1: Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
- 2: **para** iteración = 1, 2, ... **hacer**
- 3: Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta π_θ
- 4: Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
- 5: Calcular los retornos objetivo R_t^{target} utilizados para ajustar el crítico
- 6: Sortear una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
- 7: Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \dots, n\}$
- 8: **para** $m = 1 \dots n$ **hacer**
- 9: Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
- 10: Actualizar θ^{i_m} a $\theta_{k+1}^{i_m}$ maximizando el objetivo recortado de la ecuación (2.33)
- 11: Recalcular la razón post-optimización: $\rho_{\text{post}}^{i_m} \leftarrow \frac{\pi_{\theta_{k+1}^{i_m}}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}(a_t^{i_m} | o_t^{i_m})}$
- 12: **para** $j = m + 1 \dots n$ **hacer**
- 13: Actualizar el factor modificador para agentes subsecuentes:

$$M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_j}(s_t, \mathbf{a}_t)$$
- 14: **fin para**
- 15: **fin para**
- 16: Actualizar el crítico centralizado ϕ mediante descenso de gradiente:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$$
- 17: **fin para**

La diferencia esencial respecto de MAPPO está en el ciclo sobre m : antes de actualizar a cada agente, se incorpora en M el efecto de las políticas que ya fueron modificadas en esa misma iteración. Los retornos objetivo se calculan antes de modificar los actores y, una vez

completada su actualización secuencial, el crítico se ajusta para aproximar esos retornos. El orden aleatorio y el objetivo recortado describen una implementación práctica; no deben confundirse con las condiciones de región de confianza que sustentan el resultado teórico de mejora monótona.

De forma más detallada, la entrada de una iteración de HAPPO es el *rollout* $\mathcal{D}$ recolectado al comienzo con las políticas $\{\pi_{\theta_k^i}\}_{i=1}^n$. Cada registro contiene las observaciones locales, las acciones individuales y conjuntas, el estado global, la recompensa común y la transición siguiente. El crítico centralizado recibe s_t para obtener $\hat{A}(s_t, \mathbf{a}_t)$ mediante GAE; los actores solo utilizan sus respectivas observaciones y acciones para formar las razones de probabilidad. A continuación se sortea la permutación $i_{1:n}$ y se recorre el ciclo sobre m . Antes del turno del agente i_m , su política es la versión previa $\pi_{\theta_k^{i_m}}^{i_m}$; tras la optimización, pasa a ser $\pi_{\theta_{k+1}^{i_m}}^{i_m}$. Los agentes anteriores ya están en su versión posterior y los que siguen permanecen en la versión previa. Por eso, el factor inicial $M_{i_1} = \hat{A}$ se transforma para cada agente posterior mediante los cocientes de las políticas posteriores y previas de los agentes ya procesados, como expresa la ecuación (2.32).

Después de optimizar θ^{i_m} con la ecuación (2.33), se evalúa sobre el *rollout* la razón posterior a la optimización $\rho_{\text{post}}^{i_m}$, usando la política posterior en el numerador y la política previa en el denominador. Esta razón no es el mismo parámetro metodológico que ϵ_{clip} : es una cantidad calculada a partir de la actualización efectivamente realizada y se incorpora a M mediante la ecuación (2.34). Así, la salida del turno es la política actualizada del agente y un factor de ventaja modificado para los turnos restantes; la salida de la iteración es el conjunto de políticas nuevas y el crítico actualizado. El orden de la permutación, ϵ_{clip} , la tasa de aprendizaje, el tamaño de los *minibatches*, los *epochs*, el tamaño del *rollout*, la normalización y los parámetros de GAE son decisiones metodológicas, mientras que la heterogeneidad, la actualización secuencial y la reponderación de M son componentes generales de HAPPO. El recorte utilizado aquí define una aproximación práctica: no permite atribuir a HAPPO-clip, sin los supuestos y la restricción de región de confianza del análisis teórico, la garantía de mejora monótona de la ecuación (2.35).

Para finalizar el estudio comparativo de los algoritmos derivados de PPO en escenarios MARL, la Tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Tabla 2.3: Comparación sintética de familias algorítmicas PPO en entornos multi-agente cooperativos.

Eje de análisis	PPO de agente único	IPPO independiente	MAPPO / CTDE	HAPPO / HARL
Paradigma	Agente Único	IL (Descentralizado)	CTDE (Centralizado/Desc.)	CTDE / HARL
Actores	Único $\pi_\theta(a s)$	n políticas locales	Actores locales; puede compartir parámetros	n políticas heterogéneas
Arquitectura del Crítico	Único $V_\phi(s)$	n Críticos locales $V_{\phi_i}(o_i)$	Un Crítico Centralizado $V_\phi(s)$	Un Crítico Centralizado $V_\phi(s)$
Optimización	PPO estándar	PPO local simultáneo	PPO local con crítico global	Actualización secuencial $i_{1:n}$
Señal de Ventaja	Ventaja local $\hat{A}(s, a)$	Ventaja local $\hat{A}_i(o_i, a_i)$	Ventaja conjunta $\hat{A}(s, \mathbf{a})$	Factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora monótona	No estricta para PPO	Sin garantía general	Sin garantía estricta	Teórica bajo supuestos; HAPPO-clip es aproximado
Tipo de agente	No aplica	Depende de la implementación	Compatible; compartir parámetros es opcional	Diseñada para agentes heterogéneos

2.3.7 Antecedentes recientes del control semafórico multiagente

Para orientar la comparación, los antecedentes seleccionados se organizan según tres ejes: coordinación y entrenamiento, representación de las observaciones, y tratamiento de la heterogeneidad o la equidad entre intersecciones. Se priorizan trabajos directamente relacionados con la formulación y las variables de este trabajo.

El control coordinado de múltiples intersecciones mediante aprendizaje por refuerzo constituye un área de investigación activa. Entre los enfoques iniciales se encuentran aquellos que asignaban un agente a cada semáforo y utilizaban información local sobre colas, fases o vehículos detenidos. Sin embargo, la optimización aislada de cada intersección no contempla que una acción modifica el flujo recibido por los cruces vecinos. Por este motivo, trabajos posteriores incorporan mecanismos de cooperación, representaciones estructuradas del estado y esquemas de entrenamiento centralizado [ACS24, LML⁺25].

Coordinación y entrenamiento

Ault y Sharon [AS22] estudian la aplicación de MARL cooperativo al control de múltiples intersecciones. El trabajo muestra que trasladar métodos de agente único o de aprendizaje independiente a una red coordinada no garantiza un buen desempeño global, ya que cada controlador aprende mientras las políticas de los demás agentes también cambian. Este antecedente fundamenta la necesidad de evaluar explícitamente la cooperación, en lugar de asumir que la suma de controladores locales produce una política adecuada para toda la red.

Wu et al. [WWS⁺22] estudian el control de redes de gran escala mediante agentes distribuidos de aprendizaje por refuerzo profundo y una arquitectura de cómputo distribuido. Proponen dos variantes, Nash-A2C y Nash-A3C, que incorporan un equilibrio de Nash en el aprendizaje por refuerzo y se despliegan en una arquitectura distribuida para simulación y control. En sus experimentos reportan reducciones del tiempo de congestión y del retraso de red frente a controladores de referencia. Este antecedente es pertinente como referencia de escalabilidad, pero no es equivalente a la formulación de este trabajo: aquí la ejecución también es local por intersección, mientras que la coordinación se estudia mediante la recompensa compartida y el entrenamiento centralizado, sin introducir una capa de cómputo fog ni un equilibrio de Nash.

En una formulación cooperativa basada en DQN, Haddad et al. [HHA22] incorporan a la función de aprendizaje de cada intersección información sobre los estados, acciones y recompensas de sus vecinos. El estudio informa mejoras en tiempo de espera, longitud de cola y emisiones de dióxido de carbono frente a controladores de referencia. Esta evidencia respalda la necesidad de considerar el efecto de las decisiones vecinas, pero su mecanismo de coordinación es una transferencia explícita de información hacia la pérdida de DQN; la presente investigación permite contrastarlo con IPPO, MAPPO y HAPPO, donde la coordinación se expresa mediante señales de ventaja y un crítico centralizado.

Fang et al. [FYX⁺23] proponen un método de coordinación de red que formula el control semafórico como un problema multiobjetivo: combina eficiencia, seguridad y coordinación espacial entre intersecciones mediante un esquema de entrenamiento centralizado y ejecución descentralizada. Además de reportar resultados en redes sintéticas y arterias reales, el trabajo destaca que las dependencias espacio-temporales entre agentes deben representarse explícitamente. Su propuesta ofrece un contraste con la recompensa compartida y la representación interpretable utilizadas en este trabajo, que no incorporan una optimización multiobjetivo ni una representación latente basada en atención.

Representación del estado

En cuanto a la representación agregada y el aprendizaje basado en valor, Kolat et al. [KKBA23] proponen un enfoque DQN multiagente con parámetros compartidos. Los agentes observan cantidades escaladas de vehículos detenidos y reciben una recompen-

sa que penaliza el desequilibrio entre los accesos. Los resultados muestran mejoras en el tiempo de viaje y el consumo de combustible respecto de un controlador de ciclo fijo. Sirve como referencia para contrastar una representación agregada del estado y una coordinación inducida por la recompensa con la evaluación de IDQN y de métodos basados en políticas realizada en esta investigación.

Dentro de la línea centrada en la representación del estado, Yang [Yan23] propone HG-M2I, una arquitectura que combina información espacial y temporal mediante un grafo jerárquico. El modelo integra estados de los agentes y de sus vecinos en representaciones latentes utilizadas por las políticas. Posteriormente, Chen et al. [CYL⁺24] presentan CoevoMARL, que utiliza atención en grafos y una unidad recurrente para aprender cómo varía en el tiempo la dependencia entre intersecciones. Mientras HG-M2I organiza la información según una jerarquía espacial, CoevoMARL adapta las relaciones entre agentes a la evolución del tránsito. Ambos trabajos muestran la importancia de construir observaciones informativas, aunque recurren a representaciones latentes aprendidas, a diferencia de la transformación explícita e interpretable evaluada en el presente trabajo.

En el caso de una representación basada en celdas, Deng et al. [DLSZ26] introducen una partición de longitud variable. Los accesos se dividen mediante una función que combina componentes logarítmicos y lineales, y cada celda almacena la cantidad de vehículos, la velocidad media y la ocupación espacial. Esta partición asigna mayor detalle a las zonas próximas a la intersección y comprime las regiones más alejadas. Entre los trabajos seleccionados, es el antecedente más cercano al uso de una transformación logarítmica en la representación semafórica; sin embargo, el logaritmo determina la longitud espacial de las celdas y no codifica el tiempo de espera acumulado por carril.

Heterogeneidad y equidad

Ren et al. [RDZ⁺24] abordan la coordinación bajo demandas heterogéneas mediante un esquema de dos capas. Cada agente combina su ventaja local con la de las intersecciones vecinas a través de un factor de cooperación adaptable, que luego se ajusta según el rendimiento global de la red. El método considera la demora y las emisiones vehiculares, y coordina los objetivos locales y globales mediante ese factor adaptable. Este antecedente permite contrastar la coordinación adaptativa de las señales de aprendizaje con la heterogeneidad de políticas considerada por HAPPO, aunque no emplea su procedimiento de actualización secuencial.

En la coordinación mediante entrenamiento centralizado, Liu et al. [LZF⁺25] formulan el control semafórico como un juego parcialmente observable con restricciones y proponen VF-MAPPO. En este método, la equidad se expresa mediante una cota sobre el tiempo máximo de espera de los vehículos, mientras que un crítico centralizado con atención espaciotemporal estima tanto la eficiencia de la red como el cumplimiento de la restricción. Esta formulación resulta especialmente pertinente para el presente trabajo porque vincula

el paradigma de entrenamiento centralizado y ejecución descentralizada con una variable temporal de operación. No obstante, el tiempo de espera se incorpora como restricción de optimización y no mediante una codificación no lineal de la observación.

En el tratamiento de agentes heterogéneos, otro enfoque es HAPS-PPO, presentado por Lu et al. [LFYZ25]. El método completa con relleno las observaciones de distinta dimensión y agrupa a los agentes según sus espacios de acción, asignando cabezales de política específicos a cada grupo. Esto permite coordinar intersecciones con geometrías y conjuntos de fases diferentes. A pesar de la similitud entre las siglas, HAPS-PPO no corresponde a HAPPO, ya que no emplea la actualización secuencial ni la descomposición de la ventaja explicadas en la Sección 2.3.6.

Posicionamiento del presente trabajo

La selección analizada sugiere que el desempeño del control semafórico multiagente depende de tres decisiones relacionadas: el algoritmo utilizado para aprender y coordinar las políticas, la representación de las condiciones de operación y el tratamiento de intersecciones con demandas, geometrías o espacios de acción diferentes. Los enfoques basados en DQN proporcionan una referencia para el aprendizaje por valor. Los modelos de grafos construyen representaciones latentes de las relaciones espaciales, mientras que las variantes de MAPPO incorporan información global durante el entrenamiento. Los esquemas de cooperación adaptable o con múltiples cabezales atienden distintas formas de heterogeneidad.

Sin embargo, en el conjunto de fuentes y términos consultados no se identificó una aplicación directa de HAPPO al control semafórico ni una evaluación conjunta de IDQN, IPPO, MAPPO y HAPPO bajo una misma formulación del problema.

En particular, este trabajo explora una codificación híbrida logarítmica-lineal del tiempo de espera acumulado por carril. La transformación no elimina componentes del vector de observación, sino que asigna cada valor a un conjunto finito de intervalos con mayor resolución para demoras bajas. Esta decisión se diferencia de las observaciones agregadas utilizadas por los enfoques DQN, de las proyecciones latentes basadas en grafos y de la partición espacial propuesta por Deng et al.

A su vez, la evaluación bajo IDQN, IPPO, MAPPO y HAPPO permite separar el efecto de la representación del correspondiente al mecanismo de aprendizaje y coordinación. De este modo, la solución puede contrastarse con los antecedentes recientes en términos de representación, paradigma de entrenamiento y capacidad para tratar agentes heterogéneos. El capítulo siguiente describe su instanciación en el entorno experimental.

Metodología y Diseño Experimental

Este capítulo describe el diseño experimental, la operacionalización de las variables, la construcción de los escenarios de simulación y la arquitectura computacional utilizada. También presenta los protocolos de entrenamiento y evaluación empleados para contrastar las hipótesis planteadas. El estudio adopta un enfoque cuantitativo, aplicado y experimental de medidas repetidas. La simulación microscópica del tránsito funciona como un entorno controlado para evaluar el desempeño, la coordinación espacial y la robustez de las arquitecturas de MARL.

3.1 Enfoque y Diseño de Investigación

Este trabajo adopta un **diseño experimental cuantitativo de medidas repetidas**. Se manipulan sistemáticamente las familias algorítmicas, el grado de cooperación vecinal ρ_{coop} y las topologías de demanda vial. Sus efectos se miden sobre variables dependientes vinculadas con el desempeño operacional del tránsito, la equidad del servicio y las emisiones.

La simulación microscópica se utiliza como plataforma experimental por tres razones. El simulador utilizado es SUMO [DLR26]:

1. **Seguridad y viabilidad operacional:** evaluar algoritmos estocásticos de aprendizaje en tiempo real sobre una red urbana real implicaría riesgos para la seguridad vial y la fluidez del tránsito.
2. **Aislamiento de variables exógenas:** la simulación permite controlar perturbaciones que no forman parte del modelo, como condiciones meteorológicas, siniestros

viales o fluctuaciones externas.

3. **Replicabilidad:** la fijación de semillas pseudoaleatorias permite someter los distintos controladores a las mismas condiciones de demanda.

Para evaluar rigurosamente el impacto de las políticas aprendidas, se formalizan cuatro variables dependientes primarias a partir de la telemetría microscópica de SUMO (TripInfo):

- **Demora acumulada global (W_{net}):** métrica operacional que agrega la espera de los vehículos detenidos (velocidad $v < 0.1$ m/s) durante la ventana controlada:

$$W_{\text{net}} = \sum_{t=1}^T \sum_{v \in \mathcal{V}_t} w_v(t) \quad (3.1)$$

La ecuación (3.1) suma los aportes de espera de todos los vehículos y pasos de la ventana. En ella, T es el número de pasos temporales de la ventana controlada, $\mathcal{V}_t$ es el conjunto de vehículos activos en la calzada en el paso t y $w_v(t)$ es el aporte de espera del vehículo v durante dicho paso, medido en segundos. Este aporte es la duración del paso en que el vehículo permanece detenido ($v < 0.1$ m/s) y es nulo cuando no cumple esa condición; no representa la espera acumulada desde el inicio de la simulación. Por tanto, W_{net} agrega duraciones individuales y su unidad dimensional es vehículo·segundo (veh·s), es decir, el total de segundos de espera acumulados por todos los vehículos.

- **Tasa de completitud de viajes (*Completion Rate*) (CR):** Proporción de la demanda generada que completa su itinerario dentro del horizonte de evaluación. La métrica también contabiliza los vehículos retenidos en los accesos de origen por sobresaturación:

$$\text{CR} = \frac{\text{throughput}}{\text{departed} + \text{pending_vehicles_end}} \quad (3.2)$$

La ecuación (3.2) relaciona los vehículos completados con la demanda que ingresó o quedó retenida. Todos sus términos son conteos del mismo horizonte: throughput corresponde a los vehículos que completaron su itinerario, departed a los que ingresaron a la calzada y pending_vehicles_end a los retenidos al finalizar. También se registran la Tasa de inserción (*Insertion Rate*) (IR) ($\text{IR} = \frac{\text{inserted}}{\text{loaded}}$) y la Tasa de completitud de los vehículos insertados (*Completion of Inserted Vehicles*) (CI) ($\text{CI} = \frac{\text{throughput}}{\text{inserted}}$).

- **Equidad espacial de demoras (coeficiente de Gini, G_{wait}):** cuantifica la desigualdad en la distribución del tiempo de espera medio entre las intersecciones

semaforizadas de la red:

$$G_{\text{wait}} = \frac{2 \sum_{i=1}^N i \cdot \bar{w}_{(i)} - (N+1) \sum_{i=1}^N \bar{w}_{(i)}}{N \sum_{i=1}^N \bar{w}_{(i)}} \quad (3.3)$$

La ecuación (3.3) se calcula después de ordenar los tiempos de espera medios. En ella, $\bar{w}_{(1)} \leq \bar{w}_{(2)} \leq \dots \leq \bar{w}_{(N)}$ son los tiempos de espera medios ordenados de cada controlador y N es el total de intersecciones. Esta métrica se complementa con la desviación estándar inter-intersección (σ_{wait}) para evaluar si las políticas aprendidas distribuyen equitativamente el servicio o si sacrifican accesos secundarios en favor de arterias dominantes.

- **Emisiones de CO2:** masa total de dióxido de carbono emitida en gramos. Se calcula a partir de los perfiles instantáneos de aceleración y velocidad mediante los modelos microscópicos integrados en SUMO (Modelo microscópico de emisiones de vehículos livianos y pesados (*Passenger car and Heavy duty Emission Model light*) (PHEMlight)/Manual de factores de emisión para transporte por carretera (*Handbook Emission Factors for Road Transport*) (HBEFA) [DLR26]).

La Figura 3.1 resume las siete etapas de la investigación. En este contexto, *etapa* designa una parte del procedimiento metodológico y no una fase del ciclo semafórico.

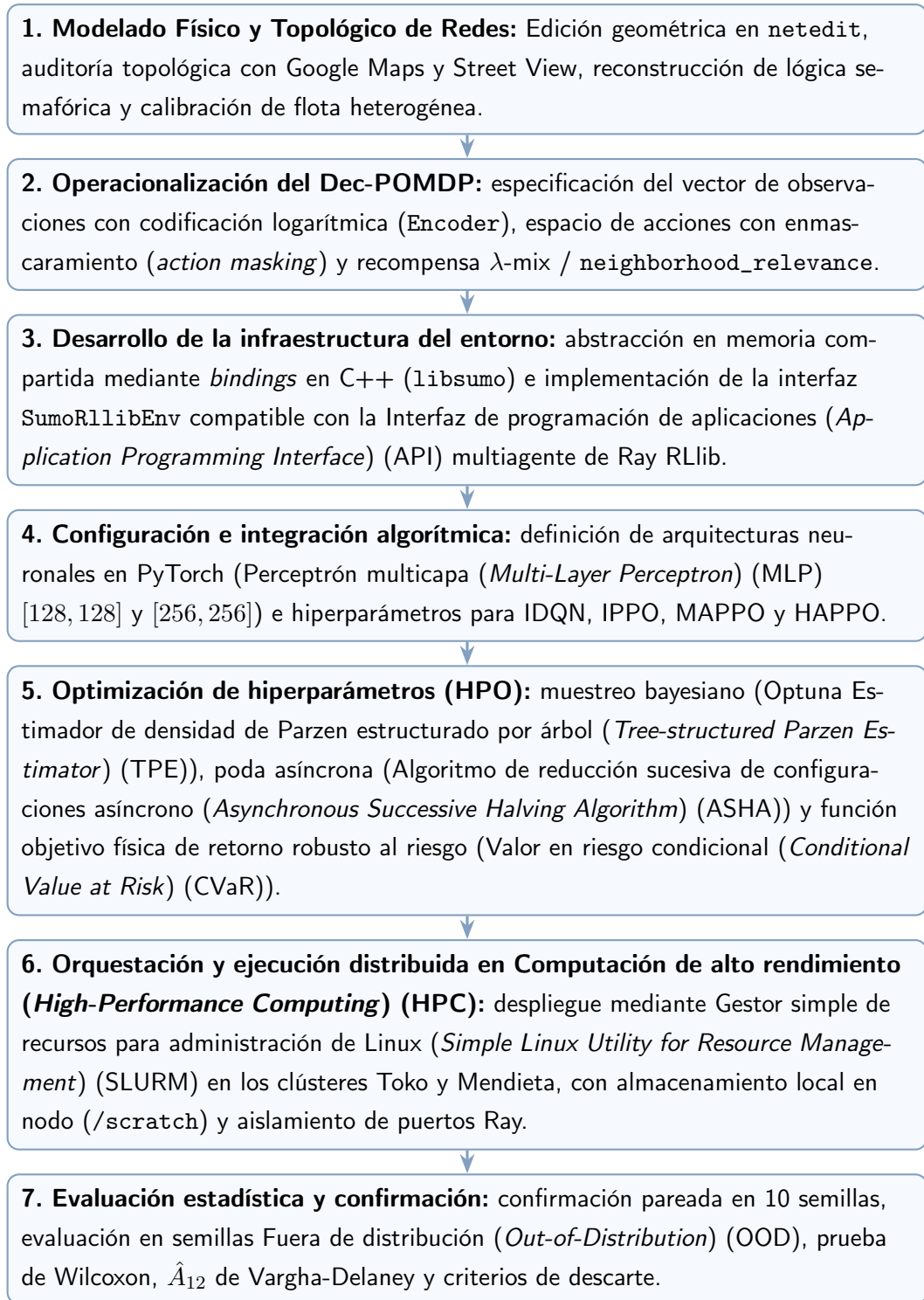


Figura 3.1: Flujo metodológico de la investigación experimental. Los siete recuadros representan, de arriba hacia abajo, las etapas de modelado de redes, formulación del problema, desarrollo del entorno, integración algorítmica, optimización de hiperparámetros, ejecución distribuida y evaluación estadística; las flechas indican la dependencia secuencial entre ellas.

La Figura 3.1 comienza con la construcción física y topológica de las redes que sirven como escenarios experimentales. Sobre esos escenarios se define el problema de decisión:

qué observa cada agente, qué acciones puede ejecutar y qué recompensa recibe. La tercera etapa implementa esas definiciones en el entorno de simulación, y la cuarta incorpora los algoritmos y sus redes neuronales. Una vez que el sistema puede entrenarse, la quinta etapa busca configuraciones de hiperparámetros, es decir, valores de configuración que controlan el proceso de aprendizaje. La sexta distribuye las ejecuciones en la infraestructura de cómputo, y la séptima compara los resultados con pruebas estadísticas. Las flechas verticales expresan que cada etapa necesita los resultados de la anterior; no representan retroalimentación ni ejecución simultánea.

3.2 Especificación del Ecosistema Tecnológico

Para garantizar la reproducibilidad científica, la Tabla 3.1 detalla las librerías, marcos de trabajo y plataformas de cómputo sobre las cuales se desarrolló y ejecutó el entorno `marlTLC`. En esta sección, la telemetría es el conjunto de mediciones que el simulador registra durante la ejecución; un marco de trabajo (*framework*) proporciona interfaces y componentes reutilizables para construir el sistema; un *binding* conecta una biblioteca escrita en un lenguaje con otro, y la serialización convierte un objeto o mensaje en una representación que puede almacenarse o transmitirse. Los procesos denominados *env runners* generan trayectorias al interactuar con el entorno y los *learners* actualizan los parámetros de las redes a partir de esas trayectorias. Estas definiciones permiten interpretar la tabla sin suponer familiaridad previa con la infraestructura de software.

Tabla 3.1: Especificación de las tecnologías y herramientas utilizadas en la investigación.

Componente	Herramienta / Versión	Función Metodológica
Lenguaje de Programación	Python 3.10+	Núcleo del sistema, tipado estático (<code>typing.Protocol</code>) y scripts de orquestación.
Simulador de Tránsito	SUMO 1.20+ / TraCI	Simulación microscópica, dinámica cinemática y telemetría de red.
<i>Bindings</i> de Simulación	<code>libsumo</code> (C++)	Ejecución de la simulación en memoria compartida mediante <i>bindings</i> en C++ (<code>LIBSUMO_AS_TRACI=1</code>).
Marco de trabajo MARL (<i>framework</i>)	Ray RLlib 2.x [LLN ⁺ 18]	Abstracción de entornos multi-agente (<code>MultiAgentEnv</code> , <code>RLModuleSpec</code> , <code>LearnerGroup</code>).
Aprendizaje Profundo	PyTorch 2.x	Modelado neuronal y optimización (Actor-Crítico MLP, <i>action masking</i>).
Optimización Automática	Optuna 3.x [ASY ⁺ 19]	Muestreo Bayesiano TPE y poda asíncrona de pruebas (ASHA [LJR ⁺ 20]).
Infraestructura HPC	Clústeres Toko y Mendieta	Cómputo distribuido multinúcleo gestionado vía SLURM con Entrada/salida (<i>Input/Output</i>) (E/S) local (<i>/scratch</i>).

Estas capas se integran de la siguiente manera. Python coordina la ejecución y Ray RLlib gestiona los procesos concurrentes de recolección de trayectorias (*env runners*). PyTorch calcula las derivadas y optimiza las redes neuronales en los módulos de aprendizaje (*learners*). `SumoRllibEnv` encapsula la lógica episódica y `libsumo` ejecuta los cálculos de cinemática vehicular en la memoria del proceso, sin comunicación por red ni serialización Protocolo de control de transmisión (*Transmission Control Protocol*) (TCP).

3.3 Diseño y Calibración de Escenarios de Tránsito en SUMO

La evaluación empírica se realiza sobre cuatro escenarios de red en SUMO. Las redes se construyen con la herramienta gráfica `netedit`, los utilitarios de conversión `netconvert` y scripts de generación en Python. En conjunto, los escenarios representan distintas complejidades topológicas y condiciones de flujo.

3.3.1 Modelado Geométrico y Sanitización Topológica

La construcción de las redes de simulación sigue un procedimiento de diseño y validación en SUMO:

1. **Diseño geométrico y accesos:** definición de calzadas, cantidad de carriles por aproximación y longitudes de almacenamiento en `netedit` o a partir de datos cartográficos de OSM.
2. **Auditoría y corrección topológica:** verificación de los sentidos de circulación, la cantidad de carriles, las conexiones de giro y las maniobras prohibidas. La revisión se contrasta con imágenes satelitales y vistas de calle de Google Maps y Google Street View.
3. **Configuración semafórica:** las configuraciones de los semáforos estáticos corresponden a los planes generados por defecto por SUMO (`netconvert -tls.rebuild`) al compilar la red. Los programas de fases y los tiempos de ciclo originales se mantienen sin cambios. La correspondencia entre los enlaces físicos y los índices de control semafórico (*linkIndex*) se recalcula para conservar la consistencia topológica.

3.3.2 Escenarios Sintéticos: 3×1 , 3×3 y 4×4 _H

Todas las redes sintéticas comparten las siguientes reglas operacionales:

- **Límite de velocidad:** 13.89 m/s (50 km/h) en todas las arterias urbanas.
- **Flota vehicular heterogénea:** se definieron tres clases de vehículos en el archivo `urban_vehicle_mix.add.xml`, a partir de los modelos cinemáticos disponibles en el simulador [DLR26]:
 1. **SLOWVEHTYPE** (25%): Conductores cautelosos o vehículos pesados con menor aceleración ($a = 1.9 \text{ m/s}^2$, $d = 4.0 \text{ m/s}^2$, $v_{\max} = 8.33 \text{ m/s} = 30 \text{ km/h}$).
 2. **MEDVEHTYPE** (60%): Vehículo de pasajeros promedio representativo. Parámetros: $a = 2.6 \text{ m/s}^2$, $d = 4.5 \text{ m/s}^2$ y $v_{\max} = 13.89 \text{ m/s} = 50 \text{ km/h}$.
 3. **FASTVEHTYPE** (15%): Conductores dinámicos con mayor aceleración desde reposo ($a = 3.1 \text{ m/s}^2$, $d = 5.0 \text{ m/s}^2$, $v_{\max} = 19.44 \text{ m/s} = 70 \text{ km/h}$, acotado a 50 km/h por el límite del carril).

Parámetros compartidos: longitud 4.5 m, brecha mínima de seguridad (*minGap*) 2.5 m e imperfección del conductor de Krauss $\sigma = 0.5$.

- **Generación de demanda por zonas (TAZ) y arribos estocásticos:** las rutas se generan mediante `random_routes_taz.py` a partir de matrices de zonas de asignación de tránsito (TAZ). Para una tasa de flujo *flow* expresada en vehículos por

hora, la tasa de arribos se convierte a segundos mediante $\lambda_{\text{arr}} = \text{flow}/3600$, cuya unidad es veh/s. En cada arribo se sortea un número U de una distribución uniforme en $(0, 1)$ y se obtiene el intervalo H mediante el método de la transformada inversa:

$$H = -\frac{\ln(U)}{\lambda_{\text{arr}}}, \quad U \sim \mathcal{U}(0, 1), \quad \mathbb{E}[H] = \frac{1}{\lambda_{\text{arr}}} = \frac{3600}{\text{flow}} \text{ s} \quad (3.4)$$

De este modo, la ecuación (3.4) produce un intervalo H con distribución exponencial de tasa λ_{arr} ; H representa el tiempo, en segundos, hasta el siguiente arribo. La notación `period="exp(rate)"` del archivo de rutas solicita este muestreo: `exp` no es la evaluación de $e^{\lambda_{\text{arr}}}$ ni el valor esperado del intervalo, sino el identificador de la distribución cuyo parámetro es la tasa de arribos.

- **Calibración por bisección:** los flujos totales de cada red se calibraron mediante el método de bisección hasta alcanzar una tasa de completitud saturada ($\text{CR} \approx 0.70$) bajo control semafórico de tiempo fijo. El horizonte de evaluación fue de 4200 s, compuesto por 600 s de calentamiento y 3600 s de control.

Las tres topologías sintéticas cumplen funciones analíticas complementarias:

- **Corredor arterial 3×1 (3.260 veh/h):** tres intersecciones dispuestas en línea recta con flujo bidireccional dominante (40% este-oeste y 35% oeste-este). Constituye un Grafo acíclico dirigido (*Directed Acyclic Graph*) (DAG) sin lazos de realimentación circular. La propagación de colas (*spillback*) es principalmente lineal, por lo que permite evaluar la sincronización de olas verdes sin la interferencia de cuellos de botella circulares.
- **Malla simétrica 3×3 (7.087 veh/h):** nueve intersecciones organizadas en una cuadrícula cerrada con demandas diagonales cruzadas (37.5% en cada eje). La alta interconexión genera dependencias circulares: la saturación de una intersección central puede bloquear a sus vecinas y producir un desbordamiento de colas (*spillback*) que afecte a la red. Este escenario permite evaluar la estabilidad de MARL frente a estados de congestión severa.
- **Cuadrícula 4×4 (7.202 veh/h):** dieciséis intersecciones asimétricas con dos arterias principales de mayor capacidad. Estas arterias canalizan el tránsito de los nodos interiores hacia los extremos y ofrecen rutas alternativas.

3.3.3 Escenario Urbano Real: Microcentro de la Ciudad de Mendoza

Para contrastar la validez y transferibilidad de las políticas aprendidas frente a condiciones de operación viales reales, se construyó un escenario del microcentro de la Ciudad de

Mendoza a partir de su infraestructura topológica y de series temporales continuas de aforo vehicular captadas mediante cámaras termográficas.

Posición Metodológica: Escenario Restringido por Datos

El escenario de Mendoza se considera un **escenario sintético restringido por datos** (*data-constrained synthetic scenario*), y no un gemelo digital completo.

La inferencia de una matriz Origen-destino (*Origin-Destination*) (OD) completa a partir de volúmenes medidos en pocos puntos de control es un problema inverso sub-determinado y mal condicionado [CyMR18, PW16]. Para una red con $|\mathcal{Z}| = 37$ zonas de asignación de tránsito (TAZ), existen $|\mathcal{Z}|(|\mathcal{Z}| - 1) = 1.332$ pares direccionales potenciales, de los cuales 520 son físicamente alcanzables. Este número contrasta con la cantidad limitada de enlaces instrumentados en el acceso oriental. Por lo tanto, las observaciones disponibles no determinan una única asignación OD.

Por consiguiente, este trabajo utiliza dos criterios para construir la demanda:

1. **Restricción dinámica empírica:** las mediciones continuas de aforo determinan el perfil horario de la demanda total y fijan el flujo observado en el corredor principal de ingreso.
2. **Estructuración espacial híbrida:** la distribución espacial de los viajes combina un conocimiento previo (*prior*) determinista, basado en la jerarquía vial, con un remanente estocástico controlado sobre los pares alcanzables.

Adquisición, Auditoría y Preprocesamiento de Telemetría Empírica

La base empírica de aforo proviene de registros continuos de sensores de conteo y clasificación vehicular, correspondientes a cámaras térmicas Tecnotrans / Tecnología infrarroja con barrido hacia adelante (*Forward Looking Infrared*) (FLIR) ThermICam. Los registros tienen una resolución de 15 minutos y corresponden al período octubre de 2025 – abril de 2026.

En las etapas preliminares se dispuso de información de cuatro dispositivos ubicados en el entorno del nudo de Costanera y Acceso Este. El relevamiento geoespacial mostró que las cámaras 192.168.1.167 (egreso hacia el este), 192.168.1.168 (circulación hacia el norte por Costanera) y 192.168.1.170 (circulación hacia el sur por Costanera) se encuentran entre 600 m y 800 m al este del perímetro de la red modelada. Por lo tanto, miden flujos de la autopista que quedan fuera del modelo vial urbano. En la generación y calibración definitiva se utiliza exclusivamente la Cámara 169 (192.168.1.169), ubicada en el viaducto sobre el canal Cacique Guaymallén, que registra el ingreso hacia el oeste por la Avenida José Vicente Zapata.

Para controlar la calidad de los datos de la Cámara 169, se aplicó un procedimiento de auditoría y saneamiento estructurado en cinco etapas:

1. **Filtrado temporal y estructural:** descarte de marcas temporales corruptas, incluida la fecha anómala 1970-01-01, o fuera del rango válido [2020–2035].
2. **Validación de zonas y clases vehiculares:** admisión de zonas válidas ($\text{ZoneID} \in [1, 19]$) y clasificación física en vehículos livianos ($l \leq 5$ m), medianos ($5 \text{ m} < l \leq 10$ m) y pesados ($l > 10$ m), según $\text{ClassID} \in \{1, 2, 3\}$.
3. **Eliminación de duplicados:** eliminación de registros redundantes mediante la clave ($\text{DateUTC}, \text{ZoneID}, \text{ClassID}$), conservando el último registro ($\text{keep} = \text{'last'}$).
4. **Alineación temporal:** reasignación de marcas con desviaciones $|\Delta t| \leq 5$ s al cuarto de hora más cercano y descarte de registros desalineados (*off-grid*).
5. **Filtro de días hábiles completos:** selección de días hábiles, de lunes a viernes, con cobertura del 100% de los intervalos diarios de 15 minutos. Se utilizó como referencia un umbral mínimo de 95%.

Este protocolo seleccionó 33 días hábiles completos no perturbados, arrojando en la Cámara 169 un volumen empírico diario medio de $\mu_{\text{obs}} = 25.283,85$ veh/día con una desviación estándar interdiaria de $\sigma_{\text{obs}} = 2.316,54$ veh/día.

Mapecto Geoespacial, Corrección Topológica y Señalización

El trazado geométrico base se obtuvo mediante la exportación de datos de OSM y su conversión al formato de red de SUMO (*.net.xml*). El dominio modelado delimita el microcentro de la Ciudad de Mendoza entre Av. José Vicente Zapata / Colón al sur, Av. General San Martín como eje central norte-sur, Av. Manuel Belgrano al oeste y Av. Las Heras al norte. La red se proyecta en coordenadas Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*) (UTM) Zona 19S (Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*) (WGS84)).

Punto de inyección este (TAZ 28): dado que el nudo de Costanera se ubica al este del perímetro del modelo, la Cámara 169 se representa mediante el primer tramo interno disponible de la calzada de ingreso. Se utiliza el enlace (*edge*) 93904440#0, asignado al origen de la zona TAZ 28.

Auditoría y corrección topológica con Google Maps y Street View: tras la conversión inicial desde OSM, se detectaron discrepancias entre la red y la infraestructura real. La red se revisó con imágenes satelitales de Google Maps y vistas de calle de Google Street View:

- **Restricciones de giro y maniobras prohibidas:** se implementaron las restricciones observadas en terreno, como la prohibición de giro a la izquierda desde Av. General San Martín hacia el sur por Av. Colón. También se conservaron los giros semaforizados permitidos, como el de calle Perú hacia Av. Las Heras.

- **Sentidos de circulación y conectividad vial:** se corrigieron los sentidos de circulación de calzadas secundarias que figuraban como bidireccionales o invertidas en OSM. También se restituyó la continuidad de los cuatro carriles de ingreso de la Av. José Vicente Zapata hacia la trama interior (93904440#0 a 93904440#1).
- **Cantidad de carriles y accesos laterales:** se ajustó el número de carriles de aproximación por calzada y se habilitaron las conexiones de inserción en las calles transversales Catamarca, Buenos Aires y Leandro N. Alem.
- **Configuración de semáforos estáticos y señalización:** las configuraciones de los semáforos de tiempo fijo corresponden a los programas generados por defecto por SUMO (`-tls.rebuild`) al compilar la red. Las señalizaciones, las secuencias de fases y las duraciones de ciclo originales se mantuvieron sin cambios. Estos programas constituyen la línea base para evaluar los controladores adaptativos MARL.

Modelado Dinámico de Demanda y Factor de Expansión Territorial

El flujo vehicular en cada paso temporal se modela a partir del perfil horario representativo $\bar{q}_{\text{obs},169}(h)$ ($h \in [0, 23]$), promediado sobre los 33 días hábiles completos. El análisis de distribución temporal revela que el período diurno de 12 horas (06 : 00–18 : 00) concentra el 69,29% del flujo diario total, mientras que la ventana matutina de hora pico (06 : 30–08 : 30) concentra el 12,53%.

Para proyectar el aforo del enlace observado a la escala de la red, se aplica un **factor de expansión territorial dinámico** [CyMR18, PW16]. Este factor se basa en el volumen diario estimado $V_{\text{diario}} = 260.000$ veh/día, adoptado tras una exploración de sensibilidad en el rango 250.000–290.000 veh/día:

$$f_{\text{exp}} = \frac{V_{\text{diario}}}{\sum_{h=0}^{23} \bar{q}_{\text{obs},169}(h)} = \frac{260.000}{25.283,85} \approx 10,2832 \quad (3.5)$$

La ecuación (3.5) compara el volumen diario objetivo con el volumen diario observado y, por ello, produce un factor adimensional. La demanda total de la red en la hora h queda determinada por:

$$Q_{\text{red}}(h) = \bar{q}_{\text{obs},169}(h) \cdot f_{\text{exp}} \quad (3.6)$$

La ecuación (3.6) conserva el perfil horario observado y lo escala al volumen de la red mediante f_{exp} . Se adopta una tasa de retención del detector `camera_retention = 1,0`, dado que la Cámara 169 captura el flujo de aproximación representativo del acceso oriental.

Estructuración espacial de la demanda: contrato OD canónico v14 (60/40)

A partir de las 37 zonas TAZ, se calcularon 520 pares OD físicamente alcanzables mediante Búsqueda en anchura (*Breadth-First Search*) (BFS) sobre el grafo de conectividad vial. La distribución de la demanda horaria $Q_{\text{red}}(h)$ se rige por el **contrato OD canónico**

v14, formulado como una partición híbrida:

$$p_{od} = \begin{cases} p_{od}^{\text{calib}} & \text{si } (o, d) \in \mathcal{P}_{\text{exp}} \quad \text{con} \quad \sum_{(o,d) \in \mathcal{P}_{\text{exp}}} p_{od}^{\text{calib}} = 0,60 \\ 0 & \text{si } (o, d) \in \mathcal{Z}_{\text{struct}} \\ \frac{0,40}{|\mathcal{P}_{\text{libre}}|} \cdot \xi_{od} & \text{si } (o, d) \in \mathcal{P}_{\text{libre}} = \mathcal{P}_{\text{reach}} \setminus (\mathcal{P}_{\text{exp}} \cup \mathcal{Z}_{\text{struct}}) \end{cases} \quad (3.7)$$

La ecuación (3.7) asigna a cada par origen-destino una única de las tres reglas de la partición. En ella, $\mathcal{P}_{\text{exp}}$ es el conjunto de pares explícitos, $\mathcal{Z}_{\text{struct}}$ los ceros estructurales, $\mathcal{P}_{\text{libre}}$ los pares libres alcanzables y $\xi_{od} \geq 0$ es un factor de asignación estocástica. Para que el remanente represente exactamente el 40% de la demanda, estos factores se normalizan mediante $\sum_{(o,d) \in \mathcal{P}_{\text{libre}}} \xi_{od} = |\mathcal{P}_{\text{libre}}|$; si todos los pares reciben el mismo peso, entonces $\xi_{od} = 1$. En consecuencia, la suma de las probabilidades de los pares explícitos positivos y del remanente libre es uno, mientras que los ceros estructurales no reciben demanda.

Componente fijo calibrado (60%): estructurado en 476 entradas de `mendoza_od_proportions.json` (473 pares con cuotas positivas y 3 ceros directos), para reflejar la jerarquía vial observada en Mendoza:

- **Jerarquía de arterias principales:** priorización de los corredores Av. Vicente Zapata (TAZ 28), Av. General San Martín (TAZ 19/29), Av. Colón / Emilio Civit (TAZ 7), Av. Las Heras (TAZ 37/38) y Av. Manuel Belgrano (TAZ 0).
- **Intervención en TAZ 12 (Av. Perú):** reducción teórica del tráfico de origen en -20% , con un desvío realizado de $-0,47\%$ respecto de la cuota objetivo de 0,03657. La intervención reduce la sobrecarga sobre calle San Lorenzo (*edge* 249618899) y evita el colapso temprano de los giros de distribución.
- **Incremento de recorridos pasantes en Patricias Mendocinas:** aumento de $+9,3\%$ en la cuota de pares longitudinales respecto de la línea base histórica, para representar la función de drenaje norte-sur de esta vía comercial.

Ceros estructurales ($\mathcal{Z}_{\text{struct}}$): los pares OD alcanzables pero topológicamente frágiles (por ejemplo, $21 \rightarrow 17$, $0 \rightarrow 3$, $10 \rightarrow 5$, $21 \rightarrow 12$, $23 \rightarrow 12$, $26 \rightarrow 12$) reciben una probabilidad de asignación $p_{od} = 0,0$. Así, el remanente aleatorio no asigna vehículos a maniobras que puedan provocar bloqueo mutuo (*gridlock*) o activar el mecanismo *teleport* de SUMO por sobreflujo en accesos estrechos.

Remanente estocástico (40%) y selección de semilla: el remanente se distribuye entre los pares libres mediante `random_routes_taz.py`. Durante el diseño del escenario se evaluaron 18 semillas pseudoaleatorias, de las cuales 10 completaron la simulación continua de 12 horas sin colisiones ni activaciones del mecanismo *teleport*. Se seleccionó la semilla 104773, que presentó la menor desviación respecto de las cuotas objetivo, y se fijó la semilla de simulación de SUMO en 104729. Una vez validada, la demanda se

congeló en `mendoza_12h.rou.xml`; así, todas las comparaciones algorítmicas utilizan el mismo conjunto determinista de viajes.

Dinámica microscópica y heterogeneidad de flota: las inserciones se programan como procesos de Poisson. Para cada par origen-destino, si Q_{od} es el flujo en vehículos por hora, la tasa de arribos es $\lambda_{od} = \frac{Q_{od}}{3600}$ veh/s y cada intervalo se obtiene como $H_{od} = -\ln(U)/\lambda_{od}$, con $U \sim \mathcal{U}(0, 1)$. Así, el intervalo tiene unidad de segundos y valor esperado $\mathbb{E}[H_{od}] = 1/\lambda_{od} = 3600/Q_{od}$ s. En el archivo de rutas, `period="exp(rate)"` designa este muestreo exponencial; no significa calcular $e^{\lambda_{od}}$. La distribución de flota urbana heterogénea `td_urban_mix` está compuesta por 25% de vehículos lentos, 60% estándar y 15% dinámicos.

Protocolo de Validación Operacional y Criterio GEH

El escenario canónico v14 se validó bajo dos horizontes temporales. Sus métricas operacionales se resumen en la Tabla 3.2:

1. **Escenario base canónico (12 horas, 06 : 00–18 : 00):** horizonte experimental completo de 43.200 s, que reproduce la evolución continua del tránsito diurno.
2. **Ventana operacional (2 horas, 06 : 30–08 : 30):** recorte temporal de la hora pico ($t = [1800, 9000]$ s), utilizado para la exploración iterativa de controladores y las evaluaciones de alta demanda.

Tabla 3.2: Métricas operacionales de validación del escenario canónico v14 de Mendoza.

Ventana Temporal	Cargados	Insertados	Completados	CR	Inserción	Compleitud	teleport
2h (06 : 30–08 : 30)	30.629	23.735	22.702	0,7412	0,7749	0,9565	0
12h (06 : 00–18 : 00)	180.163	143.819	143.022	0,7938	0,7983	0,9945	0

Validación de calidad del flujo (estadístico GEH): conforme a las recomendaciones para la calibración de demanda en simulación microscópica [DLR26, PW16], la fidelidad de la inyección en el corredor José Vicente Zapata (TAZ 28) respecto de los aforos de la Cámara 169 se evaluó mediante el estadístico GEH:

$$\text{GEH} = \sqrt{\frac{2(V_{\text{mod}} - V_{\text{obs}})^2}{V_{\text{mod}} + V_{\text{obs}}}} \quad (3.8)$$

donde V_{mod} es el volumen horario modelado y V_{obs} el conteo horario empírico observado, ambos expresados en vehículos por hora. La ecuación (3.8) resume la discrepancia relativa entre ambos volúmenes en una sola magnitud adimensional. En todos los intervalos horarios de las 12 horas de simulación se obtuvieron valores $\text{GEH} \leq 0,121$. En la hora pico de las 08:00, el valor fue $\text{GEH} = 0,075$. Estos resultados cumplen el criterio estándar de aceptación ($\text{GEH} < 5,0$ en más del 85% de los puntos de aforo).

3.4 Arquitectura de Integración Entorno-Agentes

El marco computacional `marlTLC` integra el simulador SUMO con la librería Ray RLlib a través de la interfaz personalizada `SumoRllibEnv`. La especificación detallada de las clases y patrones de diseño orientados a objetos que sustentan esta arquitectura se expone en el Apéndice A.

3.4.1 Ciclo Episódico e Interfaz de Simulación

Para mejorar la recolección de muestras durante el entrenamiento distribuido, las simulaciones se ejecutan sin interfaz gráfica mediante la abstracción nativa de C++ `libsumo`. La integración se activa con la variable de entorno `LIBSUMO_AS_TRACI=1` y enlaza el simulador con el proceso Python. De este modo se evita la serialización y la comunicación por sockets TCP de la API TraCI estándar.

El ciclo de vida episódico opera bajo el siguiente protocolo de inicialización y control:

1. **Período de calentamiento (*warmup*):** cada episodio comienza con 600 segundos bajo control semafórico de tiempo fijo. Esta fase puebla la red y evita que los agentes aprendan sobre una red vacía.
2. **Almacenamiento y restauración de puntos de control (*checkpoint*):** al finalizar el calentamiento, el estado microscópico de la red se almacena en memoria o en un archivo XML (`state_pid...xml`). Cada invocación de `reset()` restaura este estado, de modo que el entrenamiento comienza en las mismas condiciones sin volver a simular el calentamiento. El tiempo de control `sim_time` transcurre en el intervalo $[t_{\text{warmup}}, t_{\text{warmup}} + \text{sim_time}]$.
3. **Paso de decisión y transiciones semafóricas:** las decisiones se ejecutan en pasos de $\Delta t = 5.0$ s. Cuando un agente cambia la fase activa, el entorno intercala una fase amarilla de `yellow_time = 4` s y exige un tiempo mínimo de verde de `min_green_time = 5` s antes de habilitar la nueva transición.

3.5 Instanciación del Modelo Dec-POMDP

La interacción distribuida entre las intersecciones semaforizadas se formaliza mediante la tupla Dec-POMDP instanciada en la librería `marlTLC`.

3.5.1 Espacio de Observaciones y Codificación Logarítmica

La observación local $o_i^{(t)}$ recibida por el agente i en el paso t combina la fase activa actual y el tiempo de espera acumulado por carril entrante (w_l), cuantizado mediante codificación logarítmica.

Para evitar que valores extremos de tiempo de espera distorsionen la escala del vector de entrada y provoquen la saturación de los gradientes en las redes neuronales, se apli-

ca la clase `Encoder` (`src/marlTLC/utils/encoder.py`). Esta transformación aplica una compresión logarítmica no lineal híbrida que proporciona alta resolución para demoras bajas y una pendiente lineal acotada para demoras elevadas:

$$e(x) = \text{ceil} \left(F \cdot \log_2 \left(\frac{x}{M_{\text{lane}} \cdot L} + 1 \right) + L^2 \cdot x \right) \quad (3.9)$$

donde $x \geq 0$ es el tiempo de espera acumulado del carril, expresado en segundos, $M_{\text{lane}} > 0$ es la cota de saturación de ese carril (también en segundos), $I \geq 2$ es el número de intervalos discretos y $L = (I-1)/M_{\text{lane}}$ y $F = (1-L)(I-1)/\log_2(1/L+1)$ son coeficientes calculados para esa cota. La ecuación (3.9) transforma la espera continua en un índice discreto. La función `ceil` redondea hacia arriba al entero más cercano. La implementación aplica además la saturación al intervalo $[0, I-1]$ después de evaluar esta expresión; por ello, el valor que recibe la red neuronal pertenece a ese rango discreto. La letra M_{lane} se utiliza aquí exclusivamente para el codificador y no debe confundirse con el factor de ventaja M de HAPPO.

En esta formulación:

- **Número de Intervalos (I):** Corresponde al hiperparámetro `encoder_intervals` (explorado en el rango $[4, 64]$), que define la resolución del espacio discreto y es optimizado automáticamente mediante exploración Bayesiana en Optuna.
- **Cota de saturación del carril (M_{lane}):** corresponde a `max_lane_value`, determinado empíricamente calculando el percentil 99 (P_{99}) de los tiempos de espera acumulados en simulaciones preliminares de cada red: $M_{\text{lane}} = 1184$ s para 3×1 , $M_{\text{lane}} = 1136$ s para 3×3 , $M_{\text{lane}} = 712$ s para $4 \times 4_H$ y $M_{\text{lane}} = 1494$ s para el microcentro de la Ciudad de Mendoza.

3.5.2 Espacio de Acciones y Enmascaramiento Dinámico

El espacio de acciones $\mathcal{A}_i = \{0, 1, \dots, K_i - 1\}$ representa el conjunto de fases verdes seleccionables por la intersección i . En cruces con geometrías asimétricas o restricciones de maniobra, la capa personalizada `ActionMaskingTorchRLModule` aplica un enmascaramiento dinámico sobre los *logits* $z_i(a)$ antes del cómputo de la distribución *softmax*. Los *logits* son puntuaciones reales sin normalizar producidas por la red; la función *softmax* las transforma en probabilidades cuya suma es uno:

$$\text{logit}_i(a) = \begin{cases} z_i(a) & \text{si la acción } a \text{ es válida y cumple } t_{\text{verde}} \geq \text{min_green_time} \\ -10^9 & \text{en caso contrario} \end{cases} \quad (3.10)$$

donde $a \in \mathcal{A}_i$ es una acción candidata (correspondiente a una fase semafórica), $z_i(a)$ es el valor producido por la red de política antes del enmascaramiento y t_{verde} es el tiempo

transcurrido desde el inicio del verde actual. La ecuación (3.10) modifica cada valor antes de aplicar *softmax*. El valor -10^9 es una aproximación numérica a $-\infty$: al aplicar la función *softmax*, la probabilidad resultante es prácticamente cero, aunque no se trata de una probabilidad exactamente nula en aritmética de precisión finita. Así, el enmascaramiento impide seleccionar acciones incompatibles con las restricciones operacionales.

3.5.3 Función de Recompensa Compuesta y Coordinación Vecinal

La función de recompensa representa el compromiso entre el desempeño operacional local y la cooperación distribuida. No se agregan términos heurísticos adicionales.

1. **Recompensa local diferencial (L_i):** cuantifica la variación del tiempo de espera acumulado entre pasos consecutivos. En esta sección, $W_{\text{acc},i}(t)$ representa la suma de los tiempos de espera de los vehículos presentes en los carriles de entrada controlados por la intersección i en el paso t , expresada en vehículo-segundo. La ecuación (3.11) asigna una señal positiva cuando la espera disminuye:

$$L_i^{(t)} = W_{\text{acc},i}(t-1) - W_{\text{acc},i}(t) \quad (3.11)$$

2. **Recompensa compartida vecinal (N_i):** corresponde al promedio de las señales locales obtenidas por las intersecciones del vecindario topológico directo $\mathcal{N}(i)$. La ecuación (3.12) supone $|\mathcal{N}(i)| > 0$:

$$N_i^{(t)} = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} L_j^{(t)} \quad (3.12)$$

3. **Mezcla convexa normalizada:** se adopta una combinación convexa gobernada por el hiperparámetro $\rho_{\text{coop}} \in [0, 0.95]$ para desacoplar la escala de la recompensa del grado de cooperación. La ecuación (3.13) combina ambas señales sin cambiar sus unidades:

$$R_i^{(t)} = (1 - \rho_{\text{coop}})L_i^{(t)} + \rho_{\text{coop}}N_i^{(t)} \quad \text{con} \quad \rho_{\text{coop}} \in [0, 0.95] \quad (3.13)$$

El valor $\rho_{\text{coop}} = 0$ representa el ancla puramente local. Esta formulación acota la escala de la señal y permite controlar explícitamente el grado de cooperación; no garantiza por sí sola preservar el ordenamiento de recompensas ni las trayectorias de valor del componente local.

3.6 Implementación Algorítmica e Hiperparámetros

Los cuatro algoritmos evaluados, los esquemas independientes IDQN e IPPO y las arquitecturas de entrenamiento centralizado MAPPO y HAPPO, se implementaron en PyTorch y se integraron en la API de Ray RLlib mediante el módulo de fábrica `algo_factory.py`.

3.6.1 Mecanismos de Aprendizaje Independiente y Centralizado

- **IDQN e IPPO (Aprendizaje Independiente):** Cada intersección aprende una política local $\pi_{\theta_i}(a_i|o_i)$ tratando a los demás controladores como parte del entorno no estacionario. En IPPO, cada agente cuenta con su propia red de valor local $V_{\phi_i}(o_i)$.
- **MAPPO (PPO Multi-Agente Centralizado [YVV⁺22]):** Emplea el paradigma CTDE. Durante la ejecución, cada actor selecciona acciones de forma descentralizada a partir de su observación local o_i . Durante el entrenamiento, un crítico centralizado compartido (`SharedCriticTorchRLModule`) evalúa el estado global concatenado $s = (o_1, o_2, \dots, o_N)$ para estimar $V_\phi(s)$ y computar las ventajas GAE (λ_{GAE}).
- **HAPPO (PPO para Agentes Heterogéneos [KCW⁺22]):** Extiende MAPPO al aprendizaje en agentes con espacios de acción o roles heterogéneos mediante el **Lema de Descomposición de Ventajas Multi-Agente**. En cada *epoch* de entrenamiento, los agentes se actualizan de forma secuencial según un orden $(i_1, i_2, \dots, i_N)$, donde N es el número total de agentes y i_m es el agente ubicado en la posición m de ese orden. Si $\bar{A}_i$ denota la ventaja media del agente i en la muestra, el hiperparámetro p_{greedy} determina cómo se construye el orden: 0.0 produce un orden aleatorio; 1.0, un orden codicioso según la magnitud de ventaja $|\bar{A}_i|$; y un valor en $(0, 1)$, un orden semi-codicioso. Si k identifica la política antes de actualizar el agente i_m y $k+1$ la política inmediatamente posterior a esa actualización, la razón posterior a la optimización para una observación o y una acción a de la muestra es:

$$\rho_{\text{post}}^{(i_m)} = \frac{\pi_{\theta_{k+1}}^{(i_m)}(a|o)}{\pi_{\theta_k}^{(i_m)}(a|o)} \quad (3.14)$$

La ecuación (3.14) compara la política posterior con la política utilizada para recolectar la muestra. En ella, $\pi_{\theta_k}^{(i_m)}(a|o)$ y $\pi_{\theta_{k+1}}^{(i_m)}(a|o)$ son, respectivamente, las probabilidades que asigna el actor del agente i_m antes y después de su actualización; θ_k y θ_{k+1} son los parámetros correspondientes; y $\rho_{\text{post}}^{(i_m)}$ cuantifica el cambio relativo de la probabilidad de la acción observada. Para cada agente aún no actualizado i_j , con $j > m$, $M^{(i_j)}$ es el factor de ventaja reponderada acumulado hasta ese momento, inicializado con la ventaja conjunta estimada $\hat{A}(s, \mathbf{a})$ antes de la primera actualización,

y se actualiza como

$$M^{(i_j)} \leftarrow M^{(i_j)} \cdot \rho_{\text{post}}^{(i_m)}, \quad j > m. \quad (3.15)$$

La ecuación (3.15) incorpora al factor la razón de cada actor ya optimizado. La actualización de i_1 afecta a todos los agentes restantes, la de i_2 a los que siguen en el orden, y así sucesivamente. Esta propagación implementa el ajuste secuencial de HAPPO; la garantía de mejora monótona corresponde al procedimiento teórico de región de confianza bajo sus supuestos explícitos y no se transfiere automáticamente a HAPPO-clip.

En los cuatro algoritmos, el entrenamiento consume observaciones del tránsito, fases ejecutadas, recompensas y observaciones siguientes para ajustar los parámetros de las redes. IDQN conserva transiciones anteriores en un *replay buffer*; las variantes de PPO recolectan un *rollout* con la política vigente y lo reutilizan durante una cantidad acotada de *epochs*. Cada *epoch* recorre el conjunto recolectado y cada *minibatch* es el subconjunto usado en una actualización. El optimizador, los búferes y los críticos pertenecen al proceso de aprendizaje: durante la ejecución del control, cada intersección solo necesita su observación local y la red entrenada que asigna valores o probabilidades a las fases admisibles. Por tanto, la salida del entrenamiento es una política parametrizada, no un plan semafórico fijo.

3.7 Búsqueda de Hiperparámetros (HPO)

La exploración de hiperparámetros se realizó con Optuna [ASY⁺19], Ray Tune y el algoritmo de poda asíncrona ASHA [LJR⁺20]. El procedimiento tuvo dos etapas:

1. **Etapla 1 (exploración bayesiana amplia):** Muestreo mediante TPE sobre un presupuesto de 100 pruebas por combinación (ampliable a 150), evaluando las arquitecturas compacta (h128: [128, 128]) y ancha (h256: [256, 256]) como estudios independientes de Ray Tune para evitar objetos no resueltos en `model_config`. El espacio de búsqueda comprende 13 dimensiones para MAPPO y 14 para HAPPO:

- $\rho_{\text{coop}} \in [0.0, 0.95]$ (**uniform**)
- $\alpha_{\text{lr}} \in [3 \times 10^{-5}, 10^{-3}]$ (**loguniform**)
- $\epsilon_{\text{clip}} \in [0.10, 0.22]$ (**uniform**)
- $\lambda_{\text{GAE}} \in [0.88, 0.99]$ (**uniform**)
- $c_{\text{entropy}} \in [0.0, 0.01]$ (**uniform**)
- $\text{grad_clip} \in [0.25, 5.0]$ (**loguniform**)
- $c_{\text{KL}} \in [0.05, 0.5]$ (**loguniform**)
- $d_{\text{KL_target}} \in [0.005, 0.03]$ (**loguniform**)

- $\text{vf_clip} \in [5.0, 20.0]$ (`loguniform`)
- $K \in [5, 15]$ MAPPO / $[5, 18]$ HAPPO (`randint`)
- $N_{\text{episodes}} \in [2, 12]$ (`randint`)
- $I \in [4, 64]$ (`randint`)
- $B_{\text{mini}} \in \{128, 256, 512\}$ (`choice`)
- $p_{\text{greedy}} \in [0.20, 0.90]$ (solo HAPPO, `uniform`)

Parámetros fijos: $\gamma = 0.99$, $\text{lane_metric} = \text{acc_waiting_time}$, $\text{use_kl_loss} = \text{True}$. Semillas HPO de entrenamiento: 101 (3×1), 137 (3×3), 179 ($4 \times 4_H$); semillas de evaluación: 100101, 100137, 100179. La semilla histórica 42 queda prohibida.

2. **Etapa 2 (confirmación pareada):** Reentrenamiento a presupuesto completo (40 iteraciones) sobre 10 semillas pareadas independientes de entrenamiento (211, 223, 227, 229, 233, 239, 241, 251, 257, 263) y 10 semillas de evaluación (200211, ..., 200263), comparando el ρ_{coop} óptimo congelado frente a la línea base puramente local ($\rho_{\text{coop}} = 0$).

Para evitar seleccionar políticas que, pese a presentar una espera global media baja, incurran en episodios de bloqueo total de la red (*gridlock*), la función objetivo incorpora una penalización robusta al riesgo mediante CVaR. Sea X la variable aleatoria que representa un incremento positivo de la espera global entre dos iteraciones consecutivas; en particular, para la iteración r , $X_r = \max\{0, W_{\text{global}}^{(r)} - W_{\text{global}}^{(r-1)}\}$. La colección de realizaciones $\{X_r\}$ corresponde a `positive_interiteration_increases`. Para la cola superior de probabilidad α_{CVaR} , con $0 < \alpha_{\text{CVaR}} < 1$, se define:

$$\text{CVaR}_{\alpha_{\text{CVaR}}}(X) = \inf_{\eta \in \mathbb{R}} \left\{ \eta + \frac{1}{\alpha_{\text{CVaR}}} \mathbb{E}[(X - \eta)_+] \right\}, \quad (z)_+ = \max\{z, 0\}. \quad (3.16)$$

La ecuación (3.16) define la penalización de cola. Aquí α_{CVaR} es la fracción de resultados extremos que se penaliza, η es un umbral de pérdida que delimita esa cola, $\mathbb{E}$ representa la esperanza matemática y $(z)_+$ conserva solo la parte positiva. Por consiguiente, $\text{CVaR}_{0.2}(X)$ es el promedio de los incrementos más desfavorables que componen el 20% superior de la distribución. En esta expresión, $W_{\text{global}}^{(r)}$ es la espera global de la iteración r , calculada con las métricas físicas de SUMO obtenidas de `TripInfo`, y $\bar{W}_{\text{global}}$ es su promedio en la ventana de evaluación; ambas magnitudes se expresan en vehículo·segundo.

$$\text{stable_global_waiting} = \bar{W}_{\text{global}} + 1.0 \cdot \text{CVaR}_{0.2}(\text{positive_interiteration_increases}) \quad (3.17)$$

La ecuación (3.17) se calcula sobre las últimas 10 iteraciones de entrenamiento (tras 5 iteraciones de calentamiento). La primera parte prioriza una espera media baja, mientras que el término CVaR penaliza el comportamiento desfavorable de cola: un aumento

grande de W_{global} refleja la formación o el agravamiento de colas y demoras, incluso si el promedio no resulta elevado. La ventana de 10 iteraciones reduce la dependencia de una sola medición y permite evaluar la estabilidad reciente; las 5 iteraciones iniciales de calentamiento se excluyen para que la métrica no mezcle la transitoriedad de la puesta en marcha con el comportamiento de aprendizaje. El coeficiente 1.0 es adimensional y conserva la escala de la penalización respecto de la espera global.

3.7.1 Infraestructura de Cómputo y Orquestación

Dado el tamaño del espacio de búsqueda y el costo computacional de la simulación microscópica en SUMO, la exploración y el entrenamiento final se desplegaron en los clústeres de cómputo **Toko** (Centro de Cómputo de Alto Rendimiento (CCAR) - Facultad de Ciencias Exactas, Físicas y Naturales (FCEF) / UNCuyo / Consejo Nacional de Investigaciones Científicas y Técnicas (CONICET)) y **Mendieta** (Centro de Cómputo de Alto Desempeño (CCAD) - Universidad Nacional de Córdoba).

La ejecución distribuida se gestionó mediante el sistema de colas SLURM. Se aplicaron las siguientes medidas:

- **Aislamiento de procesos Ray en un nodo:** configuración de instancias Ray de nodo único (`setup_ray_single_node`) con asignación dinámica de puertos basada en el identificador de trabajo:

$$\text{ray_port} = 20000 + (\text{SLURM_JOB_ID} \pmod{20000}) \quad (3.18)$$

La ecuación (3.18) transforma el identificador del trabajo en un puerto dentro de un intervalo acotado. En ella, `SLURM_JOB_ID` es el identificador entero del trabajo y $(\text{mod } 20000)$ devuelve su resto al dividirlo por 20.000. Por tanto, el puerto calculado pertenece al intervalo entero $[20000, 39999]$; la fórmula reduce la probabilidad de colisión entre trabajos concurrentes, pero no constituye una reserva de puertos del sistema. Se utiliza junto con la autodetección de dirección IP mediante `hostname -ip-address`.

- **Gestión de E/S en almacenamiento local (/scratch):** para evitar saturar el sistema de archivos distribuido en red (Sistema de archivos de red (*Network File System*) (NFS)) con la telemetría microscópica de SUMO (`tripinfo.xml`, `summary.xml`), las simulaciones y la recolección de trayectorias se ejecutaron en el almacenamiento local de cada nodo (`/scratch/$USER/ray_tmp/`).
- **Persistencia ante preempción:** los scripts de envío incorporan trampas de señales POSIX (`trap ... EXIT USR1 TERM INT`). Estas trampas sincronizan mediante `rsync` los resultados y las bases de datos SQLite

de Optuna con el almacenamiento persistente del usuario (\$HOME), en el directorio `ray\_results/`. Esto permite reanudar los trabajos mediante `Tuner.restore()`.

3.8 Evaluación de Rendimiento

El protocolo de evaluación final de los modelos entrenados se definió mediante los siguientes criterios:

1. **Evaluación en semillas OOD:** para verificar la generalización y descartar sobreajuste a secuencias de demanda particulares, las políticas finales se evaluaron con la configuración `final_evaluation` y las semillas 300101, 300137, 300179, 300191 y 300193.
2. **Pruebas no paramétricas y tamaño del efecto:** las comparaciones de demora acumulada W_{net} entre algoritmos se evaluaron mediante la prueba de rangos con signo de Wilcoxon, con nivel de significancia $\alpha_{\text{sig}} = 0.05$. También se calcularon el estimador $\hat{A}_{12}$ de Vargha-Delaney y la Media intercuartílica (*Interquartile Mean*) (IQM), con intervalos de confianza del 95% obtenidos mediante *bootstrap*, es decir, remuestreo con reemplazo de las ejecuciones disponibles.
3. **Criterios de descarte:** se descartaron las políticas candidatas que redujeran los vehículos completados (throughput) en más del 5%, incrementaran las activaciones de *teleport* por colapso de carril o degradaran la equidad de demoras (G_{wait}) en más del 10% respecto del control de tiempo fijo.

Análisis y discusión de resultados

En este capítulo se presentan y analizan los resultados de las ejecuciones de entrenamiento y evaluación de los algoritmos estudiados. Con el objetivo de distinguir con claridad las etapas metodológicas del entrenamiento de los agentes, la sección se organiza en tres partes.

En primer lugar, se reportan los resultados del proceso de ajuste de hiperparámetros. En segundo lugar, se evalúan los modelos finales seleccionados en los escenarios planteados, desde redes sintéticas hasta el escenario ubicado en Ciudad de Mendoza. Finalmente, se desarrolla una discusión integradora de los hallazgos, sus implicancias y las principales limitaciones del estudio.

4.1 Resultados del ajuste de hiperparámetros

En esta sección se resume el proceso de ajuste de hiperparámetros y se justifica la configuración utilizada en la evaluación posterior de los modelos.

4.1.1 Diseño del proceso de ajuste

4.1.2 Resultados del ajuste y análisis de sensibilidad

4.1.3 Configuración seleccionada para la evaluación final

4.2 Resultados de los modelos finales

Esta sección reúne los resultados obtenidos con los modelos finales una vez fijada la configuración experimental. La exposición se organiza según el tipo de escenario analizado.

4.2.1 Criterios de evaluación y métricas de comparación

4.2.2 Desempeño en escenarios sintéticos: 3×1 , 3×3 , 4×4 heterogéneo

4.2.3 Desempeño en la red de carreteras de la Ciudad de Mendoza

4.3 Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo.

5

Conclusiones

Este capítulo sintetiza los hallazgos del estudio, relaciona la evidencia experimental con los objetivos planteados y delimita el alcance de las conclusiones. También identifica las limitaciones del modelo y propone líneas de trabajo futuro. Las afirmaciones finales deben interpretarse dentro de los escenarios, métricas y protocolos descritos en la metodología.



Arquitectura de Software y Patrones de Diseño

En este apéndice se documenta la especificación arquitectónica del marco computacional `marlTLC`, su organización orientada a objetos, los diagramas de clases UML y los patrones de diseño previstos para desacoplar los componentes de simulación, aprendizaje por refuerzo y cálculo de recompensas. El repositorio que acompaña a este documento no contiene el código fuente completo del marco; por ello, las clases, métodos y relaciones que se presentan deben interpretarse como una especificación de diseño y no como una auditoría de archivos ejecutables.

A.1 Diseño Orientado a Objetos del Entorno de Simulación

El diseño del entorno separa las responsabilidades de sus componentes. En la especificación, la clase abstracta `BaseSumoEnvironment` encapsula la inicialización de SUMO, el control temporal y la interacción de bajo nivel con la API `libsumo` / `traci`. A partir de ella, `SumoRllibEnv` se define como el adaptador de la simulación a la interfaz `MultiAgentEnv` de Ray `RLLib`.

La Figura A.1 ilustra el diagrama de clases UML correspondiente a las entidades del entorno y su relación con los controladores semafóricos. Un diagrama de clases representa los tipos de objetos que componen un programa. Cada recuadro se divide en tres partes: el nombre de la clase, sus atributos o datos almacenados y sus métodos u operaciones. El signo “-” indica un miembro de uso interno o privado; “+”, uno público; y “#”, uno

protegido, destinado a la propia clase y a sus derivadas. El texto situado después de los dos puntos indica el tipo de dato esperado. Los nombres en cursiva corresponden a clases u operaciones abstractas: definen una interfaz que debe concretarse en otra clase. Una flecha con la leyenda “hereda de” apunta desde la clase especializada hacia la clase general, mientras que la multiplicidad 1..* significa “uno o más” objetos relacionados.

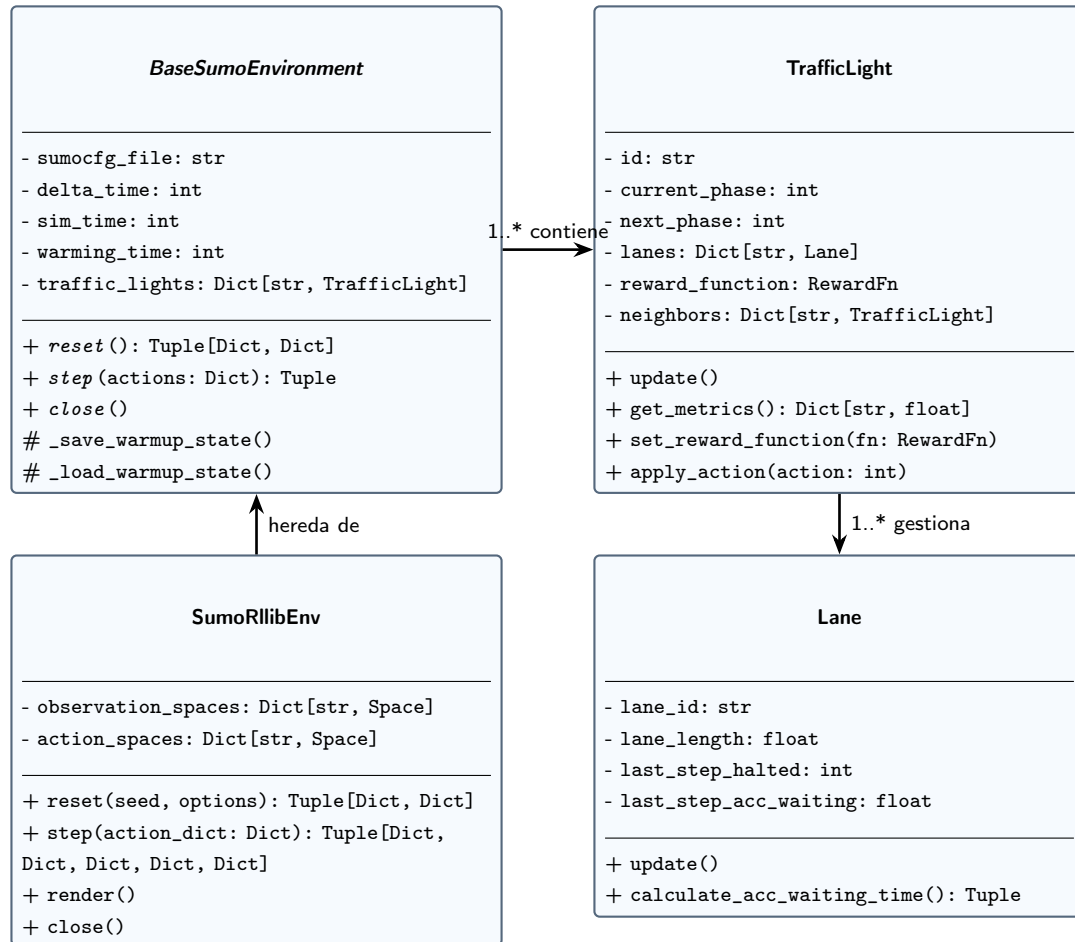


Figura A.1: Diagrama de clases UML del entorno. **SumoRllibEnv** hereda la interfaz de **BaseSumoEnvironment**; el entorno contiene uno o más objetos **TrafficLight**, y cada controlador semafórico gestiona uno o más objetos **Lane**. En cada clase se muestran sus principales atributos y métodos.

La Figura A.1 se organiza desde la abstracción general hacia las entidades de la red. En la columna izquierda, **BaseSumoEnvironment** concentra el archivo de configuración, los parámetros temporales, los controladores y las operaciones generales de reinicio, avance y cierre. Debajo, **SumoRllibEnv** especializa esa clase para recibir un diccionario de acciones de los agentes y devolver las observaciones, recompensas y estados definidos por la interfaz de Ray Rllib. A la derecha, el diseño asigna a **TrafficLight** la fase actual, la siguiente fase, sus carriles, su función de recompensa y sus vecinos; también le asigna la actualización de métricas y la aplicación de acciones. Finalmente, **Lane** representa la longitud y las medidas recientes de detención y espera. Las flechas muestran las dependencias previstas

entre la interfaz de aprendizaje, el entorno, los controladores y sus carriles.

A.2 Jerarquía de Funciones de Recompensa y Patrón Strategy

En la especificación, el cálculo de la recompensa se desacopla mediante el patrón **Strategy**. Esta organización permite sustituir la métrica de optimización sin modificar el bucle principal de simulación.

La clase base abstracta **RewardFn** define la interfaz común `compute_reward(tl_id)`. En el diseño se contemplan formulaciones locales basadas en tres señales: demora acumulada, longitud de cola y presión de red. La coordinación distribuida se representa mediante el patrón **Composite**: una clase de recompensa compuesta combina la señal local y la señal promedio de los semáforos adyacentes.

La Figura A.2 detalla las clases que intervienen directamente en la recompensa cooperativa. Las demás estrategias locales registradas en el sistema no se dibujan para conservar la legibilidad. Se mantienen las convenciones explicadas para la Figura A.1; en este segundo diagrama, la flecha discontinua rotulada “compone” indica que una clase utiliza otra como parte de su cálculo, no que herede sus atributos y métodos.

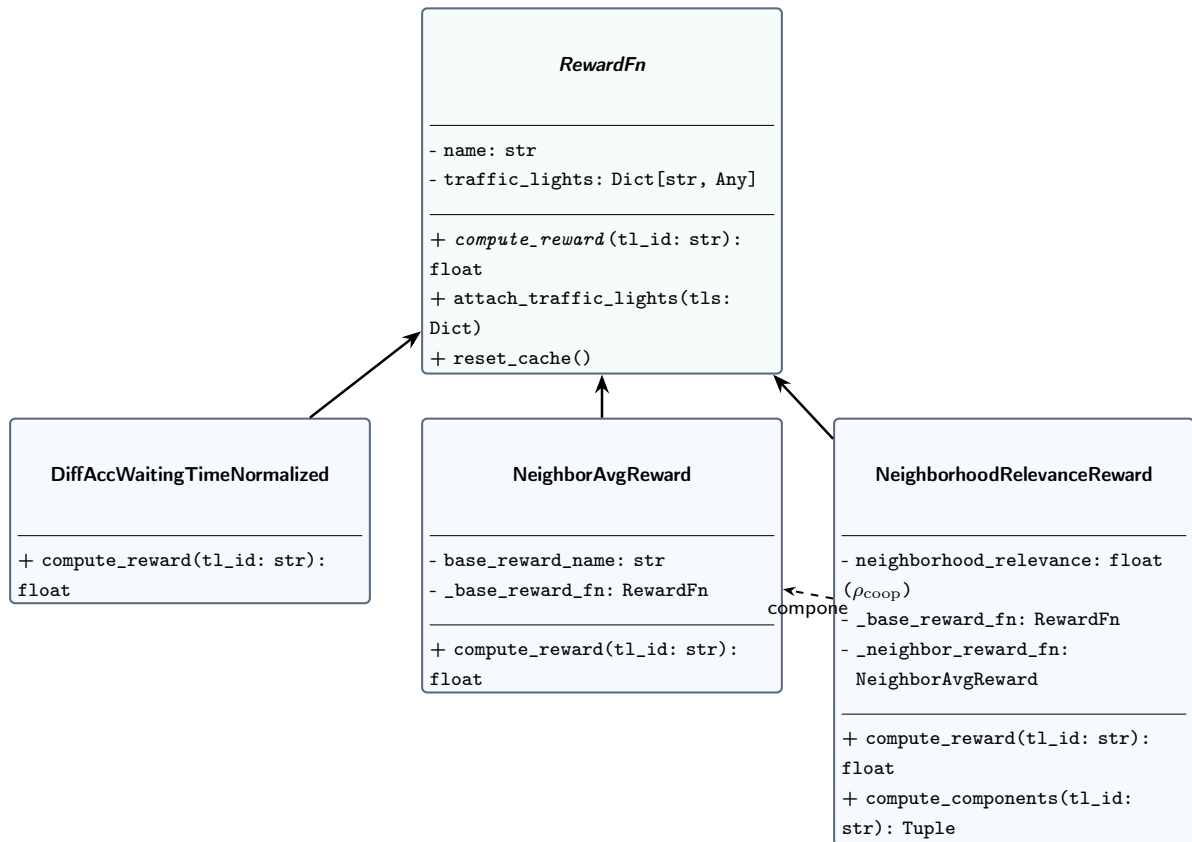


Figura A.2: Especificación UML del módulo de recompensas. Las clases inferiores se definen como realizaciones de **RewardFn**; la clase compuesta incorpora la señal promedio de los controladores vecinos.

La Figura A.2 sitúa a `RewardFn` en la parte superior porque define la operación común `compute_reward`, que recibe el identificador de un controlador semafórico y devuelve un valor numérico. Las tres flechas ascendentes indican que las clases inferiores se especializan para proporcionar variantes de esa operación. `DiffAccWaitingTimeNormalized` representa una señal local normalizada a partir de la variación del tiempo de espera acumulado, mientras que `NeighborAvgReward` promedia la señal base de los controladores vecinos. `NeighborhoodRelevanceReward` conserva ambas funciones y las combina según ρ_{coop} , el peso de cooperación vecinal. La flecha discontinua entre las dos últimas clases muestra esa relación de composición. Esta separación permite cambiar la formulación de la recompensa sin modificar el bucle que avanza la simulación.

A.3 Patrones de Diseño Implementados

A continuación se resumen los patrones contemplados en la especificación de `marlTLC`. La lista describe responsabilidades y relaciones de diseño; no certifica la presencia de una implementación ejecutable de cada componente en el repositorio.

1. Patrón Strategy (Estrategia):

- **Propósito:** encapsular los algoritmos de recompensa para intercambiarlos sin modificar los controladores semafóricos.
- **Rol en el diseño:** interfaz base `RewardFn` y variantes concretas de recompensa.

2. Patrón Factory Method y Registry (Fábrica con Registro Dinámico):

- **Propósito:** instanciar funciones de recompensa y módulos neuronales a partir de cadenas de configuración o especificaciones compuestas (`RewardSpec`). También permite registrar nuevas funciones sin modificar la fábrica, de acuerdo con el principio abierto/cerrado.
- **Rol en el diseño:** registro mediante `RewardFactory` y `@register_reward`, que identifica el mecanismo previsto para asociar nombres de configuración con funciones de recompensa.

3. Patrón Composite (Objeto Compuesto):

- **Propósito:** combinar recompensas locales y vecinales mediante una interfaz uniforme.
- **Rol en el diseño:** clases de recompensa compuesta que agregan señales locales y vecinales.

4. Patrón Protocol / Structural Subtyping (Tipado Estructural):

- **Propósito:** desacoplar el proveedor de métricas vecinales del tipo concreto `TrafficLight`, facilitando las pruebas unitarias con objetos simulados (*mocks*) y evitando dependencias circulares.
- **Rol en el diseño:** protocolo `NeighborInfoProvider` para describir la información vecinal requerida sin depender de una clase concreta. El detalle del decorador de comprobación en tiempo de ejecución no forma parte de la evidencia disponible en este repositorio.

5. Patrón Adapter (Adaptador):

- **Propósito:** traducir la API procedural y basada en comandos de TraCI / libsumo a la interfaz orientada a objetos de `MultiAgentEnv` (Ray RLlib).
- **Rol en el diseño:** `SumoRllibEnv` como punto de adaptación entre el simulador y la interfaz de aprendizaje.

6. Patrón Decorator (Decorador):

- **Propósito:** extender dinámicamente las capacidades de los módulos sin alterar su jerarquía de herencia.
- **Rol en el diseño:** decoradores de registro y una envoltura para los módulos de acción en PyTorch; los nombres concretos de esas piezas se mantienen como identificadores de la especificación.

Bibliografía

- [ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. <https://www.marl-book.com/>, Consultado en 2025.
- [AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.
- [AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the Third Workshop on Data-driven Intelligent Transportation, co-located with CIKM '22*, 2022. <https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf>, Fuente: [7, 8] Destaca el "éxito limitado" de SARL y los enfoques independientes en redes.
- [Aso18] Asociación de Fábricas Argentinas de Componentes: *Flota circulante en Argentina 2017*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, mayo 2018. <https://cdn.motor1.com/pdf-files/afac-informe-parque-circulante-2017pdf.pdf>, Informe elaborado con datos de AFAC y Promotive.
- [Aso26] Asociación de Fábricas Argentinas de Componentes: *Flota vehicular circulante en Argentina*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, 2026. https://www.automotrix.com.ar/adm/enviosmasivos/Plantillas/PRENSA/2026/AFAC_PRENSA_FLOTACIRCULANTE_2025.pdf, Datos al cierre de 2025; informe elaborado en el marco del convenio entre AFAC y Promotive S.A.
- [ASY⁺19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takuya Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.
- [BB24] Bishop, Christopher M. y Hugh Bishop: *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.

- [BV02] Bowling, Michael y Manuela Veloso: *Multiagent learning using a variable learning rate*. Artificial Intelligence, 136(2):215–250, 2002.
- [CYL⁺24] Chen, Wubing, Shangdong Yang, Wenbin Li, Yujing Hu, Xiao Liu y Yang Gao: *Learning Multi-Intersection Traffic Signal Control via Coevolutionary Multi-Agent Reinforcement Learning*. IEEE Transactions on Intelligent Transportation Systems, 25(11):15947–15963, 2024.
- [CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.
- [DLR26] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2026. <https://sumo.dlr.de/docs/>, Documentación web consultada en 2026.
- [DLSZ26] Deng, Maojiang, Shoufeng Lu, Jiazhao Shi y Wen Zhang: *Adaptive Traffic Signal Control Optimization Using a Novel Road Partition and Multi-Channel State Representation Method*. Urban Lifeline, 4(1):9, 2026.
- [dWGM⁺20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr y cols.: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.
- [FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.
- [Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Capítulo 4: Traffic Signal Design. Disponible en: <https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter4.htm>.
- [FNF⁺17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.
- [FYX⁺23] Fang, Jie, Ya You, Mengyun Xu, Juanmeizi Wang y Sibin Cai: *Multi-Objective Traffic Signal Control Using Network-Wide Agent Coordinated Reinforcement Learning*. Expert Systems with Applications, 229:120535, 2023.

- [GBC16] Goodfellow, Ian, Yoshua Bengio y Aaron Courville: *Deep Learning*. MIT Press, Cambridge, MA, 2016. <https://www.deeplearningbook.org/>.
- [HHA22] Haddad, Tarek Amine, Djalal Hedjazi y Sofiane Aouag: *A Deep Reinforcement Learning-Based Cooperative Approach for Multi-Intersection Traffic Signal Control*. Engineering Applications of Artificial Intelligence, 114:105019, 2022.
- [KCW⁺22] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fuchun Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. En *International Conference on Learning Representations (ICLR)*, 2022. <https://arxiv.org/abs/2109.11251>.
- [KKBA23] Kolat, Máté, Bálint Kővári, Tamás Bécsi y Szilárd Aradi: *Multi-Agent Reinforcement Learning for Traffic Signal Control: A Cooperative Approach*. Sustainability, 15(4):3479, 2023.
- [KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.
- [KT03] Konda, Vijay R. y John N. Tsitsiklis: *On Actor-Critic Algorithms*. SIAM Journal on Control and Optimization, 42(4):1143–1166, 2003.
- [LFYZ25] Lu, Qiong, Haoda Fang, Zhangcheng Yin y Guliang Zhu: *HAPS-PPO: A Multi-Agent Reinforcement Learning Architecture for Coordinated Regional Control of Traffic Signals in Heterogeneous Road Networks*. Applied Sciences, 15(20):10945, 2025.
- [LJR⁺20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-tzvi, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning*. En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.
- [LLN⁺18] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstractions for distributed reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.
- [LML⁺25] Liang, Jiaxin, Haotian Miao, Kai Li, Jianheng Tan, Xi Wang, Rui Luo y Yueqiu Jiang: *A Review of Multi-Agent Reinforcement Learning Algorithms*. Electronics, 14(4):820, 2025. <https://doi.org/10.3390/electronics14040820>.

- [LWT⁺17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.
- [LZF⁺25] Liu, Wanting, Chengwei Zhang, Wanqing Fang, Kailing Zhou, Yihong Li, Furui Zhan, Qi Wang, Wanli Xue y Rong Chen: *Vehicle-Level Fairness-Oriented Constrained Multi-Agent Reinforcement Learning for Adaptive Traffic Signal Control*. *IEEE Transactions on Intelligent Transportation Systems*, 26(4):4878–4890, 2025.
- [MKS⁺15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning*. *Nature*, 518(7540):529–533, 2015. <https://doi.org/10.1038/nature14236>.
- [MMLK25] Michailidis, Panagiotis, Iakovos Michailidis, Charalampos Rafail Lazaridis y Elias Kosmatopoulos: *Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations*. *Infrastructures*, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.
- [PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.
- [RDZ⁺24] Ren, Fuyue, Wei Dong, Xiaodong Zhao, Fan Zhang, Yaguang Kong y Qiang Yang: *Two-Layer Coordinated Reinforcement Learning for Traffic Signal Control in Traffic Network*. *Expert Systems with Applications*, 235:121111, 2024.
- [RSDW⁺18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, páginas 4295–4304. PMLR, 2018.
- [SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction*. The MIT Press, Cambridge, MA, 2nd edición, 2018.
- [Sha53] Shapley, Lloyd S: *Stochastic games*. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.

- [SLA⁺15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization*. En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.
- [SLG⁺18] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls y Thore Graepel: *Value-Decomposition Networks For Cooperative Multi-Agent Learning*. En *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, páginas 2085–2087. International Foundation for Autonomous Agents and Multiagent Systems, 2018.
- [SML⁺16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation*. En *International Conference on Learning Representations (ICLR)*, 2016.
- [SWD⁺17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.
- [TMK⁺17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning*. PloS one, 12(4):e0172395, 2017.
- [W⁺19] Wei, H. y cols.: *A survey on traffic signal control methods*, 2019. Available at: <https://arxiv.org/abs/1904.08117>.
- [Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.
- [WWS⁺22] Wu, Qiang, Jianqing Wu, Jun Shen, Bo Du, Akbar Telikani, Mahdi Fahmideh y Chao Liang: *Distributed Agent-Based Deep Reinforcement Learning for Large Scale Traffic Signal Control*. Knowledge-Based Systems, 241:108304, 2022.
- [Yan23] Yang, Shantian: *Hierarchical Graph Multi-Agent Reinforcement Learning for Traffic Signal Control*. Information Sciences, 634:55–72, 2023.
- [YVV⁺22] Yu, Chao, Akash Velu, Eugene Vinitisky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.

- [ZKF⁺24] Zhong, Yifan, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji y Yaodong Yang: *Heterogeneous-Agent Reinforcement Learning*. Journal of Machine Learning Research, 25:1–67, 2024. <https://www.jmlr.org/papers/v25/23-0488.html>.
- [ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.